

3X-UI Panel — User Manual

 [Русская версия](#)

3X-UI version: 3.3.1. This manual is written for and applies to that version. A summary of 3.3.1 changes relative to 3.3.0 is in the ["What's new in 3.3.1"](#) section.

A detailed user manual for the **3X-UI** web panel (an Xray-core manager): features, configuration, and operation, with every field and toggle explained.

Field names and labels match the panel's interface.

Contents

- What's new in 3.3.1
- 1. Introduction, requirements, and installation
 - 1.1. What 3X-UI is
 - 1.2. Supported operating systems and architectures
 - 1.3. Installation methods
 - 1.4. First launch and default credentials
 - 1.5. File locations
 - 1.6. The `x-ui` management command (script menu)
 - 1.7. `x-ui` subcommands (without the interactive menu)
 - 1.8. SQLite → PostgreSQL migration
- 2. Panel login and access security
 - 2.1. Login form
 - 2.2. Two-factor authentication (2FA / TOTP)
 - 2.3. Login attempt limiting (login limiter / brute-force protection)
 - 2.4. Changing the administrator login and password
 - 2.5. Secret path (URI path / webBasePath) and panel port
 - 2.6. Session lifetime (timeout)
 - 2.7. LDAP (synchronization and authentication)
- 3. Overview / Dashboard
 - 3.1. General principles of data collection
 - 3.2. CPU
 - 3.3. Memory (RAM)
 - 3.4. Swap
 - 3.5. Storage (Disk)
 - 3.6. System uptime
 - 3.7. System load (Load average)
 - 3.8. Network: speed and total traffic volume
 - 3.9. Server IP addresses
 - 3.10. TCP/UDP connections
 - 3.11. Xray status and process control
 - 3.12. Panel update (3X-UI)
 - 3.13. Updating geo files (GeoIP / GeoSite)
 - 3.14. Database backup and restore
 - 3.15. Additional interface elements
- 4. Inbounds: creation and common parameters
 - 4.1. Common form fields
 - 4.2. Sniffing
 - 4.3. Allocate (port allocation strategy)
 - 4.4. External Proxy
 - 4.5. Fallbacks
 - 4.6. Periodic traffic reset
 - 4.7. Inbound JSON (advanced)
 - 4.8. Inbound actions: QR / Edit / Reset / Delete and statistics

- 5. Protocols
 - 5.1. List of supported protocols
 - 5.2. Which protocols support TLS / REALITY / transport
 - 5.3. VLESS
 - 5.4. VMess
 - 5.5. Trojan
 - 5.6. Shadowsocks
 - 5.7. Dokodemo-door / Tunnel (transparent forwarder)
 - 5.8. SOCKS / HTTP (the `mixed` protocol)
 - 5.9. WireGuard (inbound)
 - 5.10. Hysteria (v2 by default)
 - 5.11. MTPROTO (Telegram proxy)
 - 5.12. Quick cheat sheet for choosing a protocol
- 6. Transport (Stream Settings)
 - 6.1. Choosing the transmission network
 - 6.2. RAW / TCP (`tcpSettings`)
 - 6.3. mKCP (`kcpSettings`)
 - 6.4. WebSocket (`wsSettings`)
 - 6.5. gRPC (`grpcSettings`)
 - 6.6. HTTPUpgrade (`httpupgradeSettings`)
 - 6.7. XHTTP / SplitHTTP (`xhttpSettings`)
 - 6.8. Hysteria transport (`hysteriaSettings`)
 - 6.9. Related parameters
- 7. Connection Security: TLS,XTLS, and REALITY
 - 7.1. What's the difference: TLS vs XTLS vs REALITY
 - 7.2. "None" mode (`none`)
 - 7.3. TLS mode
 - 7.4. REALITY mode
 - 7.5. Practical configuration recommendations
- 8. Clients
 - 8.1. Client fields
 - 8.2. Binding to inbounds
 - 8.3. Client operations
 - 8.4. Bulk operations
 - 8.5. Search, filters, and sorting
 - 8.6. Badges and statuses
- 9. Client groups
 - 9.1. What a client group is and why you need it
 - 9.2. How a group relates to clients, inbounds, nodes, and protocols
 - 9.3. The groups directory and "empty" groups
 - 9.4. Group fields and columns
 - 9.5. Creating a group
 - 9.6. Renaming a group
 - 9.7. Adding clients to a group
 - 9.8. Removing clients from a group (without deleting the clients themselves)
 - 9.9. Resetting group traffic

- 9.10. Deleting a group and deleting group clients
- 9.11. Relationship with the "Clients" page
- 9.12. API endpoints summary
- 9.13. Traffic by group
- 10. Subscriptions (Subscription)
 - 10.1. What subld is and how the link is built
 - 10.2. Subscription server settings
 - 10.3. Output formats
 - 10.4. Subscription information page and QR codes
 - 10.5. Custom subscription page templates
- 11. Xray: routing, outbounds, DNS, and extensions
 - 11.1. Editor layout: tabs/modes
 - 11.2. General settings (General)
 - 11.3. Routing rules (routing)
 - 11.4. Outbounds (outgoing connections)
 - 11.5. Balancers
 - 11.6. DNS
 - 11.7. Fake DNS
 - 11.8. WireGuard / WARP / NordVPN
 - 11.9. Reverse proxy and TUN
 - 11.10. Logs and statistics (Stats, metrics)
 - 11.11. Saving, restart, and automatic transformations
 - 11.12. Outbounds from a subscription (with auto-update)
 - 11.13. IP rotation in WARP
- 12. Nodes (multi-panel, master/slave)
 - 12.1. Summary at the top of the list
 - 12.2. Adding and editing a node
 - 12.3. TLS verification (for https nodes)
 - 12.4. What is shown for each node
 - 12.5. Actions on a node
 - 12.6. Metric history
 - 12.7. How inbounds and clients are synchronized
 - 12.8. Node chains (sub-nodes / transitive nodes)
 - 12.9. Nodes: new in 3.3.0
- 13. Panel Settings
 - 13.1. Saving and restarting the panel
 - 13.2. General settings ("Panel" tab / *General*)
 - 13.3. Panel access: IP, port, path, domain, certificate
 - 13.4. Session, panel proxy, and trusted proxies ("Proxy and Server" tab / *Proxy and Server*)
 - 13.5. Telegram bot ("Telegram Bot" tab / *Telegram Bot*)
 - 13.6. Date and time ("Date and Time" tab / *Date and Time*)
 - 13.7. External traffic and Xray behavior ("External Traffic" tab / *External Traffic*)
 - 13.8. Other: Xray configuration template and test URL
 - 13.9. Administrator account and API tokens
 - 13.10. API changes in 3.3.0 (important for integrations)

- 14. Telegram Bot
 - 14.1. Enabling and configuring the bot
 - 14.2. Main menu and buttons
 - 14.3. Bot commands
 - 14.4. Client management (administrator only)
 - 14.5. Notifications and reports
 - 14.6. Backup and logs
 - 14.7. Operational details
 - 15. Geo databases (geoip / geosite and custom)
 - 15.1. What geoip.dat and geosite.dat are
 - 15.2. Standard geo files and their update
 - 15.3. Custom geo resources (Custom GeoSite / GeoIP)
 - 15.4. Validation and constraints
 - 15.5. Auto-check at panel startup
 - 15.6. Using geo databases in routing rules
 - 16. Operations: Backups, Logs, Updates, CLI
 - 16.1. Database Backup and Restore
 - 16.2. Viewing Logs
 - 16.3. Xray Logging Level and Configuration
 - 16.4. Managing Xray: Stopping and Restarting
 - 16.5. Restarting and Updating the Panel
 - 16.6. Periodic Tasks (cron)
 - 16.7. Console Menu and CLI (`x-ui`)
 - 16.8. Uninstalling the Panel
 - 16.9. The `x-ui migrateDB` Command
-

What's new in 3.3.1

This section briefly lists the changes in version **3.3.1** relative to 3.3.0 that are visible to a panel user, grouped by manual section. Details for each feature are in the corresponding section below.

1. Introduction, requirements, and installation

- **XUI_INIT_WEB_BASE_PATH environment variable for the initial web-panel path** — A new XUI_INIT_WEB_BASE_PATH environment variable has been added — it sets the initial URI path of the web panel on first initialization (default "/"). This is convenient for Docker deployments, so the panel path can be set right away. Leading and trailing slashes are added automatically; an empty value is ignored. The variable affects only the initial initialization — afterwards the path is changed in the panel settings or via the CLI.

3. Overview / Dashboard

- **The "Xray Logs" button on the dashboard appears only when an access log is configured** — The Xray log viewer button on the dashboard now appears only when an access log is configured in the Xray configuration (the viewer reads exactly that log). If no access log is set, the button is hidden — meanwhile the online list and the IP limit keep working without it.
- **Total inbound traffic is shown with up/down arrows** — In the summary-statistics block on the Inbounds page, total traffic is now shown with explicit up (sent) and down (received) arrows instead of the previous ambiguous icon.

4. Inbounds: creation and common parameters

- **Share address strategy for sharing links on an inbound** — An inbound now has a "Share address strategy" setting. It determines which address is inserted into exported links and QR codes: Node address — the node's address, Inbound listen — the listen address of the inbound itself, Custom — your own address (enter a host or IP without scheme and port in the Custom share address field). The Node address option is available only if there is an enabled node on which this inbound can run. This strategy does not affect subscription output.
- **Abstract unix sockets (@-prefix) in the inbound Address field** — In the inbound's Address field you can now specify not only a path to a unix socket (for example /run/xray/in.sock) but also an abstract socket name with the @ prefix (for example @xray/in.sock) — then the inbound listens on the socket rather than on a TCP port. In both cases set Port to 0.
- **A hint on how to unlock Fallbacks on the inbound form** — If a VLESS/Trojan over RAW (TCP) inbound has no security configured yet, instead of the Fallbacks section a hint is now shown: fallbacks become available after you select TLS or Reality on the Security tab.
- **JSON editor (CodeMirror) for inbound import and Inbound JSON view** — The inbound import and inbound JSON view dialogs now use a full code editor with JSON syntax highlighting instead of a plain text field, which makes reading and editing the configuration easier.
- **Finished clients on the Inbounds page are counted as "Ended/Depleted" rather than "disabled"** — In the client summary on the Inbounds page, clients whose term has expired or whose traffic is exhausted now fall into the "depleted/ended" status rather than mistakenly into "disabled", and are not counted twice.

5. Protocols

- **WireGuard peer comments for identifying devices** — On a WireGuard inbound each peer now has an optional "Comment" field. The label is shown next to the Peer N heading and is added to the sharing link and to the remark of the .conf file — this is handy for telling devices apart in client apps.
- **New WireGuard capabilities from Xray-core: Workers, Domain Strategy, Transport tab** — The WireGuard inbound has new Workers (number of workers) and Domain Strategy (ForcelP / ForcelPv4 / ForcelPv4v6 / ForcelPv6 / ForcelPv6v4) fields. WireGuard also gains a Transport tab where you can configure sockopt and Finalmask obfuscation — the transport selector is hidden there, since WireGuard always runs over UDP. In Finalmask noise records a Rand Range (0-255) field has been added, and for WireGuard and Hysteria the Salamander obfuscation method is available.
- **MTPROTO: domain fronting and extended mtg options** — The MTPROTO inbound has new extended mtg options: Domain fronting IP, Domain fronting port, and Domain fronting PROXY protocol — they set where non-Telegram traffic is sent (for example to a fake NGINX site); if the IP is left empty, the FakeTLS domain is used via DNS, port 443 by default. Also added are Accept PROXY protocol (for the listener), IP preference (prefer-ipv6/prefer-ipv4/only-ipv6/only-ipv4), and Debug logging. The Sniffing tab is hidden for MTPROTO, since this protocol is served by a separate mtg process rather than Xray.
- **MTPROTO: routing Telegram traffic through Xray** — The MTPROTO inbound has a "Route through Xray" toggle (off by default) and an optional Outbound field. When enabled, this inbound's Telegram traffic is routed through Xray (the panel adds a local SOCKS bridge tagged with the inbound), and it can be subjected to routing rules on the Routing tab or forced into a chosen outbound or balancer via the Outbound field (empty — routing rules decide).
- **Tunnel (dokodemo-door): Transport tab for TProxy** — The Tunnel (dokodemo-door) inbound has a Transport tab limited to the sockopt setting — that is enough for TProxy mode (transparent proxying/redirect via sockopt.tproxy). The transport selector and the Security tab are hidden for Tunnel, since TLS/Reality are not supported by this type.

6. Transport (Stream Settings)

- **XMUX multiplexing for the XHTTP transport on an inbound** — For the XHTTP transport in the inbound form there is now an XMUX toggle — a multiplexing layer that distributes parallel requests across a small pool of connections. After it is enabled, these fields become available: Max Concurrency (default 16-32), Max Connections (default 0), Max Reuse Times, Max Request Times (600-900), Max Reusable Secs (1800-3000), and Keep Alive Period. Note: you cannot set both Max Connections and Max Concurrency at once — xray-core rejects such a config, so when Max Connections is set, the Max Concurrency value is not used.
- **Hysteria: UDP idle timeout limited to the 2-600 s range** — On the Hysteria transport the "UDP idle timeout (s)" field now accepts only values in the 2-600 second range — xray-core rejects values outside this interval on startup.
- **XHTTP: the panel no longer inserts scMaxEachPostBytes / scMinPostsIntervalMs by default** — The panel no longer inserts the default service values scMaxEachPostBytes and scMinPostsIntervalMs into XHTTP configs — xray-core's internal values are used. This removes a constant DPI signature (scMinPostsIntervalMs=30) by which traffic could previously be blocked. For already-saved inbounds, values matching the defaults are not emitted in links and the subscription; values you set manually are preserved.

- **Free packet ranges for fragmentation (Freedom outbound) — not just presets** — In the fragmentation settings the "Packets" field is no longer limited to presets (tlshello / 1-3 / 1-5): it is now an autocomplete field where you can enter an arbitrary packet range of the form n-m (for example 1-3) or tlshello. The presets remain as suggestions.

7. Connection Security: TLS, XTLS, and REALITY

- **XTLS Vision (flow) for VLESS+XHTTP with VLESS encryption (vlessenc)** — The XTLS Vision flow (flow = xtls-rprx-vision) can now be enabled not only for VLESS over TCP with TLS/REALITY, but also for VLESS over the XHTTP transport when VLESS encryption (vlessenc) is enabled. The flow value is shown correctly in the client form and is included in the vless:// link.

8. Clients

- **The IP limit and client IP log no longer depend on the Xray access log** — The limit on the number of simultaneous IPs now works through the Xray core API and does not require the access log to be enabled. The "IP Limit" field and the IP-log view button are always available, regardless of the logging settings. A 0 in the "IP Limit" field means "no limit".
- **The "Reset traffic" button is back in the client edit form** — The client edit form now has a "Reset traffic" button (at the bottom of the form, next to "Cancel"/"Save"). It zeroes the client's used-traffic counter; a confirmation is requested before the reset.
- **Traffic-usage indicators (progress bar) restored in the client table** — The "Traffic" column on the Clients page now shows a colored usage bar: the amount of used traffic, the limit (or the ∞ sign for unlimited), and a tooltip on hover with a breakdown into sent/received and remaining. The same indicator is shown in the client cards on mobile.
- **Client filter by node (Nodes) in the filter panel** — On the multi-panel a "Nodes" multiselect has appeared in the client filters: you can narrow the list to the clients of selected nodes. A separate "Local panel" item selects clients of inbounds not bound to any node. The filter is shown only when nodes are present.
- **Reworked client form: new tab, field, and hint names** — The tabs are now called "General" and "Credentials". The quota field has been renamed to "Traffic limit (GB)", and both it and the "IP Limit" field have hints (0 = no limit). Terms are specified as "Duration (days)" and "Auto-renew (days)". The IP-log button is placed right next to the "IP Limit" field and shows the number of recorded addresses. When editing an existing client, the random-email generation button is hidden.
- **Resetting traffic automatically re-enables a disabled client** — When a client's traffic is reset (whether individually from the edit form or in bulk), the panel now automatically re-enables the client if it was disabled due to traffic exhaustion, and immediately propagates this change to the nodes. You no longer need to enable the client manually after a reset.
- **"Select all / Clear" buttons above the inbound list when working with clients** — When selecting inbounds for a client, "Select all" and "Clear" buttons appear above the list. They are available in the client form, in bulk client addition, and in the bulk inbound attach/detach dialogs.
- **Inbound label in client lists: remark first, otherwise tag** — In lists of inbounds associated with clients, an inbound is labeled with its remark (if one is set), otherwise with the inbound tag.
- **The client-get API returns used traffic (usedTraffic)** — A client request via the API (GET /panel/api/clients/get/:email) now additionally returns a usedTraffic field — the actually consumed traffic (sent + received, accounting for node data), which makes it easier to compare consumption against the totalGB quota.

9. Client groups

- **Group traffic summary: sent/received breakdown and column rename** — On the Groups page each group now shows separate "Sent" and "Received" columns. Two cards have appeared in the summary at the top: "Total sent / received" and "Total traffic". The client-count column is now simply called "Clients".

10. Subscriptions (Subscription)

- **Order of inbound links in the subscription**
(`subSortIndex`) — Each inbound has a "Subscription order" field — a number from 1 that sets the position of this inbound's links in the subscription output (raw text, the subscription page, the JSON and Clash formats). Smaller values come first; for equal values the original creation order is preserved. This field does not affect the order of inbounds in the panel itself. If at least one inbound has an order other than 1, an "Order" column appears in the Inbounds list.
- **"Copy all configs" button on the subscription page** — Copying all configs. The subscription page has a "Copy all configs" button — it copies all config links to the clipboard at once (each on a new line), without requiring you to copy them one by one.
- **Node name in the remark of records from nodes** — For an inbound hosted on a node, the node name is automatically appended to the remark in the subscription via "@" (for example, my-inbound@node1), unless the administrator set it manually. This removes the situation where, in a multi-node subscription, several records with identical names differed only by address.
- **The subscription respects the inbound's share address strategy** — The subscription now inserts the server address using the same share address strategy as ordinary links and QR codes: "listen" — the routable bind address, "custom" — the user-defined custom address, "node" (default) — the node's address. For an inbound without an explicitly set strategy, the subscription output does not change. This lets a node inbound bound to a specific public IP hand clients a reachable address.
- **Reworked tabs in the subscription settings** — The "Settings → Subscription" section is split into tabs: "Panel settings", "Information", "Profile", "Certificates", "Happ", and "Clash / Mihomo". The title, support URL, profile page, announcement, and theme directory fields have been moved to the "Profile" tab. The Happ and Clash/Mihomo routing rules are moved to their own tabs. The subscription update interval is now on the "Information" tab.
- **Removing duplicate clients in the subscription** — Previously, due to desynchronization between nodes, the same client could end up in the subscription twice, creating duplicate profiles. Now the subscription output automatically removes such duplicates (by email) in all formats — raw, Clash, and JSON.
- **URLs for JSON/Clash are taken from the configured subURI (reverse proxy)** — If a single subscription address (subURI) is set in the settings for a reverse-proxy scheme, but separate JSON and Clash addresses are not specified, these links now automatically inherit the scheme and host from the configured subURI (rather than the sub-server port and http). This way the JSON and Clash links on the subscription page match the reverse-proxy address.
- **Link to the subscription theme template guide** — In the description of the "Subscription theme directory" setting, a "Template guide ↗" link has been added to the documentation on creating your own subscription-page design templates.

11. Xray: routing, outbounds, DNS, and extensions

- **Inbound/outbound/routing changes are applied "live" via Xray gRPC (without a full restart)** — Starting with 3.3.1, changes to inbounds, outbounds, and routing rules are applied "live": when you click "Save", the panel applies only the changed parts of the configuration via the Xray gRPC API, without restarting the process. A full Xray restart is performed automatically only when sections that do not support hot reload change (logs, DNS, policy, observatory, etc.). The separate "Restart" button on the Xray page is no longer needed and has been removed.
- **New "Route Tester" on the Routing tab** — A "Route Tester" has appeared on the Routing tab. Specify a domain or IP, port, network (TCP/UDP) and, if needed, an inbound and the sniffed protocol (http/tls/quic/bittorrent), then click "Test Route". The panel asks the running Xray which outbound would handle such a connection; no real traffic is sent. The response shows the matched outbound (and the balancer tag, if the choice goes through a balancer). If no rule matched, traffic goes to the default outbound (the first in the list).
- **The outbound for panel traffic is set by choosing an Xray outbound instead of a manual proxy URL** — Instead of manually entering a proxy address (socks5://, http://), you now choose a ready Xray outbound in the "Panel Traffic Outbound" list. The panel's own requests go through it: version checks and downloads of the panel and Xray, Telegram, and ordinary geo-file updates — this lets you bypass server-side blocking of GitHub/Telegram. The panel itself adds a service loopback SOCKS bridge to the working config and applies the setting without a restart. Outbounds of type blackhole are not shown in the list. Leave the field empty for a direct connection. This choice does not affect Xray's native auto-update of geo data — it has its own separate outbound.
- **A balancer can be chosen as the outbound for the panel's own traffic** — In the outbound selection for the panel's own traffic you can now also specify a balancer: in the drop-down list, balancers are placed in a separate "Balancers" group. If you choose a balancer, the panel's outbound traffic will be distributed across its members, just like ordinary traffic through that balancer.
- **Outbound test: batch checking with a real HTTP request, broken down by stage and with the response code** — The outbound check is now performed with a real HTTP request through each outbound, and in bulk and in parallel. In the result dialog, in addition to success/failure, the HTTP response code and a stage breakdown are shown: "Proxy connect", "TLS via outbound", and "First byte" (time to first byte). The "Test all" button checks all outbounds at once. TCP mode is a quick dial-only check, HTTP mode is a full request through xray.
- **Balancers: live-state and manual target-override columns** — On the Balancers tab there are new "Live Target" columns — the balancer's current active target in the running Xray — and "Override", which lets you manually override the target choice (the value "Auto (strategy)" returns to the strategy-based choice). The state can be refreshed with the refresh button. If the balancer is not yet active in the running Xray, the panel offers to save the changes or to start Xray first.
- **WARP/WireGuard outbound: IPv4 priority with IPv6 fallback and userspace TUN** — Generated WARP and NordVPN outbounds now prefer IPv4 with an IPv6 fallback (the ForceIPv4v6 strategy) — this removes the situation where, on a server with partially configured IPv6, the WARP handshake "hung" with no log entries. In addition, such outbounds use userspace TUN, which works more reliably on typical VPS and matches the path the built-in connection test checks. The change applies only to newly added or reset outbounds; already-saved settings are not changed.
- **MTProto inbound: routing Telegram traffic through Xray rules** — The MTProto inbound has a new "Route through Xray" option (off by default): when enabled, this proxy's Telegram traffic goes through Xray and is subject to your routing rules. The panel creates a service loopback SOCKS bridge tagged with this inbound,

which you can reference in rules on the Routing tab. The optional "Outbound" field lets you force Telegram traffic through a specific outbound or balancer; leave it empty for the routing rules to decide.

- **Outbound test: increased timeouts and the failure reason is shown** — The outbound check timeouts have been increased so that working outbounds on slow or tunneled channels are no longer mistakenly marked as "Failed". On failure, the real reason (DNS error, connection refused, timeout, TLS error, etc.) is now written to the panel/Xray log to which the timeout messages point.
- **The routing-rules list shows the tag together with the remark** — In the routing-rules list (and in the rule form, the route tester, and other places with an inbound list), an inbound is now shown as "tag (remark)" if it has a separate remark; otherwise only the tag is shown. Saved rules still store only tags.
- **Fixed: editing a DNS server showed default values instead of the saved ones** — A bug has been fixed: when editing a DNS-server record, the dialog opened with default values instead of the previously saved ones. The DNS-server edit form now correctly fills in the saved parameters.

12. Nodes (multi-panel, master/slave)

- **Selecting which inbounds to import when adding a node** — The node form has a new "Import inbound" setting: you can choose "All inbounds" (default) or "Selected". In "Selected" mode, click "Load inbound" to pull the list of inbounds from the node, and check the tags you need. The panel will synchronize only the checked inbounds, and the rest of the inbounds on the node will be left untouched. Existing nodes use "All inbounds" mode by default.
- **The master sends nodes the client's total traffic** — For a client bound to several panels, the master now broadcasts to the nodes the total traffic consumption across all panels. On the node, the larger of the values (local or received) is shown in the client's traffic, and when the total quota is exceeded the client is disabled locally on the node. This removes the situation where a node undercounted traffic and kept serving a client who had already exhausted the shared limit.

13. Panel Settings

- **"Panel proxy URL" replaced by an outbound selection for panel traffic** — Instead of the "Panel proxy URL" field, an "Outbound for panel traffic" drop-down list is now used. Choose the Xray outbound (or balancer) through which the panel will send its own requests — version checks and their downloads, Telegram, geo-file updates; this helps bypass server-side blocking of GitHub/Telegram. The panel itself adds a service loopback inbound and a routing rule and applies them without a restart. Leave the field empty for a direct connection. Xray's built-in auto-update of geo data is not affected by this setting — it has its own outbound.
- **The "Restart Xray after auto-disable" toggle moved to the panel settings** — The "Restart Xray after auto-disable" toggle has moved from the external-traffic tab to the main panel settings. The function itself has not changed: when a client is automatically disabled (by term expiry or traffic limit), Xray is restarted to break already-established connections.

14. Telegram Bot

- **A cron-schedule guide instead of a text field for the notification time** — The Telegram bot's "Notification time" field is now set via a schedule builder: choose an "@every" interval (a number and a unit — seconds/minutes/hours) or a preset (hourly / daily / weekly / monthly). The "Custom" option lets you enter an arbitrary 6-field crontab expression (cron in the panel accounts for seconds).

15. Geo databases (geoip / geosite and custom)

- **Auto-update of geo data by Xray-core (replacing the custom geo manager)** — The panel's custom geo-file manager has been replaced by native auto-update of geo data by Xray-core. In the Xray update dialog there is a "Geodata Auto-Update" section: set a schedule (cron, 5 fields, for example 0 4 * * *), optionally an outbound for downloads, and list files as "file name + HTTPS URL" pairs. File names are given without paths (for example geosite_custom.dat); they are referenced in routing rules as ext:geosite_custom.dat:category. Xray downloads these files on schedule and picks them up without a restart. Each file must be present in the bin folder once before Xray can update it. Previously downloaded geo files keep working.
- **A "Geo-data auto-update" section instead of panel uploads of custom geo files** — Instead of the previous panel upload of custom geo files, a "Geo-data auto-update" section has been added to the Xray update dialog (next to the core update). It uses Xray's native geodata section: set a schedule (a 5-field cron, for example "0 4 * * *"), optionally an outbound for downloads, and a list of files — for each, specify the HTTPS URL and the file name (for example geosite_custom.dat, without paths). The file must already exist in the bin folder so that Xray can update it. After saving, Xray is restarted. Xray downloads the files on schedule and picks them up without a restart; in routing rules they are available as ext:geosite_custom.dat:category.

16. Operations: Backups, Logs, Updates, CLI

- **Online detection and the IP limit now work without an access log** — Starting with 3.3.1, the panel detects online clients and counts their IP addresses through the Xray core's online-stats API rather than through the access log. Thanks to this, the online-client list and the IP-count limit work even without the Xray access log enabled. On older core versions the panel automatically falls back to the previous method (reading the access log), and then the IP limit still needs the log.
- **SSL certificate issuance works correctly on IPv4-only hosts** — Let's Encrypt certificate issuance via the CLI has been fixed for servers without IPv6: standalone validation listens on IPv4 by default and switches to IPv6 only if the host has no global IPv4. Previously, HTTP-01 validation could fail when the domain's A record pointed to IPv4. Additionally, the SSL menu reads the names (SAN) directly from the certificate and shows which other domains it covers.
- **The CLI no longer shows an existing API token; a new fallback token is issued** — API tokens are now stored only as hashes, so a previously issued token cannot be retrieved in clear text via the CLI. If tokens already exist, the CLI command reports their count and generates a new fallback token (a name like cli-fallback-...); managing tokens is recommended in the panel (the API tokens section).
- **Restarting the panel after changing webBasePath during an update** — During an update, if the script generated a new random web path for the panel, the x-ui service is now restarted automatically so the new path works right away. Previously, after such an update you had to restart the panel manually, otherwise the new path was unreachable.
- **Applying BBR sysctl settings without side errors** — When enabling/disabling BBR via the x-ui CLI menu, the panel now applies only its own sysctl file and no longer re-applies all system sysctl settings. This removes extraneous errors in the output that are unrelated to x-ui.
- **CLI: adding a certificate at an arbitrary path and a correct SAN-based domain indicator** — The CLI SSL menu now lets you set arbitrary paths to the certificate and key (for certificates outside /root/cert, for example certbot). The access address is now determined by the certificate's real names (SAN), and a certificate with a custom path is visible in the list of existing domains.

- **fail2ban support (IP limit) on Ubuntu 22.04+** — Installing the IP-limit feature (fail2ban) via the CLI now works correctly on Ubuntu 22.04 and newer: the backend is correctly switched to systemd and the required dependencies are additionally installed.

1. Introduction, requirements, and installation

1.1. What 3X-UI is

3X-UI is an open-source web management panel for [Xray-core](#) servers. The panel provides a single multilingual web interface for deploying, configuring, and monitoring a wide range of proxy and VPN protocols: from a single VPS to distributed configurations made up of several nodes.

3X-UI is an extended fork of the original X-UI project. Compared to it, support for more protocols, increased stability, per-client traffic accounting, and many convenient features have been added.

Key capabilities:

- **Inbounds of various protocols** – VLESS, VMess, Trojan, Shadowsocks, WireGuard, Hysteria2, HTTP, SOCKS (Mixed), Dokodemo-door / Tunnel, TUN, and **MTPROTO** (Telegram proxy, added in 3.3.0).
- **Modern transports and encryption** – TCP (Raw), mKCP, WebSocket, gRPC, HTTPUpgrade, and XHTTP, secured with TLS, XTLS, and REALITY.
- **Fallback** – serving multiple protocols on a single port (for example, VLESS and Trojan on 443) using Xray's fallback mechanism.
- **Per-client management** – traffic quotas, expiry dates, IP limits, "online" status display, one-click invite links, QR codes, and subscriptions.
- **Traffic statistics** – per inbound, client, and outbound, with the ability to reset.
- **Multiple node support** – managing and scaling across several servers from a single panel.
- **Outbounds and routing** – WARP, NordVPN, custom routing rules, load balancers, proxy chains.
- **Built-in subscription server** with several output formats.
- **Telegram bot** for remote monitoring and management.
- **REST API** with built-in Swagger documentation.
- **Flexible storage** – SQLite (default) or PostgreSQL.
- **13 interface languages**, dark and light themes.
- **Fail2ban integration** for enforcing per-client IP limits.

Important: the project is intended for personal use only. It is not recommended to use it for illegal purposes or in a production environment.

1.2. Supported operating systems and architectures

Operating systems

The installation script detects the distribution by the `ID` field from `/etc/os-release` (or `/usr/lib/os-release`). Officially supported are:

Ubuntu, Debian, Armbian, Fedora, CentOS, RHEL, AlmaLinux, Rocky Linux, Oracle Linux, Amazon Linux, Virtuozzo, Arch, Manjaro, Parch, openSUSE (Tumbleweed / Leap), Alpine, as well as Windows.

On Alpine-family systems the OpenRC service is used (`rc-service` / `rc-update`), on the rest — `systemd`. For CentOS 7 packages are installed via `yum`, for newer releases — via `dnf`. If the distribution is not recognized, the script defaults to attempting to use the `apt-get` package manager.

CPU architectures

The architecture is detected from the output of `uname -m` and normalized to one of the supported values:

<code>uname -m</code> value	3X-UI architecture
<code>x86_64</code> , <code>x64</code> , <code>amd64</code>	<code>amd64</code>
<code>i*86</code> , <code>x86</code>	<code>386</code>
<code>armv8*</code> , <code>arm64</code> , <code>aarch64</code>	<code>arm64</code>
<code>armv7*</code> , <code>arm</code>	<code>armv7</code>
<code>armv6*</code>	<code>armv6</code>
<code>armv5*</code>	<code>armv5</code>
<code>s390x</code>	<code>s390x</code>

If the architecture is not in this list, the script prints the message "Unsupported CPU architecture!" and aborts the installation.

Base dependencies

Before installing the panel, the script automatically installs a base set of packages (names differ by distribution): `cron` / `cronie` / `dcron`, `curl`, `tar`, `tzdata` / `timezone`, `socat`, `ca-certificates`, `openssl`.

1.3. Installation methods

Method 1. Installation script (recommended)

Installation is performed with a single command run as root:

```
bash <(curl -Ls https://raw.githubusercontent.com/mhsanaei/3x-ui/master/install.sh)
```

The script strictly requires root privileges: when not run as root, it prints "Please run this script with root privilege" and exits with an error.

What the installer does step by step:

1. Detects the OS and architecture.
2. Installs the base dependencies.
3. Downloads the release archive `x-ui-linux-<arch>.tar.gz` and extracts it into the `/usr/local/x-ui` directory.
4. Downloads the management script `x-ui.sh` and installs it as the `/usr/bin/x-ui` command.

5. Creates the log directory `/var/log/x-ui`.
6. Runs the initial setup: database selection, credential generation, port selection, optional SSL configuration.
7. Installs and starts the autostart service (the `x-ui.service` systemd unit or the OpenRC init script for Alpine).

Database selection during installation. The installer offers:

- 1) `SQLite` (default, recommended when the number of clients is < 500) – a single file `/etc/x-ui/x-ui.db`, requires no configuration.
- 2) `PostgreSQL` (recommended for a large number of clients or several nodes). PostgreSQL can be installed locally (a dedicated user and a database named `xui` are created) or you can specify a DSN to an existing server. The connection parameters are written to the service environment file (`/etc/default/x-ui`, `/etc/conf.d/x-ui`, or `/etc/sysconfig/x-ui` depending on the distribution) as the `XUI_DB_TYPE` and `XUI_DB_DSN` variables.

Example: PostgreSQL parameters written to the service environment file. After choosing PostgreSQL and providing a DSN, the installer adds lines like the following to the environment file:

```
XUI_DB_TYPE=postgres
XUI_DB_DSN=postgres://xui:S3cretPass@127.0.0.1:5432/xui?sslmode=disable
```

Here `xui` is the username and database name, `127.0.0.1:5432` is the server address and port, and `sslmode=disable` is fine for a local connection (for a remote server `require` is typically used).

Installing a specific (older) version. You can explicitly specify a version tag – the installer will download the corresponding release:

```
bash <(curl -Ls https://raw.githubusercontent.com/mhsanaei/3x-ui/v2.4.0/install.sh)
v2.4.0
```

The minimum allowed version for such an installation is `v2.3.5`; when an older one is specified, it prints "Please use a newer version (at least v2.3.5)".

Method 2. Docker

Running with the default `SQLite` database:

```
docker compose up -d
```

To run with the built-in `PostgreSQL` service, uncomment the `XUI_DB_*` lines in `docker-compose.yml` and start with the profile:

```
docker compose --profile postgres up -d
```

The image includes `Fail2ban` (active by default) for enforcing per-client IP limits. `Fail2ban` blocks offenders via `iptables`, which requires the `NET_ADMIN` capability. In `docker-compose.yml` it is already granted via

`cap_add`. When running manually via `docker run`, you need to add the capabilities yourself, otherwise blocks will only be logged but not enforced:

Example: a full `docker run` command. A minimal variant that publishes the panel port, adds the network capabilities, and mounts a persistent volume for the database:

```
docker run -d \
  --name 3x-ui \
  --restart unless-stopped \
  --cap-add=NET_ADMIN --cap-add=NET_RAW \
  -v $PWD/db:/etc/x-ui \
  -v $PWD/cert:/root/cert \
  -p 2053:2053 \
  ghcr.io/mhsanaei/3x-ui:latest
```

The `/etc/x-ui` volume preserves the `x-ui.db` file across container restarts; otherwise settings and accounts would be lost.

```
docker run -d --cap-add=NET_ADMIN --cap-add=NET_RAW ... ghcr.io/mhsanaei/3x-ui
```

In Docker the panel is the container's main process: autostart is governed by the container's restart policy (for example, `restart: unless-stopped`), not by a service inside the container.

1.4. First launch and default credentials

On the first installation (when the default credentials are still in use), the installer **generates random values** for the username, password, and web path, as well as the port:

Parameter	How it is formed during installation	Note
Username	a random 10-character string	generated automatically
Password	a random 10-character string	generated automatically
Panel web path (WebBasePath)	a random 18-character string	protects the panel from being discovered by its root URL
Panel port (Port)	by default a random port in the range 1024–62000; you can set it manually if you wish	the "factory" <code>webPort</code> value is <code>2053</code> , but the installer overwrites it

At the end of installation the script prints a final summary: username, password, port, web path, API token, and a ready-to-use login link (Access URL) of the form:

```
https://<domain-or-IP>:<port>/<web-path>
```

If no SSL certificate is configured, the link will be over `http://`, and the script will print a warning about the need to configure SSL (menu item 19).

Mandatory change of credentials. Since the login and password are generated randomly, they should be **saved immediately after installation**. You can change them at any time via the "Reset Username &

Password" menu item (see below) or from the web interface in the panel settings. After the reset, the script reminds you: "Please use the new login username and password to access the X-UI panel. Also remember them!".

After installation, the `x-ui` command is used to open the management menu (see section 1.6).

1.5. File locations

Path	Purpose
<code>/usr/local/x-ui/</code>	panel installation directory (the <code>x-ui</code> binary, the <code>x-ui.sh</code> script)
<code>/usr/local/x-ui/bin/xray-linux-<arch></code>	Xray-core binary (on armv5/armv6/armv7 it is renamed to <code>xray-linux-arm</code>)
<code>/usr/bin/x-ui</code>	management script (the <code>x-ui</code> command)
<code>/etc/x-ui/x-ui.db</code>	SQLite database file (default)
<code>/var/log/x-ui/</code>	panel log directory
<code>/etc/systemd/system/x-ui.service</code>	systemd service unit (not for Alpine)
<code>/etc/init.d/x-ui</code>	OpenRC init script (Alpine only)
<code>/etc/default/x-ui</code> · <code>/etc/conf.d/x-ui</code> · <code>/etc/sysconfig/x-ui</code>	service environment variables file (path depends on the distribution); <code>XUI_DB_TYPE</code> / <code>XUI_DB_DSN</code> are written here

The database directory can be overridden with the `XUI_DB_FOLDER` environment variable (default `/etc/x-ui`), and the Xray binaries directory with the `XUI_BIN_FOLDER` variable (default `bin` relative to the panel directory). The database file name is `x-ui.db`.

Example: moving the database to a separate disk. To store `x-ui.db` outside `/etc/x-ui` – for example, on a mounted disk at `/data` – set the variable in the service environment file and restart the panel:

```
echo 'XUI_DB_FOLDER=/data/x-ui' >> /etc/default/x-ui
mkdir -p /data/x-ui
systemctl restart x-ui
```

The full path to the database then becomes `/data/x-ui/x-ui.db`.

Main environment variables

Variable	Purpose	Default
<code>XUI_DB_TYPE</code>	database backend: <code>sqlite</code> or <code>postgres</code>	<code>sqlite</code>
<code>XUI_DB_DSN</code>	PostgreSQL connection string (when <code>XUI_DB_TYPE=postgres</code>)	–
<code>XUI_DB_FOLDER</code>	SQLite database file directory	<code>/etc/x-ui</code>
<code>XUI_INIT_WEB_BASE_PATH</code>	initial web panel URI path (first-time initialization only)	<code>/</code>

Variable	Purpose	Default
XUI_DB_MAX_OPEN_CONNS	maximum open connections (PostgreSQL pool)	—
XUI_DB_MAX_IDLE_CONNS	maximum idle connections (PostgreSQL pool)	—
XUI_ENABLE_FAIL2BAN	enable enforcement of IP limits via Fail2ban	true
XUI_LOG_LEVEL	logging level (debug , info , warning , error)	info
XUI_DEBUG	debug mode	false

Example: temporarily enable verbose logging. To diagnose an issue, raise the log level to `debug` and restart the service:

```
echo 'XUI_LOG_LEVEL=debug' >> /etc/default/x-ui
systemctl restart x-ui
x-ui log      # view the debug log
```

After diagnosing, set the value back to `info` so the log does not grow unbounded.

Setting the initial web panel path via the environment. The `XUI_INIT_WEB_BASE_PATH` variable sets the web panel's URI path (`webBasePath`) during the first-time initialization of the settings. This is convenient when deploying via Docker or systemd, so the panel's login path is fixed from the start. The value is normalized automatically — a leading and trailing slash are added if needed, and an empty or whitespace-only value is ignored (the default path `/` is used instead). The variable affects **first-time initialization only**: once the settings exist, the path is changed in the web interface or via the "Reset Web Base Path" menu item.

1.6. The `x-ui` management command (script menu)

After installation, the `x-ui` command (run as root) opens the interactive "3X-UI Panel Management Script" menu. An item is selected by entering its number (range 0–27). Many items are also available as subcommands for scripts (see section 1.7).

The menu is divided into thematic blocks.

Installation and updates

- **1. Install** — install the panel (runs `install.sh`). Before installation it checks that the panel is not already installed.
- **2. Update** — update all x-ui components to the latest version. No data is lost in the process; after the update the panel is restarted automatically. Requires confirmation.
- **3. Update Menu** — update only the management script (`x-ui.sh` / the `x-ui` command) to the current version without reinstalling the panel.
- **4. Legacy Version** — install a specified (older) version of the panel. The script asks for the version number (for example, `2.4.0`) and downloads the corresponding release.
- **5. Uninstall** — completely remove the panel **together with Xray**. The service is stopped and disabled, the `/etc/x-ui/` and `/usr/local/x-ui/` directories, the service environment file, and the management script itself are removed. Requires confirmation (default "no").

Credentials and settings

- **6. Reset Username & Password** — reset the panel's username and password. You can enter your own values or leave them empty for random generation (random username — 10 characters, random password — 18 characters). Additionally, it offers to disable two-factor authentication (2FA) if it is configured. After the reset the panel is restarted.
- **7. Reset Web Base Path** — reset the panel's web path: a new random path is generated (18 characters), the panel is restarted. Used if the previous path has been compromised or forgotten.
- **8. Reset Settings** — reset all panel settings to their default values. **Credentials (username and password) and account data are not lost in the process.** Requires confirmation; after the reset the panel is restarted.
- **9. Change Port** — change the web panel's port. A port number (1–65535) is requested; after setting it, a restart is required for the port to take effect.
- **10. View Current Settings** — view the current settings (`x-ui setting -show`). It shows, among other things, the database backend in use (SQLite or PostgreSQL with the password masked in the DSN) and the ready-to-use access link (Access URL). If SSL is not configured, it offers to issue a Let's Encrypt certificate for the IP address.

Service management

- **11. Start** — start the panel service. If the panel is already running, a message is printed that restarting is not necessary.
- **12. Stop** — stop the panel service.
- **13. Restart** — restart the panel service.
- **14. Restart Xray** — restart only the Xray-core engine without restarting the panel itself (via `systemctl reload x-ui`, in Docker — with the `USR1` signal to the panel process).
- **15. Check Status** — check the service status (`systemctl status x-ui` or `rc-service x-ui status`).
- **16. Logs Management** — log management: viewing the debug log (Debug Log, via `journalctl`) and, except on Alpine, clearing all logs (Clear All logs).

Autostart

- **17. Enable Autostart** — enable autostart of the panel at OS boot (`systemctl enable x-ui` or `rc-update add`).
- **18. Disable Autostart** — disable autostart at OS boot.

In Docker, autostart is governed by the container's restart policy, so these items only print a corresponding hint.

Security and network

- **19. SSL Certificate Management** — managing SSL certificates via acme.sh: issuing a certificate for a domain, revocation, forced renewal, viewing existing domains, specifying certificate paths for the panel, as well as issuing a short-lived (~6 days, with auto-renewal) certificate for an IP address.
- **20. Cloudflare SSL Certificate** — issuing an SSL certificate via Cloudflare DNS validation.
- **21. IP Limit Management** — managing per-client IP count limits (based on Fail2ban): viewing and removing blocks, etc.
- **22. Firewall Management** — firewall management (opening/closing ports and viewing rules).

- **23. SSH Port Forwarding Management** — configuring SSH port forwarding to open the panel from a local machine through an SSH tunnel.

Performance and maintenance

- **24. Enable BBR** — enabling/disabling the BBR TCP congestion control algorithm (a submenu with Enable BBR / Disable BBR items).
- **25. Update Geo Files** — updating geo databases (`.dat` files) with a choice of source: Loyalsoldier (`geoip.dat` , `geosite.dat`), chocolate4u (`geoip_IR.dat` , `geosite_IR.dat`), runetfreedom (`geoip_RU.dat` , `geosite_RU.dat`), or All (all at once). After the update the panel is restarted.
- **26. Speedtest by Ookla** — running a network speed test via Speedtest by Ookla.
- **27. PostgreSQL Management** — managing the built-in/linked PostgreSQL instance (enabling and related operations).
- **0. Exit Script** — exit the menu.

1.7. `x-ui` subcommands (without the interactive menu)

For use in scripts, the `x-ui` command supports direct subcommands (running `x-ui` without arguments opens the menu):

Command	Action
<code>x-ui</code>	open the management menu
<code>x-ui start</code>	start the panel
<code>x-ui stop</code>	stop the panel
<code>x-ui restart</code>	restart the panel
<code>x-ui restart-xray</code>	restart Xray
<code>x-ui status</code>	current service status
<code>x-ui settings</code>	current settings
<code>x-ui enable</code>	enable autostart at OS boot
<code>x-ui disable</code>	disable autostart
<code>x-ui log</code>	view logs
<code>x-ui banlog</code>	view Fail2ban block logs
<code>x-ui update</code>	update the panel
<code>x-ui update-all-geofiles</code>	update all geo files
<code>x-ui migrateDB [file]</code>	convert <code>.db</code> <code>[?]</code> <code>.dump</code> (SQLite)
<code>x-ui legacy</code>	install an older version
<code>x-ui install</code>	install the panel
<code>x-ui uninstall</code>	uninstall the panel

1.8. SQLite → PostgreSQL migration

An existing SQLite installation can be migrated to PostgreSQL:

```
x-ui migrate-db --dsn "postgres://xui:password@127.0.0.1:5432/xui?sslmode=disable"  
# then set XUI_DB_TYPE and XUI_DB_DSN in /etc/default/x-ui and restart:  
systemctl restart x-ui
```

The original SQLite file remains untouched – delete it manually only after verifying that the new backend works.

Example: verifying the switch to PostgreSQL. After migration, confirm the panel really runs on the new backend with the settings command – the output should list PostgreSQL (the password in the DSN is masked):

```
x-ui settings | grep -i -E 'db|dsn'
```

Once the panel opens and the accounts are present, the original `x-ui.db` can be removed.

2. Panel login and access security

This section covers everything related to authenticating the administrator of the 3X-UI panel: the login form, two-factor authentication (TOTP), brute-force protection, changing credentials, changing the panel's secret path and port, session lifetime, and synchronization/authentication via LDAP.

2.1. Login form

The login page is served at the root of the panel's secret path (`webBasePath`). If the user is already authenticated, they are automatically redirected to `.../panel/`. The page has a theme switcher, an interface language selector, and the form itself.

Form fields:

Field	Hint/label (RU)	Required	Description
Username	"Username"	Yes	The administrator's login. An empty value is rejected on the client side, and on the server side with the message "Enter username".
Password	"Password"	Yes	The administrator's password. An empty value is rejected with the message "Enter password".
2FA code	"2FA code"	Only when 2FA is enabled	The field appears only if two-factor authentication is enabled for the panel. A 6-digit code from the authenticator app.

The **"Login"** button submits the form to `POST /login`.

Behavior and messages:

- On successful login, "Login successful" is shown and the user is taken to `.../panel/`.
- On any credentials error or an incorrect 2FA code, the server returns a **single** message: "Invalid account data." (English: *Invalid username or password or two-factor code.*). This is intentional — the panel does not reveal exactly what is wrong (login, password, or code) so as not to make brute-forcing easier.
- The panel shows or hides the "2FA code" field based on the `POST /getTwoFactorEnable` request, which returns the current 2FA status even before authorization.
- If the server-side session has expired, the next request shows "Session expired. Please log in again", and the user is redirected to the login page.

Note about CSRF: before submitting the form, the client obtains a CSRF token (`GET /csrf-token`); the `/login` and `/logout` requests are protected by a CSRF check.

Example: logging in via the API. When 2FA is off, the login and password are enough; when 2FA is on, the `twoFactorCode` field is added:

```
# Without 2FA
curl -i -X POST https://panel.example.com:2053/my-secret/login \
  -H 'Content-Type: application/x-www-form-urlencoded' \
  --data 'username=admin&password=YourPassword'

# With 2FA enabled – a 6-digit code is added
curl -i -X POST https://panel.example.com:2053/my-secret/login \
  -H 'Content-Type: application/x-www-form-urlencoded' \
  --data 'username=admin&password=YourPassword&twoFactorCode=123456'
```

On success the server returns a `Set-Cookie` header with the session cookie – pass it in subsequent requests to `/panel/api/...`.

2.2. Two-factor authentication (2FA / TOTP)

2FA in 3X-UI is implemented according to the **TOTP** standard and is compatible with any authenticator app (Google Authenticator, Aegis, FreeOTP, etc.). The parameters are hard-coded: algorithm **SHA1**, 6 digits, period **30** seconds, issuer `3x-ui`, label `Administrator`.

Example: the otpauth URI encoded by the QR code. If the authenticator app cannot use the camera, the token can be added manually via a link like this (substitute your own Base32 secret instead of `JBSWY3DPEHPK3PXP`):

```
otpauth://totp/3x-ui:Administrator?secret=JBSWY3DPEHPK3PXP&issuer=3x-
ui&algorithm=SHA1&digits=6&period=30
```

The parameters `algorithm=SHA1`, `digits=6`, `period=30` match the panel's hard-coded values – there is no need to change them.

The settings are located in **Settings** → **Account**, on the **"Two-factor authentication"** tab.

Element	Text (RU)	Description
Toggle	"Enable 2FA"	Enables/disables two-factor authentication.
Description	"Adds an additional layer of authentication to improve security."	The hint below the toggle.

How to enable 2FA

When the toggle is turned on, the panel **generates a new secret locally** – a random string in Base32 encoding (alphabet `A–Z` and `2–7`). The "Enable two-factor authentication" window opens with a step-by-step guide:

1. **"Scan this QR code in your authenticator app, or copy the token next to the QR code and paste it into the app".** Below the QR code, the secret itself is displayed in text form – clicking the QR code copies the secret to the clipboard (a "Copied" notification pops up).
2. **"Enter the code from the app"** – you must enter the 6-digit code generated by the app. The code is verified **on the browser side**: the panel itself computes the current TOTP from the just-generated secret and compares it with the entered one. If the code is incorrect – "Invalid code"; the field accepts only exactly 6 digits.

Only after a successful confirmation are the secret and the enable flag saved. On saving, "Two-factor authentication has been set up successfully" is shown.

Important: changes in the settings section are applied with the common **"Save"** button, after which a panel restart is usually required ("Save the changes and restart the panel to apply them"). When 2FA is enabled for the first time, the server additionally **invalidates all active sessions** (increments the "login epoch"), so after applying the setting a new login will be required – now with the 2FA code.

How to disable 2FA

Toggling the switch again opens the "Disable two-factor authentication" window with the hint "Enter the code from the app to disable two-factor authentication.". After entering a valid code, the flag and the secret are cleared, and "Two-factor authentication has been removed successfully" is shown.

Code verification at login

At login, the server takes the stored secret and compares the current TOTP with the submitted 2FA code. A mismatch is treated as a failed login, but the user is shown the same combined message "Invalid account data."

Recovery of access

There is **no** separate "recovery codes" mechanism in 3X-UI. If access to the authenticator app is lost, login cannot be recovered through the panel interface. The only way is to disable 2FA directly in the database on the server: reset the `twoFactorEnable` key to `false` (and, if necessary, clear `twoFactorToken`) in the settings table, then restart the panel. For this reason, it is recommended to store the secret (the Base32 token) in a safe place when enabling 2FA.

Example: emergency 2FA disable on the server. After getting SSH access to the server, stop the panel, reset the keys in the settings table, and start the panel again:

```
x-ui stop
sqlite3 /etc/x-ui/x-ui.db "UPDATE settings SET value='false' WHERE
key='twoFactorEnable';"
sqlite3 /etc/x-ui/x-ui.db "UPDATE settings SET value='' WHERE key='twoFactorToken';"
x-ui start
```

After this, login works with the username and password only, and 2FA can be set up again if desired.

Relation to changing credentials: when the login/password is changed (see 2.4), 2FA is **automatically disabled** on the server so that the old secret does not block access under the new account.

2.3. Login attempt limiting (login limiter / brute-force protection)

The panel includes a built-in failed-login limiter (an application-level analog of fail2ban). The parameters are set in code and are **not configurable** through the interface:

Parameter	Value	Purpose
Maximum failures	5	How many failed attempts are allowed within the window.
Counting window	5 minutes	The sliding window in which failures accumulate (older ones are discarded).
Lockout (cooldown)	15 minutes	How long the key is blocked after exceeding the threshold.

How it works:

- The lockout key is built from the **"IP + login" pair** (the login is lowercased, whitespace is trimmed). That is, the lockout applies to a specific "address + username" pair, not to the entire panel.
- On each failed attempt (incorrect login/password or an incorrect 2FA code) the counter grows. After reaching **5** failures within **5 minutes**, the key is blocked for **15 minutes**. During the lockout, any attempts by this pair are immediately rejected with the same message "Invalid account data.", even if the data is correct.
- A **successful login immediately resets** the counter and lifts the lockout for that pair.
- The client's IP address is determined taking trusted proxies into account (see `trustedProxyCIDRs`): the `X-Real-IP` and `X-Forwarded-` headers are accepted only if the request came from a trusted address. Otherwise the real connection address is used, and if it cannot be extracted – the string `unknown`.

All attempts are logged. For failed ones, a warning is written to the server log with the username, IP, reason and, on lockout, the `blocked_until` time. If login notifications via the Telegram bot are enabled (`tgNotifyLogin` – "Login notification"), the administrator additionally receives the username, IP, and time of both successful and failed and blocked attempts.

Example: login notification in Telegram. With `tgNotifyLogin` enabled, after each attempt the administrator receives a message roughly like this:

```
Login notification
User: admin
IP: 203.0.113.45
Time: 2026-06-10 14:32:07
Status: success
```

For a blocked "IP + login" pair, the status will indicate that the attempt was rejected by the limiter.

2.4. Changing the administrator login and password

The **Settings** → **Account** section, the **"Administrator credentials"** tab. Fields:

Field	Text (RU)	Description
Current login	"Current login"	The current username. It must match the current login, otherwise the change is rejected.
Current password	"Current password"	The current password, for identity confirmation.
New login	"New login"	The new username. Cannot be empty.

Field	Text (RU)	Description
New password	"New password"	The new password. Cannot be empty.

The change is applied with the **"Confirm"** button and submitted to `POST /panel/setting/updateUser`.

Server logic and messages:

- If "Current login" does not match the actual one or "Current password" is incorrect — "An error occurred while changing the administrator credentials." with the explanation "Incorrect username or password".
- If the new login or new password is empty — the explanation "The new username and new password must not be empty".
- On success — "You have successfully changed the administrator credentials.". The password is stored as a bcrypt hash.

Example: changing credentials via the API. The request requires a valid session cookie (obtained at login) and confirmation of the current login/password:

```
curl -X POST https://panel.example.com:2053/my-secret/panel/setting/updateUser \
  -b 'session=YOUR_SESSION_COOKIE' \
  -H 'Content-Type: application/x-www-form-urlencoded' \
  --data 'oldUsername=admin&oldPassword=OldPassword&newUsername=root&newPassword=NewStrongPassword'
```

After success the current session is invalidated — you will need to log in again with the new credentials.

Important effects of changing credentials:

- **All existing sessions are invalidated** (the user's `login_epoch` counter is incremented), so after the change the panel automatically logs out and redirects to the login page — you need to log in again.
- If **2FA was enabled** at the time of the change, **it is automatically disabled** (the flag and the secret are reset). Two-factor authentication will have to be set up again after changing the login/password.

If 2FA is enabled, before the form is submitted the "Change credentials" window opens with the hint "Enter the code from the app to change the administrator credentials." — credentials can be changed only by confirming the current 2FA code.

2.5. Secret path (URI path / webBasePath) and panel port

These parameters are located in the **Settings** → **Panel** section and directly affect the panel's "stealth" and accessibility. They take effect after saving and **restarting the panel**.

Field	Text (RU)	Default value	Description
Panel port	"Panel port" (<code>panelPort</code>), hint "The port the panel runs on"	2053	The TCP port of the web interface.

Field	Text (RU)	Default value	Description
URI path	"URI path" (<code>panelUrlPath</code>), hint "Must start with '/' and end with '/'"	/	The secret base path (<code>webBasePath</code>). The panel is accessible only at it (for example, <code>/my-secret/</code>).
Panel management IP address	"Panel management IP address" (<code>panelListeningIP</code>), hint "Leave empty to connect from any IP"	empty	The address the panel listens on. Empty = all interfaces.
Panel domain	"Panel domain" (<code>panelListeningDomain</code>), hint "Leave empty to connect from any domains and IPs."	empty	Restricts access by domain (Host).
Path to the panel certificate's public key	<code>publicKeyPath</code> , hint "Enter the full path starting with '/'"	empty	The TLS certificate for HTTPS access to the panel.
Path to the panel certificate's private key	<code>privateKeyPath</code> , same hint	empty	The TLS private key.

Behavior of the base path (`webBasePath`):

- The value is normalized automatically: if it does not start with `/` , the character is added at the beginning; if it does not end with `/` , one is added at the end. So in practice the path is always of the form `/.../` .
- The base path applies to the panel itself, to the assets, and to the session cookie (the cookie is issued only for this path).

Security recommendations (the "Security warnings" section): the panel itself shows warnings if the configuration is "too public":

- "The panel runs over plain HTTP — set up TLS for production."
- "The default port 2053 is widely known — change it to a random one."
- "The default base path `/` is widely known — change it to a random one."

In other words, for a production server you should set a **non-standard port**, a **non-trivial URI path**, and a **TLS certificate**.

Example: a "stealth" panel configuration for production. In the **Settings** → **Panel** section set roughly these values:

Field	Value
Panel port	<code>34571</code> (random, instead of 2053)
URI path	<code>/aXf9Qm2/</code> (non-trivial, starts and ends with <code>/</code>)
Path to the panel certificate's public key	<code>/etc/letsencrypt/live/panel.example.com/fullchain.pem</code>
Path to the panel certificate's private key	<code>/etc/letsencrypt/live/panel.example.com/privkey.pem</code>

After saving and restarting, the panel will be reachable only at `https://panel.example.com:34571/aXf9Qm2/`, and the security warnings will disappear.

2.6. Session lifetime (timeout)

The **"Session duration"** field (`sessionMaxAge`) is located among the panel/interval settings.

Field	Text (RU)	Default value	Unit	Description
Session duration	"Session duration", hint "The session duration in the system (value: minute)"	360	minutes	The lifetime of the administrator session cookie.

Behavior:

- The value is specified in **minutes** (default 360 minutes = 6 hours) and is converted to seconds when configuring the cookie.
- If the value is **greater than 0**, the session cookie is given a corresponding `MaxAge`. After this period expires, the cookie stops being valid and on the next request the user gets "Session expired. Please log in again".
- The session also becomes invalid prematurely when credentials are changed or 2FA is enabled for the first time (via the `login_epoch` mechanism, see 2.4 and 2.2) and on an explicit logout (`POST /logout`).
- The session cookie is marked `HttpOnly`, with the `SameSite=Lax` policy; the `Secure` flag is set on direct HTTPS access to the panel.

In addition to the timeout itself, there is a related notification: **"Session expiry notification lead time"** (`expireTimeDiff`, hint "Receive a notification about session expiry before the threshold is reached (value: day)", default `0`) — it allows receiving a warning in advance.

2.7. LDAP (synchronization and authentication)

The LDAP section provides two capabilities: (1) authenticating the administrator's login via LDAP if the local password did not match, and (2) periodically synchronizing the state of clients (the enabled/disabled VLESS flag) from the directory.

How it is used at login: the server first checks the local bcrypt password hash. If it **did not match** and LDAP is enabled, the panel attempts to authenticate the user in the directory: when a `Bind DN` is set, a service bind is performed, then the user record is searched by the filter and attribute, and a bind under the found DN with the entered password is attempted. Success means a login. (After a successful LDAP authentication, if 2FA is enabled, the TOTP code is still verified.)

Section fields:

Field	Text (RU)	Default value	Description
Enable LDAP synchronization	"Enable LDAP synchronization" (<code>enable</code>)	false	The master switch for the LDAP integration.
LDAP host	"LDAP host" (<code>host</code>)	empty	The address of the LDAP server.

Field	Text (RU)	Default value	Description
LDAP port	"LDAP port" (<code>port</code>)	389	The port. For LDAPS usually 636.
Use TLS (LDAPS)	"Use TLS (LDAPS)" (<code>useTls</code>)	false	When enabled, the <code>ldaps://</code> scheme is used with server certificate verification (without skipping the check).
Bind DN	"Bind DN" (<code>bindDn</code>)	empty	The DN of the service account for the initial bind/search. If empty – no bind is performed (anonymous search).
Bind password	hints: "Configured; leave empty to keep the current password." / "Not configured." / "Configured – enter a new value to replace"	empty	The password for the <code>Bind DN</code> . Stored separately; to keep the previous one, the field is left empty.
Base DN	"Base DN" (<code>baseDn</code>)	empty	The root of the subtree in which the search is performed (the search is recursive, over the entire subtree).
User filter	"User filter" (<code>userFilter</code>)	<code>(objectClass=person)</code>	The LDAP filter for selecting accounts. During authentication, the login is substituted into the filter with escaping.
User attribute (username/email)	"User attribute (username/email)" (<code>userAttr</code>)	<code>mail</code>	The attribute matched against the login/client identifier (for example, <code>mail</code> or <code>uid</code>).
VLESS flag attribute	"VLESS flag attribute" (<code>vlessField</code>)	<code>vless_enabled</code>	The attribute that determines whether the client's VLESS access should be enabled.
General flag attribute (opt.)	"General flag attribute (opt.)" (<code>flagField</code>), hint "If set, overrides the VLESS flag – e.g. <code>shadowInactive</code> ."	empty	If set, it is used instead of <code>vless_enabled</code> .
Truthy values	"Truthy values" (<code>truthyValues</code>), hint "Comma-separated; default: <code>true,1,yes,on</code> "	<code>true,1,yes,on</code>	The list of flag-attribute values treated as "enabled".
Invert flag	"Invert flag" (<code>invertFlag</code>), hint "Enable when the attribute means 'disabled' (e.g. <code>shadowInactive</code>)."	false	Inverts the meaning of the flag.
Sync schedule	"Sync schedule" (<code>syncSchedule</code>), hint "A cron-like string, e.g. <code>@every 1m</code> "	<code>@every 1m</code>	The synchronization frequency in a cron-like format.

Field	Text (RU)	Default value	Description
Inbound tags	"Inbound tags" (<code>inboundTags</code>), hint "The inbounds on which LDAP synchronization may auto-create or auto-delete clients."	empty	Restricts which inbounds allow auto-operations. If there are no inbounds: "No inbounds found. Create an inbound first."
Auto-create clients	"Auto-create clients" (<code>autoCreate</code>)	false	Create a client in the specified inbounds if it appeared in the directory.
Auto-delete clients	"Auto-delete clients" (<code>autoDelete</code>)	false	Delete a client if it disappeared from the directory.
Default volume (GB)	"Default volume (GB)" (<code>defaultTotalGb</code>)	0	The traffic limit for auto-created clients (0 = no limit).
Default term (days)	"Default term (days)" (<code>defaultExpiryDays</code>)	0	The validity period for auto-created clients (0 = unlimited).
Default IP limit	"Default IP limit" (<code>defaultIpLimit</code>)	0	The limit on the number of simultaneous IPs (0 = no limit).

Specifics of the synchronization flag logic: when reading the flag attribute (`flagField` , default `vless_enabled`), the value is considered "enabled" if it is in the list of truthy values; when inversion is enabled, the result is flipped to the opposite. The user attribute (`userAttr`) is used as the matching key (email/name) – records without a value for this attribute are skipped.

Security: it is recommended to enable **TLS (LDAPS)** so that bind passwords and verified passwords are not transmitted in plain text, and to use an account with the minimum necessary read permissions for the Bind DN.

Example: a typical LDAP synchronization configuration (Active Directory). Filling in the section fields for a directory where the access status is stored in a flag attribute and matching is done by email:

Field	Value
LDAP host	<code>ldap.example.com</code>
LDAP port	<code>636</code>
Use TLS (LDAPS)	<code>enabled</code>
Bind DN	<code>CN=svc-3xui,OU=Service,DC=example,DC=com</code>
Base DN	<code>OU=Users,DC=example,DC=com</code>
User filter	<code>(objectClass=person)</code>
User attribute (username/email)	<code>mail</code>
VLESS flag attribute	<code>vless_enabled</code>
Truthy values	<code>true,1,yes,on</code>

Field	Value
Sync schedule	@every 5m

With this setup, every 5 minutes the panel walks the `OU=Users` subtree, matches clients by `mail`, and enables/disables VLESS access based on the `vless_enabled` value.

3. Overview / Dashboard

The Dashboard ("Dashboard"; in the English interface — *Overview*) is the panel's start page. It shows the state of the server and the Xray process in real time. All metrics arrive from the server side. A background scheduler rebuilds the snapshot **every 2 seconds** and broadcasts it to all open tabs over WebSocket; once a minute the accumulated metric series are flushed to disk. The `GET /status` HTTP endpoint returns the last cached snapshot.

Below, each metric and each control on the page is explained.

3.1. General principles of data collection

- The snapshot is collected by the `gopsutil` library. If a particular reading fails, the field stays zero and a warning is written to the log (`get cpu percent failed`, `get uptime failed`, etc.) — this does not bring down the whole dashboard, the corresponding tile simply shows 0/N-A.
- "Instantaneous" rates (CPU %, network, disk I/O) are computed as the difference between the current and the previous snapshot divided by the interval in seconds. Therefore, on the first page load the rate values may be zero until a second reading has accumulated.
- The history can be viewed in the "System History" (*System History*) section — the charts are built from the same data series described below (see 3.12).

3.2. CPU

The "CPU" (*CPU*) tile shows the current processor load as a percentage, along with the parameters of the processor itself.

Metric	Description
CPU load, %	The share of busy processor time over the last interval. It is smoothed with an exponential moving average (EMA, coefficient <code>alpha = 0.3</code>) so that spikes do not jerk the indicator. The value is always clamped to the 0–100 % range. On the very first reading 0 is returned (initialization of the baseline point).
Logical Processors	The number of logical cores — that is, taking Hyper-Threading into account.
Physical cores	The number of physical cores.
Frequency	The processor's base frequency in MHz. It is queried lazily and cached: the first successful reading is stored, a retry is made no more often than once every 5 minutes, and the request itself is limited by a 1.5 s timeout (on some systems the frequency query responds slowly).

CPU load is computed algorithmically as follows: if a native platform implementation is available, it is used, otherwise it is computed from the deltas of the processor-time counters (busy / total). Guest and GuestNice time is excluded so as not to count it twice.

3.3. Memory (RAM)

The "Memory" (*RAM*) tile shows used and total. It is displayed as "used / total" and/or a fill percentage. The percentage is recorded into the history.

3.4. Swap

The "Swap" (*Swap*) tile shows used and total. If a swap file/partition is not configured (total = 0), the metric is zero; when there is no swap, 0 is written into the historical series.

3.5. Storage (Disk)

The "Storage" (*Storage*) tile shows used and total, meaning **only the root partition** / is taken into account. The fill percentage is written into the "Disk Usage" (*Disk Usage*) history. Separately, disk I/O is collected (read / write, bytes/s) as the delta of the counters over the interval – it is shown on the "Disk I/O" tab of the history.

3.6. System uptime

The "Uptime" (*Uptime*) metric. This is the time since the boot of **the entire server** (in seconds), not the uptime of the panel or Xray. The uptime of the Xray process is stored separately (see 3.9), as is the number of the panel's threads (in the translation – "Threads" / *Threads*).

3.7. System load (Load average)

The "System Load" (*System Load*) block – an array of three numbers [Load1, Load5, Load15]. Tooltip label: "System load average for the past 1, 5, and 15 minutes" (*System load average for the past 1, 5, and 15 minutes*). The history chart is called "System load average (1 / 5 / 15 min)". Values are written into the historical series separately: load1, load5, load15.

This is a standard Unix metric: the average number of processes in the run queue. As a reference point, compare it with the number of cores: a load that consistently exceeds the number of physical cores indicates overload.

3.8. Network: speed and total traffic volume

Only physical interfaces are counted. Virtual and tunnel interfaces are excluded: these are lo / lo0, as well as everything starting with loopback, docker, br-, veth, virbr, tun, tap, wg, tailscale, zt. The values are summed across all the remaining interfaces.

Overall Speed (*Overall Speed*) – the instantaneous speed, the delta of the counters over the interval:

Metric	Description
Upload (label "Upload" / <i>Upload</i>)	Outbound speed, bytes/s.
Download (label "Download" / <i>Download</i>)	Inbound speed, bytes/s.

Total Data (*Total Data*) – accumulated counters since the system start:

Metric	Description
Sent (label "Sent" / <i>Sent</i>)	Total bytes sent.
Received (label "Received" / <i>Received</i>)	Total bytes received.

Additionally, packet rates (packets/s) and total packet counters are collected — they are shown on the "Network Packets" (*Network Packets*) tab of the history. Network history series: `netUp` , `netDown` , `pktUp` , `pktDown` .

3.9. Server IP addresses

The "IP Addresses" (*IP Addresses*) block shows `IPv4` and `IPv6` . External addresses are determined through third-party services (`api4.ipify.org` , `ipv4.icanhazip.com` , `v4.api.ipinfo.io/ip` , `ipv4.myexternalip.com/raw` , `4.ident.me` , `check-host.net/ip` for IPv4, and analogous ones for IPv6). The list is tried in order until the first successful response; the timeout for each request is 3 s.

Specifics:

- The result is **cached** for the lifetime of the process: an address that has been successfully determined is not requested again.
- If no service responds, the field keeps `N/A` . For IPv6, on the first `N/A` , IPv6 requests are disabled altogether so as not to waste time on networks without IPv6.
- Next to it there is an "eye" button to hide/show the addresses — tooltip "Toggle visibility of the IP" (*Toggle visibility of the IP*). This is only a visual hide in the interface (for example, for screenshots); it does not affect the addresses themselves.

3.10. TCP/UDP connections

The "Connection Stats" (*Connection Stats*) block shows the total number of active TCP and UDP connections on the server (system-wide, not just Xray). The history chart is "Active Connections (TCP / UDP)" (*Active Connections*), series `tcpCount` , `udpCount` .

3.11. Xray status and process control

The "Xray" card shows the state of the Xray-core process and lets you control it.

States

Value	Label	Translation	When it is set
<code>running</code>	"Running"	<i>Running</i>	The Xray process is running.
<code>stop</code>	"Stopped"	<i>Stopped</i>	The process is not running and there is no recorded startup error.
<code>error</code>	"Error"	<i>Error</i>	The process is not running, but a startup error has been recorded. The error text is shown in a popover with the title "An error occurred while running Xray" (<i>An error occurred while running Xray</i>).
—	"Unknown"	<i>Unknown</i>	Displayed while the status has not yet been received.

Next to the status, the **Xray version** is displayed.

Control buttons

- **Stop** (*Stop*). Calls `POST /stopXrayService`. On success the panel broadcasts the new `stop` state over WebSocket and the notification "Xray service has been stopped" (*Xray service has been stopped*); on failure — the `error` state with text. Important: if the panel is reachable *through* Xray itself, stopping Xray may break the connection to the panel — with a direct connection to the panel there is no problem.
- **Restart** (*Restart*). Calls `POST /restartXrayService`. Before the action a confirmation is shown: "Restart xray?" with the explanation "Reloads the xray service with the saved configuration". On success — the `running` state and the notification "Xray service has been restarted successfully" (*Xray service has been restarted successfully*). The restart applies the current saved configuration — use it after changing settings.

Note. In this fork, full Start / Stop / Restart control has been added to the dashboard for all authorization types; the original 3x-ui UI has no separate "start" button — starting is done via a restart.

Xray log viewer button

The Xray card has a button that opens the Xray logs (*Logs*). It appears only when an access log is configured in the Xray configuration: the built-in viewer reads exactly that file, so without an access log the button is hidden. The button's visibility is tied to a separate `accessLogEnable` flag and no longer depends on the IP limit — the online list and the IP-address limit keep working even without an access log (see 8).

Choosing the Xray version

The "Version" (*Version*) section lets you switch Xray-core to a different release. The list of versions is loaded via `GET /getXrayVersion`:

- The source is the GitHub API of the `XTLS/Xray-core` repository (`/releases`). Requests are cached for **15 minutes**; on a GitHub failure the last successfully fetched list is returned so the picker does not become empty.
- Only releases of the form `X.Y.Z` and **not older than 26.4.25** make it into the list.

Tooltips: "Choose the version you want to switch to." (*Choose the version you want to switch to.*) and the warning "Choose carefully, as older versions may not be compatible with current configurations." (*Choose carefully, as older versions may not be compatible with current configurations.*).

Switching: `POST /installXray/:version`. The scenario:

Example. Switch to a specific Xray-core version (the session cookie must already be obtained via login):

```
curl -X POST 'https://panel.example.com:2053/xpanel/installXray/v25.6.8' \
  -b cookie.txt
```

Here `v25.6.8` is a tag from the list returned by `GET /getXrayVersion`. The version must be present in that list, otherwise the panel responds with a refusal.

1. The selected version is checked for presence in the current list of releases (otherwise – refusal).
2. Xray is stopped.
3. For the current OS and architecture, an archive `Xray-<os>-<arch>.zip` is downloaded from GitHub (amd64/64, arm64-v8a, arm32-v7a/v6/v5, 386/32, s390x are supported; for Windows – `xray.exe`). The size of the archive and of the binary is limited to 200 MB.
4. The binary is replaced atomically (via a temporary file + rename) and marked executable.
5. Xray is started again.

Before switching, the dialog "Do you really want to change the Xray version?" (*Do you really want to change the Xray version?*) is shown, with the description "This will change the Xray version to #version#". On success – the notification "Xray updated successfully" (*Xray updated successfully*).

3.12. Panel update (3X-UI)

The panel update check block. The data arrives via `GET /getPanelUpdateInfo`:

Field	Description
Current panel version	The version of the installed panel.
Latest panel version	The latest 3x-ui release fetched from GitHub.
Update available	An indication that the latest version is newer than the current one. If no update is needed – "Panel is up to date" / "Up to date" is shown.

The **"Update Panel"** button (*Update Panel*) launches `POST /updatePanel`. Tooltip: "This will update 3X-UI to the latest release and restart the panel service". Before launching – the confirmation "Do you really want to update the panel?" with the text "This will update 3X-UI to version #version# and restart the panel service".

Specifics and limitations:

- Self-update is supported **only on Linux** (on other operating systems an error is returned).
- The updater script is downloaded from the official repository (`raw.githubusercontent.com/MHSanaei/3x-ui/main/update.sh`, limit 2 MB) and run via `bash`, isolated through `systemd-run` where possible.
- On a successful launch "Panel update started" (*Panel update started*) is shown; if the update check failed – "Panel update check failed". During installation the warning "Installation in progress. Do not refresh the page" is displayed.

3.13. Updating geo files (GeoIP / GeoSite)

The button/dialog for updating the geo databases calls `POST /updateGeofile` (all files) or `POST /updateGeofile/:fileName` (a single file). The update works against a strict whitelist of names and sources:

File	Source
geoup.dat , geosite.dat	Loyalsoldier/v2ray-rules-dat (latest)
geoup_IR.dat , geosite_IR.dat	chocolate4u/Iran-v2ray-rules (latest)
geoup_RU.dat , geosite_RU.dat	runetfreedom/russia-v2ray-rules-dat (latest)

Behavior:

- The file name is validated: `..`, slashes, and absolute paths are forbidden; only `[a-zA-Z0-9._-]+.dat` is allowed. Files outside the whitelist are not downloaded.
- A conditional request `If-Modified-Since` is used: if the file has not changed on the source server (HTTP 304), it is not downloaded again, only the timestamp is updated.
- After downloading, Xray is **restarted** (to pick up the new databases).

Example. Update only the Russian geo databases without touching the other files:

```
curl -X POST 'https://panel.example.com:2053/xpanel/updateGeofile/geoup_RU.dat' -b cookie.txt
curl -X POST 'https://panel.example.com:2053/xpanel/updateGeofile/geosite_RU.dat' -b cookie.txt
```

To update every file from the whitelist at once, call `POST /updateGeofile` without a file name.

- Dialogs: "Do you really want to update the geo file?" with "This will update the file #filename#" for a single file, and "This will update all geo files" for the "Update all" button. Success — "Geo files updated successfully".

3.14. Database backup and restore

The "Backup & Restore" (*Backup & Restore*) block. The behavior depends on the DBMS in use (SQLite by default or PostgreSQL).

Exporting the database (Backup)

The "Export database" / "Back Up" button (*Back Up*) calls `GET /getDb`. The file is returned as an attachment:

- **SQLite**: first a checkpoint (WAL flush) is performed, then the `x-ui.db` file is downloaded. Tooltip: "Click to download a .db file containing a backup of your current database...".
- **PostgreSQL**: a dump `x-ui.dump` in the custom format is downloaded (`pg_dump --format=custom --no-owner --no-privileges`). The PostgreSQL client tools must be installed on the server; otherwise — an error about the missing `pg_dump`.

Importing the database (Restore)

The "Import database" / "Restore" button (*Restore*) uploads a file via `POST /importDB` (form field `db`). Tooltip: "Click to select and upload a .db file... to restore the database from a backup".

The scenario for **SQLite** is safe, with rollback:

1. The file is checked for the SQLite format and saved to a temporary file, then integrity is verified.
2. Xray is stopped, the current DB is closed and renamed to `*.backup` (the fallback).
3. The new file takes the place of the working DB, then initialization and migration are performed. If something goes wrong — the fallback is restored.
4. Xray is started again.

For **PostgreSQL**, a `.dump` is uploaded (the PGDMP signature is checked) and applied via `pg_restore --clean --if-exists --single-transaction ...`. The tooltip warns directly: "This will replace all current data".

Messages: "Database imported successfully", "An error occurred while importing the database", "...while reading the database", "...while retrieving the database".

Migration file (between SQLite and PostgreSQL)

The "Download Migration" button (*Download Migration*) calls `GET /getMigration` and produces a portable export for running the panel on a different DBMS:

- On **SQLite**, `x-ui.dump` (a textual SQL dump) is downloaded.
- On **PostgreSQL**, `x-ui.db` is downloaded — a ready-made SQLite database assembled from the PostgreSQL data.

3.15. Additional interface elements

- **Online clients indicator.** The dashboard maintains the `online` series (*Online Clients* / "Online Clients") — the number of clients with an active connection. It is computed while Xray is running (0 otherwise) and recorded into the history on the same 2-second tick. The chart is the "Online" tab.
- **System History (charts).** The "Charts" button/section → "System History" with tabs: "Bandwidth", "Packets", "Disk I/O", "Online", "Load", "Connections", "Disk Usage". The data is pulled via `GET /history/:metric/:bucket`; the allowed aggregation intervals (bucket, sec) are **2, 30, 60, 120, 180, 300**, and up to 60 points arrive per tab. The allowed metrics are: `cpu`, `mem`, `swap`, `netUp`, `netDown`, `pktUp`, `pktDown`, `diskRead`, `diskWrite`, `diskUsage`, `tcpCount`, `udpCount`, `online`, `load1`, `load5`, `load15`. The label "Last 2 minutes" corresponds to bucket = 2 (real-time mode).

Example. Fetch the CPU-load series for the last ~2 minutes (bucket = 2 s, up to 60 points) and the same series aggregated over 5 minutes (bucket = 300 s):

```
curl 'https://panel.example.com:2053/xpanel/history/cpu/2' -b cookie.txt
curl 'https://panel.example.com:2053/xpanel/history/cpu/300' -b cookie.txt
```

The metric can be replaced with any allowed one (`mem`, `netUp`, `tcpCount`, `load1`, etc.). A bucket outside the list **2, 30, 60, 120, 180, 300** is rejected.

- **Xray metrics** — a separate block with Xray's memory consumption and garbage collection (series `xrAlloc`, `xrSys`, `xrHeapObjects`, `xrNumGC`, `xrPauseNs`) and the "Observatory" (the state of outbound connections). They work only if the `metrics` block is set in the Xray configuration (`listen 127.0.0.1:11111`, tag `metrics_out`); otherwise "Xray metrics endpoint is not configured" is shown.

Example of the block that enables the Xray metrics tile. The Xray settings must contain both `metrics` (with a tag) and an inbound that listens on that tag:

```
{
  "metrics": {
    "tag": "metrics_out"
  },
  "inbounds": [
    {
      "listen": "127.0.0.1",
      "port": 11111,
      "protocol": "dokodemo-door",
      "settings": { "address": "127.0.0.1" },
      "tag": "metrics_out"
    }
  ]
}
```

The `127.0.0.1:11111` address is intentionally not exposed externally – the panel polls it locally.

- **Dark theme toggle.** It is located in the common menu/header, not in the dashboard itself. Options: "Theme" (*Theme*) with the options "Dark" and "Ultra Dark" (*Ultra Dark*). This is a purely visual appearance setting; it does not affect the panel's operation.
 - **Other links** in the dashboard's surroundings (from the menu/bottom bar): "Logs", "Config" – viewing the resulting Xray JSON (`GET /getConfigJson`), "Documentation".
-

4. Inbounds: creation and common parameters

The **Inbounds** section is the list of all Xray entry points through which clients connect. Each inbound stores both "panel" fields (remark, traffic limit, reset schedule) and raw JSON blocks of the Xray configuration (settings , streamSettings , sniffing).

Creation is done via the **Add Inbound** button, editing via **Modify Inbound**. Both operations are sent to the API endpoints `POST /add` and `POST /update/:id`.

Below are all form fields that are **not** related to the settings of a specific protocol (clients, encryption, REALITY/TLS) and **not** related to transport/stream (the **Stream** and **Security** tabs) – those are the subjects of separate sections.

4.1. Common form fields

Remark

Parameter	Value
Field	remark
Type	string
Default	empty

A human-readable name for the inbound, shown in the list and in dialog titles ("Delete inbound "{remark}"?", etc.). The field label is **"Remark"**. It does not affect Xray operation and is only for administrative convenience; it is recommended to set unique, meaningful names, since they are inserted into the names of exported files and into confirmations of bulk operations.

Protocol

Parameter	Value
Field	protocol
Label	"Protocol"
Validation	required, oneof=vmess vless trojan shadowsocks wireguard hysteria http mixed tunnel tun

A drop-down list of the inbound's protocol. The allowed values are:

Value	Note
vmess	
vless	
trojan	

Value	Note
shadowsocks	
wireguard	
hysteria	Hysteria v2 is hysteria with streamSettings.version = 2 ; there is no separate protocol
http	
mixed	socks/http on a single port
tunnel	
tun	accepted by the validator, no separate protocol constant

The field is required (`required`). The choice of protocol determines which client settings fields and which transport will be available (see the protocol-specific sections).

Important: when saving, the service normalizes `streamSettings` . Transport settings are kept only for the protocols `vmess` , `vless` , `trojan` , `shadowsocks` , `hysteria` ; for the others (`http` , `mixed` , `tunnel` , `wireguard` , `tun`) the `streamSettings` field is **forcibly cleared**.

Listen IP

Parameter	Value
Field	<code>listen</code>
Type	string
Default	empty → Xray listens on <code>0.0.0.0</code> (all IPs)

The IP address on which the inbound accepts connections. The field hint:

"Leave blank to listen on all IP addresses."

When generating the Xray configuration, an empty value is replaced with `0.0.0.0` . In addition to an IP, the field accepts a **Unix socket path** — hint:

"You can also enter a Unix socket path (e.g. `/run/xray/in.sock`), or an abstract socket name prefixed with `@` (e.g. `@xray/in.sock`), to listen on a socket instead of a TCP port — set Port to 0 in that case."

Thus the field accepts two Unix socket forms: a filesystem path (`/run/xray/in.sock`) and an abstract socket name prefixed with `@` (`@xray/in.sock`). In both cases set `Port` to `0` .

You change this field when you need to restrict the inbound to a single interface (for example, `127.0.0.1` for an inbound that works only as a fallback target behind Nginx), or when the inbound listens on a Unix socket.

Example. An inbound that listens only on the local interface (a typical fallback target behind Nginx), and one that listens on a Unix socket:

```
listen = 127.0.0.1    port = 8443
listen = /run/xray/in.sock    port = 0
```

Port

Parameter	Value
Field	port
Label	"Port"
Validation	gte=0, lte=65535
Default	– (set by the user)

The TCP/UDP listening port. Values from 0 to 65535 are allowed. The value 0 is used only in combination with listening on a Unix socket (see above).

When saving, the service checks for a port conflict: two inbounds cannot simultaneously occupy overlapping listen:port for the same transport (TCP/UDP). The transport is derived from the protocol and streamSettings / settings: for example, hysteria and wireguard always occupy UDP, kcp / quic – UDP, and most others – TCP. On a conflict, saving is rejected with an error.

The Xray tag (Tag , unique) is generated automatically from the port and transport in the format in-<port>-<tcp|udp|tcpudp|any> ; for an inbound deployed on a node, the prefix n<nodeId>- is added. On a collision, -2 , -3 , etc. is appended to the tag. The user usually does not edit the tag.

Total traffic (GB)

Parameter	Value
Field	total (in bytes)
Label	"Total flow"
Default	0

The total traffic limit of the inbound. In the form, the value is entered in gigabytes; in the database it is stored in bytes. The field hint:

"= Unlimited. (unit: GB)."

That is, 0 means unlimited. This is a limit at the level of the entire inbound (not individual clients); the actual consumed traffic is stored in the up (sent) and down (received) fields and compared against total .

Expiry date / Duration

Parameter	Value
Field	<code>expiryTime</code> (Unix timestamp)
Label	"Expiry date" (Duration)
Default	empty / <code>0</code>

The validity period of the inbound. The hint:

"Leave blank to never expire."

An empty value (`0`) means the inbound never expires. The value is stored as a Unix timestamp; the form allows you to specify either a specific date or a period in days (a relative countdown from the current moment – the English field label is *Duration*).

Enabled

Parameter	Value
Field	<code>enable</code>
Label	"Enable" (Enabled)
Default	set at creation

The inbound's active flag. Toggling this flag in the list is handled by a separate "lightweight" endpoint `POST /setEnabled/:id`, rather than by a full update – this is done deliberately to avoid re-serializing the entire `settings` block (all clients) on every click of the toggle on an inbound with thousands of clients. When an inbound is disabled, it is removed from the running Xray; when enabled, it is added back.

Node / Deploy to

Parameter	Value
Field	<code>nodeId</code>
Label	"Deploy to" , "Local panel"
Default	empty (local panel)

A choice of where the inbound physically runs: on the local panel or on one of the registered nodes. An implementation detail: `nodeId = 0` is normalized to `nil`, since `0` is not a valid node id but an artifact of form binding; `nil / 0` means the local panel. When saving an inbound on an offline node, a toast is possible – the change is synchronized when the node reconnects.

Share address strategy

Parameter	Value
Field	strategy + (optional) custom address
Label	"Share address strategy"
Default	"Inbound listen"

A drop-down list that controls which address is inserted into this inbound's **exported share links and QR codes**. The values:

Value	Label	What is inserted
node	"Node address"	the address of the node on which the inbound runs
listen	"Inbound listen"	the listen address of the inbound itself
custom	"Custom"	a custom address from the " Custom share address " field

When "**Custom**" is selected, the "**Custom share address**" field appears; enter a host or IP **without a scheme or port** (the value is validated). The "**Node address**" option is shown in the list only if there is an enabled node on which this inbound can run; otherwise it is hidden and the value is coerced to "**Inbound listen**".

This strategy affects **only** the direct share links and QR codes. It does **not** affect subscription output — there the address is still resolved by the usual panel logic.

4.2. Sniffing

The **Sniffing** tab edits the `sniffing` block of the Xray configuration, which is stored as raw JSON. Sniffing allows Xray to "peek" at the real domain name/protocol inside a connection for routing purposes.

Subfield	Label	Purpose
enabled	(tab toggle)	Enables/disables sniffing for the inbound
destOverride	—	The list of protocols for which the destination address is intercepted: <code>http</code> , <code>tls</code> , <code>quic</code> , <code>fakedns</code>
metadataOnly	"Metadata only"	Use only the connection's metadata, without reading the payload
routeOnly	"Route only"	Apply the sniffing result only for routing, without rewriting the destination address
domainsExcluded	"Excluded domains"	Domains excluded from sniffing
(excluded IPs)	"Excluded IPs"	IP addresses excluded from sniffing

- **destOverride** — the set of sniffers: `http` (determines the domain from the HTTP Host header), `tls` (from the SNI), `quic` (from the QUIC ClientHello), `fakedns` (matching against the FakeDNS pool). Usually `http` and `tls` are enabled to determine the domain.

Example of a `sniffing` block (determine the domain from HTTP and TLS, use the result for routing only, without rewriting the destination):

```
{
  "enabled": true,
  "destOverride": ["http", "tls"],
  "routeOnly": true,
  "domainsExcluded": ["courier.push.apple.com"]
}
```

- **`metadataOnly`** — when enabled, Xray does not read the contents of the first packet and relies only on metadata; useful to avoid breaking protocols whose data cannot be "peeked" at.
- **`routeOnly`** — the sniffing result is used only by routing rules; the connection address in the outbound is not rewritten to the recognized domain.

Note: the panel stores `sniffing` as an opaque JSON block and adds nothing to it when saving — all default values for these checkboxes are formed on the client-application side. The raw block can be edited via the "Inbound JSON" section (see below).

4.3. Allocate (port allocation strategy)

The `allocate` block in `streamSettings` controls how Xray allocates listening ports. This is part of the Xray configuration; the panel stores and passes it as part of the inbound's `streamSettings` /JSON. Parameters (per Xray-core terminology):

Subfield	Purpose	Values / default
<code>strategy</code>	Port allocation strategy	<code>always</code> — always listen on the specified port (default); <code>random</code> — periodically change the listened ports within a range
<code>refresh</code>	Port change interval (minutes) for <code>random</code>	an integer number of minutes (5 is recommended; minimum — 2)
<code>concurrency</code>	How many ports to keep open simultaneously for <code>random</code>	an integer (default 3; no more than a third of the width of the port range)

`strategy: always` keeps the inbound on a single port (the standard mode). `strategy: random` is needed for anti-blocking scenarios, where the inbound periodically "hops" across a port range; in that case `refresh` and `concurrency` make sense. You should change these values only when deliberately using the random-port mode.

Example of an `allocate` block in `streamSettings` (random-port mode: keep 3 ports open, change them every 5 minutes):

```
{
  "allocate": {
    "strategy": "random",
    "refresh": 5,
    "concurrency": 3
  }
}
```

For this to work, the inbound's `port` is set as a range (for example, `20000–20100`).

4.4. External Proxy

The **External Proxy** field relates to the settings for generating invitation links and is stored in the inbound's `streamSettings`. It specifies a list of alternative external addresses (host/port, optionally with forced TLS – **"Force TLS"**) that are inserted into client links instead of the inbound's real `listen:port`.

It is used when clients should connect not directly to the server but through an external proxy/reverse/CDN: in that case the public address of such a frontend is specified in the shared links. This does not affect Xray's connection-accepting process itself – it is "cosmetics" of the generated links. Related form fields: **"Force TLS"**, **"Fingerprint"**, and a label for each entry.

4.5. Fallbacks

The **Fallbacks** section defines rules for redirecting connections that did not match any of the inbound's clients. It is available for a master inbound on a TLS transport (VLESS/Trojan TCP-TLS). It is managed via the endpoints `GET /:id/fallbacks` / `POST /:id/fallbacks`.

The section hint:

"When a connection on this inbound does not match any client, it is redirected elsewhere. Select a child inbound below to auto-fill the routing fields (SNI / ALPN / Path / xver) from its transport, or leave the selection empty and set Dest directly (for example, 8080 or 127.0.0.1:8080) to redirect to an external server such as Nginx. Each child inbound must listen on 127.0.0.1 with security=none."

The fallbacks section is shown only for a VLESS/Trojan inbound over RAW (TCP) with TLS or REALITY security. A new inbound starts at `security=none`, so the section may at first look as if it is missing. In that state (VLESS/Trojan, RAW/TCP, security not yet configured), an inline hint is shown instead of the section: fallbacks become available once TLS or Reality is selected on the **Security** tab.

Fallback row fields

Field	Default	Description
(child inbound)	—	Selection of the child inbound (label "Pick inbound"). If selected, the Name/Alpn/Path/Dest fields may be auto-filled from its transport
Name	empty (= any)	Match condition on the name (SNI/name). The "any" label – "any"

Field	Default	Description
Alpn	empty	Match condition on ALPN
Path	empty	Match condition on path (for the WS/HTTP transports of the child inbound)
Dest	auto	Where to redirect. Placeholder "auto (child's listen:port)" . You can specify a port (<code>8080</code>) or <code>host:port</code> (<code>127.0.0.1:8080</code>)
Xver	<code>0</code>	PROXY protocol version ("Xver"): <code>0</code> — disabled, <code>1</code> or <code>2</code> — the corresponding PROXY protocol version
(order)	by position	The order in which the rules are applied; set with the Up/Down buttons

Save logic: the entire fallback list of the master is replaced atomically. A row that has neither a selected child inbound (`childId <= 0`) nor a specified `Dest` is **skipped**. If the selected child inbound equals the id of the master itself, it is zeroed out. When generating the final JSON: if `Dest` is empty, it is computed from the child inbound as `listen:port`, with `0.0.0.0 / :: / ::0` replaced with `127.0.0.1`; empty `name / alpn / path` fields do not end up in the output JSON; `xver` is added only if it is greater than 0.

Example of the resulting `settings.fallbacks` (traffic with `alpn=h2` goes to a WS target at path `/ws`, everything else to a local Nginx on port 8080):

```
{
  "fallbacks": [
    { "alpn": "h2", "path": "/ws", "dest": "127.0.0.1:2001", "xver": 1 },
    { "dest": 8080 }
  ]
}
```

The last row, without `name / alpn / path`, is the "default" rule that catches everything else.

Buttons and hints of the fallbacks section

- **"Add fallback"** — add a row; **"No fallbacks yet"** — the empty state.
- **"Quick add all matching" / "Add all"** — adds a fallback row for every matching inbound that is not yet connected. Result: "Added {n} fallback(s)" or "No new matching inbounds".
- **"Fill from child"** — re-pull the routing fields (SNI/ALPN/Path/xver) from the transport of the selected child inbound; after execution — "Filled from child".
- **"Edit routing fields" / "Hide advanced"** — show/hide the fine-grained fields of the row.
- The labels **"Routes when"** and **"Default — catches everything else"** explain the trigger condition of each row.

After fallbacks are saved, the server triggers an Xray restart so that the new `settings.fallbacks` take effect.

4.6. Periodic traffic reset

The **Traffic reset** block configures automatic reset of the inbound's traffic counters on a schedule. Description:

"Automatically reset the traffic counter at the specified intervals."

Parameter	Value
Field	<code>trafficReset</code>
Validation	<code>omitempty,oneof=never hourly daily weekly monthly</code>
Default	<code>never</code>
Companion field	<code>lastTrafficResetTime</code> – the timestamp of the last reset (label "Last reset")

Drop-down list:

Value	Label
<code>never</code>	"Never"
<code>hourly</code>	"Hourly"
<code>daily</code>	"Daily"
<code>weekly</code>	"Weekly"
<code>monthly</code>	"Monthly"

For each period a cron job is registered that runs on the corresponding schedule (`@hourly` , `@daily` , `@weekly` , `@monthly`). The job selects all inbounds with the given `trafficReset` and, for each, resets the counters of the inbound itself (`up=0` , `down=0`) **and** the traffic of all its clients. That is, a periodic reset affects both the inbound and its clients.

Example field value. To have the counters zeroed on the first day of each month, select **"Monthly"** in the form, which is stored as:

```
{ "trafficReset": "monthly" }
```

The value `never` (the default) disables auto-reset entirely.

4.7. Inbound JSON (advanced)

The **Inbound JSON sections** section gives direct access to the inbound's raw JSON blocks. Description:

"The full inbound JSON and individual editors for settings, sniffing and streamSettings."

The following editors are available:

Tab	Label	What it edits
All	"The full inbound object with all fields in a single editor"	the entire Inbound object
Settings	"Wrapper for the Xray settings block"	the <code>settings</code> field

Tab	Label	What it edits
Sniffing	"Wrapper for the Xray sniffing block"	the <code>sniffing</code> field
Stream	"Wrapper for the Xray stream block"	the <code>streamSettings</code> field

These fields are serialized as nested JSON objects: empty blocks are returned as `null`, and text that is not valid JSON is wrapped in a string so that data is not lost. Parsing errors on save are shown with the prefix **"Advanced JSON"**.

The "Inbound JSON" viewer, like the inbound import dialog, uses a full code editor with JSON syntax highlighting (instead of a plain text area): the configuration view is a highlighted read-only mode, while import is editable – which makes reading and editing the configuration easier.

4.8. Inbound actions: QR / Edit / Reset / Delete and statistics

The following actions are available in the list and in the inbound card (the **"Menu"** menu):

Traffic statistics

The inbound's aggregated traffic is displayed: **"Sent/received"** (the `up / down` fields), **"Total traffic"**, **"Total connections"**. The card also shows **"Created"**, **"Updated"**, **"Expiry date"**.

In the client summary on the inbounds page, status is determined depleted-first: clients that have expired or exhausted their traffic (and that the auto-job cleared `enable` for) fall into the **"Depleted/Ended"** status rather than the grey **"Disabled"** one, and are not counted twice. The classification matches the one shown in the client's own card and correctly accounts for clients attached to several inbounds.

QR code and copying links

- **"Details"** – expands the connection and subscription links.
- Client QR code: hint **"Click the QR code to copy"**.
- **"Copy link"** (*Copy URL*), **"Export links"**.

Edit

"Modify inbound" – opens the editing form (`POST /update/:id`). On update, the service re-reads the existing row, carries over the changed fields, regenerates the tag if necessary (if the old tag was auto-generated) and synchronizes the Xray runtime. Success – toast **"Inbound updated successfully"**.

Reset Traffic

"Reset traffic" – zeroes the `up / down` counters of this specific inbound (`POST /:id/resetTraffic` , sets `up=0` , `down=0`). Confirmation:

"Reset traffic for "{remark}"?" / "Resets the sent/received counters of this inbound to 0."

Resetting an inbound's traffic does **not** touch its clients' counters (there are separate "Reset clients' traffic" actions for those). After the reset, an Xray restart is initiated. Success — toast **"Inbound traffic reset"**. There is also a bulk variant — **"Reset traffic of all inbounds"** (`POST /resetAllTraffics`).

Delete

"Delete inbound" (`POST /del/:id`). Confirmation:

"Delete inbound "{remark}"?" / "The inbound and all its clients will be deleted. This action cannot be undone."

Deletion removes the inbound from the running Xray (with a restart if necessary). Success — toast **"Inbound deleted successfully"**. Bulk deletion — `POST /bulkDel` , with per-item reporting and no more than one Xray restart.

Other actions with inbound clients

The menu also provides: **"Clone"** (a copy of the inbound with a new port and an empty client list), **"Delete all clients"** (`POST /:id/deleteAllClients` — deletes all clients, the inbound itself is kept), **"Delete depleted clients"**, **"Attach/Detach clients"**, **"Import"/"Export inbounds"** (`POST /import`). Details of client operations belong to the section on clients.

5. Protocols

When creating an inbound, the first thing you choose is the **Protocol** ("Protocol"). The protocol determines which authentication and traffic-encryption method Xray-core will apply to this inbound, which set of fields in `settings` you will need to fill in, and also which transports (`network`) and security types (TLS / REALITY) are available for it.

The protocol field is set once when the inbound is created and **does not change during editing** (in the edit form the dropdown is locked). To switch the protocol, you must create a new inbound.

5.1. List of supported protocols

The server accepts the following set of values for the `Protocol` field:

```
oneof=vless vless trojan shadowsocks wireguard hysteria http mixed tunnel tun mtproto
```

Starting with version **3.3.0**, the value `mtproto` (Telegram proxy) was added to the list.

Config value	Purpose	Client model
<code>vless</code>	Main proxy protocol (default when creating an inbound)	Clients with a UUID, support for flow and post-quantum encryption
<code>vmess</code>	Classic Xray proxy protocol	Clients with a UUID and a <code>security</code> parameter
<code>trojan</code>	Proxy that masquerades as regular HTTPS	Clients with a password
<code>shadowsocks</code>	Shadowsocks proxy (including SIP022 / 2022-blake3)	A single user or several (2022)
<code>wireguard</code>	WireGuard inbound	Peers (not clients)
<code>hysteria</code>	Hysteria inbound (version 2 by default)	Clients with an <code>auth</code> token
<code>http</code>	Classic HTTP proxy (forward proxy)	user/pass accounts, no traffic accounting
<code>mixed</code>	Combined SOCKS + HTTP proxy	user/pass accounts
<code>tunnel</code>	Transparent forwarder (xray <code>dokodemo-door</code>)	No clients
<code>tun</code>	TUN interface (rendering of existing ones only)	No clients
<code>mtproto</code>	Telegram (MTPROTO) proxy, added in 3.3.0; served by a separate <code>mtg</code> process, not by Xray	No clients (access via a secret)

Note on `tun` : the value is kept in the list for compatibility and for **displaying** previously saved inbounds, but in the current backend version creating one is not recommended — support is considered deprecated. There is no point in creating new inbounds of this type.

Note on Hysteria 2: there is no separate "hysteria2" protocol. It is the `hysteria` protocol with the field `streamSettings.version = 2`. The `hysteria2://` link scheme is selected automatically when generating share links if the stream version equals 2.

Not all protocols support distribution across nodes. Only the following can be deployed to nodes: `vless`, `vmess`, `trojan`, `shadowsocks`, `hysteria`, `wireguard`. The protocols `http`, `mixed`, `tunnel`, `tun`, `mtproto` work only on the local panel.

5.2. Which protocols support TLS / REALITY / transport

The ability to enable a particular security layer and transport depends on the protocol and the chosen network (`streamSettings.network`):

Capability	Available for protocols	Allowed networks (<code>network</code>)
TLS	<code>vmess</code> , <code>vless</code> , <code>trojan</code> , <code>shadowsocks</code> (and always for <code>hysteria</code>)	<code>tcp</code> , <code>ws</code> , <code>http</code> , <code>grpc</code> , <code>httpupgrade</code> , <code>xhttp</code>
REALITY	<code>vless</code> , <code>trojan</code>	<code>tcp</code> , <code>http</code> , <code>grpc</code> , <code>xhttp</code>
flow (<code>xtls-rprx-vision</code>)	<code>vless</code> only	<code>tcp</code> only, with <code>security = tls</code> or <code>reality</code>
Stream / transport ("Transport" tab)	<code>vmess</code> , <code>vless</code> , <code>trojan</code> , <code>shadowsocks</code> , <code>hysteria</code>	—

For the protocols `http`, `mixed`, `tunnel`, `tun`, `wireguard`, the transport tab is unavailable — they have no Xray stream settings.

5.3. VLESS

Purpose: the main modern proxy protocol. It supports XTLS-Vision (`flow`), REALITY, and also post-quantum encryption at the VLESS level itself (the `decryption` / `encryption` fields). Used by default for new inbounds.

Fields of the `settings` block:

Field	Default value	Description
<code>clients</code>	<code>[]</code>	List of clients. Each has: <code>id</code> (UUID), <code>email</code> (required), <code>flow</code> , <code>limits</code> (<code>limitIp</code> , <code>totalGB</code> , <code>expiryTime</code>), <code>enable</code> , <code>tgId</code> , <code>subId</code> , <code>comment</code> , <code>reset</code>
<code>decryption</code>	<code>none</code>	Server-side decryption parameter. UI label: "Decryption"
<code>encryption</code>	<code>none</code>	The paired encryption parameter (goes into the client link). Label: "Encryption"
<code>fallbacks</code>	<code>[]</code>	List of fallbacks (see the section on fallbacks); available when <code>network = tcp</code> and <code>security = TLS</code> or <code>REALITY</code>
<code>testseed</code>	(4 numbers: 900, 500, 900, 256)	"Vision testseed" — 4 positive integers for XTLS-Vision padding. Applied only to clients with the <code>xtls-rprx-vision</code> flow, otherwise ignored

flow (xtls-rprx-vision)

flow is set **on the client**, not on the inbound, and takes one of three values:

Value	Meaning
"" (empty)	No XTLS-flow (default)
xtls-rprx-vision	XTLS-Vision — the recommended mode for VLESS over TCP+TLS/REALITY
xtls-rprx-vision-udp443	The same Vision, but with UDP/443 (QUIC) handling

The flow field is available for selection only when all of the following conditions are met: protocol `vless`, network = `tcp`, and security = `tls` or `reality`. The **Vision testseed** field is shown in the form only under the same conditions.

Decryption / Encryption (VLESS post-quantum authentication)

The decryption and encryption fields are authentication at the VLESS level itself (separate from transport TLS/REALITY). By default both equal `none`. In the form, three buttons are placed alongside them:

- **X25519 auth** ("X25519 auth") — generates a decryption / encryption pair based on X25519.
- **ML-KEM-768 auth** ("ML-KEM-768 auth") — the post-quantum variant (Post-Quantum).
- **Clear** — resets both fields back to `none`.

Below the buttons, a status line "Selected: {auth}" is shown, where the value is determined by the last segment of the encryption string: empty/ `none` → "None", a very long key (>300 characters) → ML-KEM-768, a short one → X25519, otherwise "Custom".

Technically, the buttons call `GET /panel/api/server/getNewVlessEnc` (key generation via `xray vlessenc`) and fill in **both** fields with paired values of the form `mlkem768x25519plus.native.<rtt>.<role>` (for example, `decryption = mlkem768x25519plus.native.600s.server-x25519`, `encryption = mlkem768x25519plus.native.0rtt.client-x25519`). The decryption parameter stays on the server, while encryption goes into the client link.

Important: when generating the inbound configuration for Xray, the panel strips out the extras: if encryption (which belongs to the client side) remains in settings, it is **cut out** of the server config. Only decryption remains on the server itself.

When to choose VLESS: this is the recommended default option for a new inbound, especially in combination with REALITY (without your own certificate) or with TLS + XTLS-Vision.

Example: the settings block of a VLESS inbound with one client and XTLS-Vision. The flow field lives on the client; decryption stays on the server:

```
{
  "clients": [
    {
      "id": "d342d11e-d424-4583-b36e-524ab1f0afa4",
      "email": "user1",
      "flow": "xtls-rprx-vision",
      "limitIp": 2,
      "totalGB": 0,
      "expiryTime": 0,
      "enable": true
    }
  ],
  "decryption": "none"
}
```

For a REALITY pairing, the matching `streamSettings` block (the "Transport" tab -> Security: REALITY) looks like this:

```
{
  "network": "tcp",
  "security": "reality",
  "realitySettings": {
    "dest": "www.microsoft.com:443",
    "serverNames": ["www.microsoft.com"],
    "privateKey": "<X25519 private key>",
    "shortIds": ["", "6ba85179e30d4fc2"]
  }
}
```

5.4. VMess

Purpose: the classic Xray proxy protocol. Authentication is by UUID; on the client, the payload encryption method (`security`) is additionally configured.

Fields of the `settings` block:

Field	Default value	Description
<code>clients</code>	<code>[]</code>	List of clients

Each VMess client (in addition to the common fields `email` , `limits` , `enable` , `tgId` , `subId` , `comment` , `reset`):

Client field	Default value	Description
<code>id</code>	—	Client UUID
<code>security</code>	<code>auto</code>	VMess payload encryption method. Allowed values: <code>aes-128-gcm</code> , <code>chacha20-poly1305</code> , <code>auto</code> , <code>none</code> , <code>zero</code>

The `security` value:

- `auto` — Xray selects the cipher itself depending on the platform (recommended);
- `aes-128-gcm`, `chacha20-poly1305` — fixed AEAD ciphers;
- `none` — no payload encryption (makes sense only over TLS);
- `zero` — no encryption and no payload authentication.

Historical compatibility: older records may have stored `security: ""` — when read, an empty string is coerced to `auto`. When generating the server config, the `security` field of VMess clients is **removed** from `settings`, since it is not required for the inbound.

When to choose VMess: for compatibility with older clients or existing configurations. For new deployments, VLESS is usually preferable.

5.5. Trojan

Purpose: a proxy that imitates regular HTTPS traffic. Authentication is by password. Like VLESS, it supports fallbacks and (with `network = tcp`) REALITY/TLS.

Fields of the `settings` block:

Field	Default value	Description
<code>clients</code>	<code>[]</code>	List of clients
<code>fallbacks</code>	<code>[]</code>	List of fallbacks (available with <code>network = tcp</code> and TLS/REALITY)

Each Trojan client has the key field:

Client field	Default value	Description
<code>password</code>	—	Client password (required, at least 1 character)
<code>email</code>	—	Unique client identifier

The remaining client fields are common (`limitIp`, `totalGB`, `expiryTime`, `enable`, `tgId`, `subId`, `comment`, `reset`).

When to choose Trojan: when you need masquerading as HTTPS on port 443, including with fallbacks to a web server (Nginx) for unsolicited connections.

Example: a Trojan `settings` block with a fallback to a local web server. Unsolicited connections (without a valid password) are routed to Nginx listening on `127.0.0.1:8080`:


```
{
  "clients": [
    { "password": "S3cret-Pass-1", "email": "user1" }
  ],
  "fallbacks": [
    { "dest": "127.0.0.1:8080" }
  ]
}
```

Fallbacks require `network = tcp` and Security = TLS or REALITY; otherwise the fallbacks field is unavailable.

5.6. Shadowsocks

Purpose: a lightweight Shadowsocks proxy. It supports both legacy AEAD ciphers and the new SIP022 methods (2022-blake3-*). It can operate in single-user or multi-user mode.

Fields of the `settings` block:

Field	Default value	Description
<code>method</code>	<code>2022-blake3-aes-256-gcm</code>	Inbound encryption method. UI label: "Encryption method"
<code>password</code>	<code>""</code>	Inbound password (for 2022 methods it is generated automatically to fit the chosen method)
<code>network</code>	<code>tcp,udp</code>	Transport. Label: "Network". Options: <code>tcp,udp</code> (TCP,UDP), <code>tcp , udp</code>
<code>clients</code>	<code>[]</code>	List of clients
<code>ivCheck</code>	<code>false</code> (off)	The "ivCheck" toggle – protection against IV reuse

Encryption methods (`method`)

The allowed set:

Method	Category
<code>aes-256-gcm</code>	Legacy AEAD
<code>chacha20-poly1305</code>	Legacy AEAD
<code>chacha20-ietf-poly1305</code>	Legacy AEAD
<code>xchacha20-ietf-poly1305</code>	Legacy AEAD
<code>2022-blake3-aes-128-gcm</code>	SS 2022 (recommended)
<code>2022-blake3-aes-256-gcm</code>	SS 2022 (default)
<code>2022-blake3-chacha20-poly1305</code>	SS 2022, single-user

The panel's logic by method:

- **2022 methods** (`2022-blake3-*`) are considered "SS 2022". The method `2022-blake3-chacha20-poly1305` is **single-user** (multi-user is not supported); the other 2022 methods allow multiple clients. The password field (with a generation button that adjusts the key length to the method) is shown in the form precisely for the 2022 methods.
- **Legacy ciphers** (`aes-*` , `chacha20-*`) work on the classic "one method + one password" scheme.

Normalization before launching Xray: for legacy ciphers, each client must carry a `method` matching the inbound's method (otherwise Xray fails with "unsupported cipher method:"). For 2022 methods, the opposite – the client's `method` field must be **empty** (otherwise Xray rejects the inbound with "users must have empty method"). The panel brings the data into order itself when the method is switched.

When to choose Shadowsocks: for simple deployments without TLS masquerading; the modern choice is the 2022-blake3 methods.

Example: a Shadowsocks `settings` block for a 2022-blake3 method (multi-user mode). The inbound has its own password (a base64 key of the required length), each client has its own password, and the client's `method` field is **empty**:

```
{
  "method": "2022-blake3-aes-256-gcm",
  "password": "d2hhdGV2ZXItMzItYnl0ZS1iYXNlNjQta2V5LWlcmU=",
  "network": "tcp,udp",
  "clients": [
    {
      "email": "user1",
      "password": "Y2xpZW50LWtleS0zMilieXRlcy1iYXNlNjQtaGVyZQ==",
      "method": ""
    }
  ]
}
```

For legacy ciphers (`aes-256-gcm` , etc.) it is the opposite: a single password on the inbound, and the client's `method` must match the inbound's method.

5.7. Dokodemo-door / Tunnel (transparent forwarder)

Purpose: a transparent forwarder (in the panel – the `tunnel` protocol, which implements `dokodemo-door` behavior). It accepts traffic and forwards it to a specified address/port, without authentication or clients.

Fields of the `settings` block:

Field	Default value	Description
<code>rewriteAddress</code>	(none)	"Rewrite address" – the target address to which traffic is redirected
<code>rewritePort</code>	(none)	"Rewrite port" – the target port (0–65535)

Field	Default value	Description
allowedNetwork	tcp,udp	"Allowed network". Options: tcp,udp , tcp , udp
portMap	{}	"Port map" — a port→port map (Record<string,string>)
followRedirect	false (off)	"Follow redirect" — use the original destination address from the intercepted connection

Transport tab for Tunnel: an inbound of this type has a **"Transport"** tab limited to `sockopt` configuration — which is enough for **TProxy** mode (transparent proxying/redirect via `sockopt.tproxy`). The transport (`network`) dropdown and the "Security" tab are hidden for Tunnel, since TLS/REALITY are not supported by this type.

When to choose it: for transparent proxying/port forwarding to internal services.

5.8. SOCKS / HTTP (the `mixed` protocol)

In this build there is no separate `socks` protocol — SOCKS and HTTP proxies are combined into the `mixed` protocol (combined SOCKS + HTTP). In addition, there is a separate pure `http` proxy.

5.8.1. Mixed (SOCKS + HTTP)

Fields of the `settings` block:

Field	Default value	Description
auth	password	"Auth" — the authentication mode. Options: password (by login/password) or noauth (no authorization)
accounts	(optional)	"Accounts" — a list of user/pass pairs. With <code>auth = noauth</code> , the field is not written to the config
udp	false (off)	The "UDP" toggle — UDP support via SOCKS
ip	127.0.0.1	"UDP IP" — the local address for UDP associations. The field is shown only when <code>udp</code> is enabled

Accounts are added with the "Add" button; when added, a random login (8 characters) and password (12 characters) are generated, which can be edited.

5.8.2. HTTP (pure proxy)

Purpose: a classic HTTP forward proxy. At the Xray level it does not track clients as "billing" ones (no email/limits) — there is only a list of accounts.

Fields of the `settings` block:

Field	Default value	Description
<code>accounts</code>	<code>[]</code>	"Accounts" — a list of user/pass pairs (both fields required)
<code>allowTransparent</code>	<code>false</code> (off)	"Allow transparent" — forward requests with the original Host header

When to choose SOCKS/HTTP: for local or service proxy access without complex masquerading. `mixed` is convenient in that a single port serves both SOCKS and HTTP clients.

5.9. WireGuard (inbound)

Purpose: a WireGuard inbound. Unlike proxy protocols, it does not deal with "clients" — instead, **peers** (the devices that the server accepts) are configured. Transport and TLS/REALITY are not applicable to it.

Fields of the `settings` block:

Field	Default value	Description
<code>secretKey</code>	—	The server's private key (required). A generation button is next to it; the public key is derived automatically (a read-only field)
<code>mtu</code>	(optional)	The interface MTU
<code>noKernelTun</code>	<code>false</code> (off)	"No-kernel TUN" — use a userspace TUN instead of the kernel one
<code>workers</code>	(optional)	"Workers" — the number of worker threads
<code>domainStrategy</code>	(optional)	"Domain Strategy" — the domain-resolution strategy: <code>ForceIP</code> , <code>ForceIPv4</code> , <code>ForceIPv4v6</code> , <code>ForceIPv6</code> , <code>ForceIPv6v4</code>
<code>peers</code>	<code>[]</code>	List of peers

Fields of each peer:

Peer field	Default value	Description
<code>privateKey</code>	(optional)	The client's private key — stored so the panel can render the config for the user (only on inbound peers)
<code>publicKey</code>	—	The peer's public key (required)
<code>preSharedKey</code> (PSK)	(optional)	An additional shared key
<code>allowedIPs</code>	<code>[]</code>	Allowed IPs. When adding a new peer, the panel automatically suggests the next free address (default <code>10.0.0.2/32</code>)
<code>keepAlive</code>	(optional)	"Keep-alive" — the connection keep-alive interval
<code>comment</code>	(optional)	"Comment" — an arbitrary peer label; shown next to the "Peer N" header and inserted into the share link and the <code>remark</code> of the <code>.conf</code> file

The "Add peer" button generates a new key pair and fills in the next `allowedIPs`. Each peer can be deleted (deletion is unavailable for the single remaining one).

The peer "Comment" field helps tell devices apart: its text is shown in the form next to the "Peer N" header and is also inserted into the share link and the `remark` of the generated `.conf` file, so the device is easy to recognize in a client app. This is a panel-level field — xray-core ignores unknown peer fields.

Workers, Domain Strategy, and the Transport tab

Besides peers, a WireGuard inbound has the **Workers** field (the number of worker threads) and the **Domain Strategy** field (the domain-resolution strategy: `ForceIP`, `ForceIPv4`, `ForceIPv4v6`, `ForceIPv6`, `ForceIPv6v4`). Both fields are optional and are written to the config only when set.

A **"Transport"** tab is also available for WireGuard — but in a reduced form: it configures only `sockopt` and **Finalmask** obfuscation. The transport (`network`) dropdown is hidden, since WireGuard always listens over UDP. In noise Finalmask entries, a separate **Rand Range** field sets the byte range 0–255 (with validation), and **Salamander** is available as an obfuscation method for WireGuard and Hysteria.

When to choose WireGuard: when you need a genuine WireGuard VPN tunnel rather than a masquerading proxy.

5.10. Hysteria (v2 by default)

Purpose: a Hysteria inbound over QUIC. By default the panel works with version 2. Each client authenticates with an `auth` token instead of a UUID/password. TLS for Hysteria is always available (see the capability table in 5.2).

Fields of the `settings` block:

Field	Default value	Description
<code>version</code>	<code>2</code>	The protocol version (minimum 1; the panel defaults to 2)
<code>clients</code>	<code>[]</code>	List of clients

Each client's key field is `auth` (a token, required) plus the common fields (`email`, `limits`, `enable`, `tgId`, `subId`, `comment`, `reset`).

Additional parameters are set in `streamSettings.hysteriaSettings` :

Field	Value / options	Description
<code>version</code>	fixed at 2 (field locked)	"Version"
<code>udpIdleTimeout</code>	(integer $\geq$ 1, sec.)	"UDP idle timeout (s)" — the UDP idle timeout
<code>masquerade</code>	disabled by default	"Masquerade" — masquerading as a regular web server for "unsolicited" requests

When `masquerade` is enabled, a type (`type`) selection becomes available:

- ```` — default (a 404 page);
- `proxy` — reverse proxy (the "Upstream URL", "Rewrite Host", "Skip TLS verify" fields);
- `file` — serving a directory (the "Directory" field, for example `/var/www/html`);
- `string` — a fixed response (the "Status code", "Body", "Headers" fields).

When to choose Hysteria: when you need a QUIC transport and resilience on unstable/mobile channels; masquerading increases the stealthiness of the entry point.

5.11. MTProto (Telegram proxy)

Added in version **3.3.0**. The protocol value is `mtproto` .

MTProto is the protocol of Telegram's own proxy. In 3X-UI such an inbound is **served not by Xray, but by a separate `mtg` process** managed by the panel itself. The panel periodically reconciles the enabled MTProto inbounds with the running `mtg` processes: it brings up the missing ones, stops the extra ones, and collects traffic counters from the `mtg` metrics. Therefore, **traffic accounting** for such an inbound works the same as for regular protocols.

The official hint in the form:

"MTProto is served by a separate `mtg` process, not by Xray. Transport settings and clients do not apply here — share the link below in Telegram."

Consequences:

- The **"Transport" (Stream Settings) and "Clients" tabs do not apply to this inbound** — access is set by a single secret rather than a list of clients.
- An MTProto inbound runs **only on the main panel**; it is not deployed to child nodes (inbounds with a set `NodeID` are skipped).
- The **"Sniffing"** tab is hidden for MTProto — this protocol is served by the `mtg` process rather than Xray, so sniffing is not applicable to it.

Fields. Stored in the inbound's `settings` :

UI field	Key	Description
Remark	<code>remark</code>	The inbound label.
Listen IP	<code>listen</code>	The IP to listen on (empty = all interfaces).
Port	<code>port</code>	The proxy port.
Secret	<code>settings.secret</code>	The access secret in FakeTLS format.

UI field	Key	Description
Cover domain (FakeTLS)	<code>settings.fakeTlsDomain</code>	The domain whose HTTPS traffic the proxy masquerades as.

Secret format (FakeTLS). The panel automatically brings the secret into the correct form: the result = `ee` + 32 hex characters + the hex code of the cover domain, that is, `ee<hex32><hex(fakeTlsDomain)>`. The `ee` prefix enables FakeTLS mode, and the domain (for example, a well-known site) serves to disguise the traffic as regular HTTPS. It is enough to specify the domain – the panel will build the rest itself.

Domain fronting and extended mtg options

An MTProto inbound has additional `mtg` process parameters. The **Domain fronting IP**, **Domain fronting port**, and **Domain fronting PROXY protocol** fields specify where `mtg` sends non-Telegram traffic (for example, to a fake NGINX site): if the IP is left empty, the FakeTLS domain is used via DNS, and the default port is `443`. Also available are **Accept PROXY protocol** (for the listener), **IP preference** (`prefer-ipv6` / `prefer-ipv4` / `only-ipv6` / `only-ipv4`), and **Debug logging**. Each value is written to the `mtg-<id>.toml` file only when it is set.

Routing Telegram traffic through Xray

The **"Route through Xray"** toggle (off by default) and the optional **Outbound** field let you subordinate the Telegram egress to the Xray router. When enabled, the panel embeds a local SOCKS bridge tagged with the inbound's own tag into the Xray config, and `mtg` sends Telegram traffic through it. The traffic can then be matched by rules on the "Routing" tab or forced into a chosen outbound or balancer via the **Outbound** field (if the field is empty, the routing rules decide).

How to share it with a user. For an MTProto inbound, the panel generates an invitation link:

Example: a FakeTLS secret and a ready-made link. If the cover domain is `www.cloudflare.com`, the secret is assembled as `ee` + 32 hex characters + the hex code of the domain, for example:

```
secret = ee1a2b3c4d5e6f70819293a4b5c6d7e8f7777772e636c6f7564666c6172652e636f6d
```

The resulting invitation link (this and the QR code are sent to the Telegram user):

```
tg://proxy?
server=203.0.113.10&port=443&secret=ee1a2b3c4d5e6f70819293a4b5c6d7e8f7777772e636c6f7564666c6172652e636f6d
```

```
tg://proxy?server=<address>&port=<port>&secret=<secret>
```

(the equivalent is `https://t.me/proxy?server=...&port=...&secret=...`). This link and the QR code should be sent to the Telegram user – when opened, the proxy is immediately added to the app. The link is also served through the subscription server.

When to use it. The standard way to bypass Telegram blocks; the FakeTLS masquerading (the cover domain) makes the traffic look like a regular visit to the specified site.

5.12. Quick cheat sheet for choosing a protocol

- **VLESS** — the default choice; the best option with REALITY or TLS + XTLS-Vision, supports post-quantum authentication.
 - **Trojan** — masquerading as HTTPS with fallbacks to a web server.
 - **VMess** — compatibility with older clients.
 - **Shadowsocks** — a simple proxy without TLS; the modern choice is the `2022-blake3-*` methods.
 - **Hysteria** — QUIC, resilience on poor channels.
 - **mixed / http** — service SOCKS/HTTP proxies.
 - **WireGuard** — a full-fledged VPN tunnel.
 - **tunnel** — transparent port forwarding.
 - **MTProto** — a proxy for bypassing Telegram blocks (FakeTLS); a separate `mtg` process.
-

6. Transport (Stream Settings)

The transport (in the panel UI — the **"Transport"** field, *Transmission*) defines how Xray-core carries data inside an inbound: which network protocol is used on top of TLS/Reality and exactly how the traffic is framed. These parameters are stored in the `streamSettings` object of the Xray configuration and are set on the transport tab of the inbound editor. Encryption (TLS / Reality) is covered in a separate section — here only the network choice and its parameters are described.

6.1. Choosing the transmission network

The network is selected from the **"Transport"** drop-down (`streamSettings.network`). The default value is `tcp` (shown in the list as **RAW**). The following options are available:

Value in the list	network field	Transport
RAW	<code>tcp</code>	Plain TCP (renamed to RAW in newer Xray versions), optionally with HTTP obfuscation
mKCP	<code>kcp</code>	Reliable UDP transport mKCP
WebSocket	<code>ws</code>	WebSocket over HTTP(S)
gRPC	<code>grpc</code>	gRPC tunnel (HTTP/2)
HTTPUpgrade	<code>httpupgrade</code>	HTTP Upgrade
XHTTP	<code>xhttp</code>	XHTTP / SplitHTTP — a modern multiplexable transport

When the value is changed, the panel clears the settings block of the previous network and fills the new network's block with the default values from its schema, so every field of the sub-form always has a meaningful initial value.

Important. In this panel build the **HTTP/2 (h2) transport is absent from the list** — it was removed from the set of networks; for a bidirectional HTTP/2-like tunnel use gRPC, and for a modern HTTP-masked transport use XHTTP. The **Hysteria** transport (`hysteria`) is not selected through this list: it is rigidly tied to the Hysteria protocol and appears automatically when the inbound itself is created with the Hysteria protocol (see section 6.8).

Below, each network and each of its fields is examined separately.

6.2. RAW / TCP (`tcpSettings`)

The basic TCP transport. By default the traffic is transmitted "as is"; optionally it can be disguised as an ordinary HTTP/1.1 exchange.

Field	Default value	Description
Proxy Protocol (<code>acceptProxyProtocol</code>)	<code>false</code> (off)	Accept a PROXY protocol header from an upstream load balancer/proxy
HTTP obfuscation (<code>header.type</code>)	<code>none</code> (off)	Enables disguising the traffic as HTTP/1.1

Proxy Protocol

The **"Proxy Protocol"** toggle (`acceptProxyProtocol`). When enabled, Xray expects a PROXY protocol header on the incoming connection and extracts the client's real IP from it. Enable it only if a reverse proxy/load balancer sits in front of the panel (for example, HAProxy or nginx with `send-proxy`) that adds this header. Off by default.

HTTP obfuscation (camouflage)

The **"HTTP Obfuscation"** toggle. It controls the `header` field:

- **Off** → `header.type = "none"` (on the wire the `header` field is simply absent). Plain TCP.
- **On** → `header.type = "http"`. The traffic is framed to look like an HTTP/1.1 request and response. When enabled, the panel immediately fills the `request` and `response` sub-objects with default values.

When HTTP obfuscation is enabled, fields for configuring the simulated request and response appear.

Request header (`header.request`):

Field	Key	Default value	Description
Request version	<code>request.version</code>	<code>1.1</code>	The HTTP version in the request start-line
Request method	<code>request.method</code>	<code>GET</code>	The HTTP method of the simulated request
Request path	<code>request.path</code>	<code>/</code>	Path(s). Entered as a comma-separated list of values; on the wire this is an array of strings. If left empty, <code>/</code> is substituted
Request headers	<code>request.headers</code>	<code>{}</code> (empty)	A "Name/Value" table of HTTP headers. Stored as a map <code>name</code> → <code>[values]</code> (one name may have several values)

Response header (`header.response`):

Field	Key	Default value	Description
Response version	<code>response.version</code>	<code>1.1</code>	The HTTP version in the response start-line
Response status	<code>response.status</code>	<code>200</code>	The HTTP status code of the simulated response

Field	Key	Default value	Description
Response reason	<code>response.reason</code>	OK	Textual description of the status (reason-phrase)
Response headers	<code>response.headers</code>	<code>{}</code> (empty)	A "Name/Value" table of response headers (map <code>name</code> → <code>[values]</code>)

The header fields are edited line by line – each line specifies a header name (`Name`) and its value (`Value`). These parameters are used only to disguise the appearance of the traffic; they do not affect cryptography. The default values (`GET / HTTP/1.1` , response `200 OK`) suit most scenarios – change them only if you need to mimic a specific site/service.

Example `streamSettings` for RAW with HTTP obfuscation:

```
{
  "network": "tcp",
  "tcpSettings": {
    "acceptProxyProtocol": false,
    "header": {
      "type": "http",
      "request": {
        "version": "1.1",
        "method": "GET",
        "path": ["/"],
        "headers": {
          "Host": ["www.example.com"]
        }
      },
      "response": {
        "version": "1.1",
        "status": "200",
        "reason": "OK"
      }
    }
  }
}
```

Note: on the wire `path` is an array of strings, and each header is an array of values (`Host` → `["www.example.com"]`).

6.3. mKCP (`kcpSettings`)

mKCP is a reliable transport over UDP. It is useful on links with packet loss and high latency, but it produces increased overhead traffic. All default values match those recommended in xray-core.

Field	Key	Default	Allowed	Description
MTU	<code>mtu</code>	1350	576–1460	Maximum packet size (bytes). Reduce it when there are fragmentation problems

Field	Key	Default	Allowed	Description
TTI (ms)	<code>tti</code>	<code>20</code>	10–100	Transmission interval (ms). Lower — lower latency, but higher overhead
Uplink (MB/s)	<code>uplinkCapacity</code>	<code>5</code>	≥ 0	Estimated upload throughput (MB/s)
Downlink (MB/s)	<code>downlinkCapacity</code>	<code>20</code>	≥ 0	Estimated download throughput (MB/s)
CWND multiplier	<code>cwndMultiplier</code>	<code>1</code>	≥ 1	Congestion window multiplier
Max sending window	<code>maxSendingWindow</code>	<code>2097152</code>	≥ 0	Maximum sending window size

Notes on the fields:

- **Uplink / Downlink capacity** define how aggressively mKCP occupies the channel. They are set according to the actual channel width: overstated values lead to wasted traffic, understated ones to underutilization of the channel.
- **TTI** directly affects the "latency [?](#) overhead" trade-off: smaller values reduce latency but increase the volume of overhead packets.
- **MTU** limits the size of a single mKCP packet; lowering it helps on links where large UDP packets are cut or lost.

In this build the "seed" field (the mKCP obfuscation password) and the drop-down for the **header/obfuscation type** (`none`, `srtplib`, `utp`, `wechat-video`, `dtls`, `wireguard`) in the mKCP sub-form **are absent as separate fields** — transport-layer obfuscation has been moved into the common "FinalMask" mechanism (including the `mkcp-legacy` mode), described in the corresponding section. The "congestion" parameter as a separate checkbox is also not exposed; congestion control is set via `cwndMultiplier` and `maxSendingWindow`.

6.4. WebSocket (`wsSettings`)

WebSocket transport over HTTP(S). It passes well through CDNs and reverse proxies and is disguised as ordinary web traffic.

Field	Key	Default	Description
Proxy Protocol	<code>acceptProxyProtocol</code>	<code>false</code>	Accept a PROXY protocol header from an upstream proxy (see section 6.2)
Host	<code>host</code>	<code>""</code> (empty)	The value of the HTTP <code>Host</code> header. Set it when working through a CDN/domain fronting
Path	<code>path</code>	<code>/</code>	The path in the request line of the WebSocket handshake
Heartbeat period	<code>heartbeatPeriod</code>	<code>0</code>	Interval for sending heartbeat frames (in seconds). <code>0</code> disables heartbeat

Field	Key	Default	Description
Headers	<code>headers</code>	<code>{}</code> (empty)	Additional HTTP handshake headers. A "Name → Value" map (string values only, no arrays)

Notes:

- **Path** must match on the server (inbound) and on the client. Often the entry point is disguised behind this path on the web-server side.
- **Host** is worth setting if the inbound sits behind a CDN or domain fronting is used; otherwise it can be left empty.
- **Heartbeat period** keeps the connection "alive" when passing through proxies/CDNs that aggressively drop inactive sessions. By default (`0`) heartbeat is off.
- Unlike RAW, the WebSocket header table uses a "flat" `name → value` format (one value line per header).

Example `streamSettings` for WebSocket behind a CDN:

```
{
  "network": "ws",
  "wsSettings": {
    "acceptProxyProtocol": false,
    "host": "cdn.example.com",
    "path": "/ray",
    "heartbeatPeriod": 0,
    "headers": {
      "User-Agent": "Mozilla/5.0"
    }
  }
}
```

The `host` and `path` values must match on the client; unlike RAW, the header value here is a plain string, not an array.

6.5. gRPC (`grpcSettings`)

The "lightest" transport in terms of the number of parameters. It tunnels traffic inside gRPC calls (over HTTP/2); it is well compatible with CDNs that support gRPC. There is no header obfuscation.

Field	Key	Default	Description
Service Name (<code>Service Name</code>)	<code>serviceName</code>	<code>""</code> (empty)	The gRPC service name (effectively the tunnel "path"). Must match on the server and the client
Authority	<code>authority</code>	<code>""</code> (empty)	The value of the <code>:authority</code> pseudo-header (the HTTP/2 equivalent of <code>Host</code>). Set it when working through a CDN/domain
Multi Mode	<code>multiMode</code>	<code>false</code> (off)	Enables multiplexing several parallel gRPC streams within a single connection

Notes:

- **Service Name** is the main identifier of the gRPC channel; it must be the same on both sides. An empty value is allowed, but a non-obvious string is usually set for disguise.
- **Authority** affects which `:authority` is sent in HTTP/2 frames; it is needed primarily when proxying through a CDN.
- **Multi Mode** allows several logical streams to go through a single physical connection; enable it to improve performance when both the server and the client support it.

Example `streamSettings` for gRPC:

```
{
  "network": "grpc",
  "grpcSettings": {
    "serviceName": "GunService",
    "authority": "grpc.example.com",
    "multiMode": false
  }
}
```

The `serviceName` (here `GunService`) acts as the tunnel "path" and must match on the server and the client.

6.6. HTTPUpgrade (`httpupgradeSettings`)

A transport based on the HTTP Upgrade mechanism (like WebSocket, but without the WebSocket protocol itself). It also passes well through proxies and CDNs. The set of fields repeats WebSocket, but **without** the heartbeat period.

Field	Key	Default	Description
Proxy Protocol	<code>acceptProxyProtocol</code>	<code>false</code>	Accept a PROXY protocol header from an upstream proxy
Host	<code>host</code>	<code>""</code> (empty)	The value of the HTTP <code>Host</code> header
Path	<code>path</code>	<code>/</code>	The path of the HTTP request with the <code>Upgrade</code> header
Headers	<code>headers</code>	<code>{}</code> (empty)	Additional HTTP headers. A "flat" <code>name → value</code> map (as in WebSocket)

The purpose of the **Host**, **Path** and **Headers** fields is the same as in WebSocket (section 6.4). Heartbeat is not provided for HTTPUpgrade — that is specific to WebSocket.

6.7. XHTTP / SplitHTTP (`xhttpSettings`)

XHTTP (also known as SplitHTTP) is the modern multiplexable HTTP transport of xray-core. It splits the upstream and downstream flows into separate HTTP requests, which suits CDNs and environments with

connection-duration limits well. Not all fields are visible in the editor at once: some of them appear depending on the selected mode (`mode`) and the enabled toggles.

Basic fields (always visible)

Field	Key	Default	Description
Host	<code>host</code>	<code>""</code> (empty)	The value of the HTTP <code>Host</code> header
Path	<code>path</code>	<code>/</code>	The base path of HTTP requests
Mode (<code>Mode</code>)	<code>mode</code>	<code>auto</code>	Transmission mode (see below)
Server Max Header Bytes	<code>serverMaxHeaderBytes</code>	<code>0</code>	The limit on the request header size on the server (bytes). <code>0</code> — the xray-core default value
Padding Bytes	<code>xPaddingBytes</code>	<code>100-1000</code>	The range of random "filler" padding (in bytes, format <code>min-max</code>) to hinder size analysis
Headers	<code>headers</code>	<code>{}</code> (empty)	Additional HTTP headers. A "flat" <code>name → value</code> map
Uplink HTTP method	<code>uplinkHTTPMethod</code>	<code>""</code> (Default = <code>POST</code>)	The HTTP method of upstream requests. Options: empty (<code>POST</code> by default), <code>POST</code> , <code>PUT</code> , <code>GET</code> (the last is available only in <code>packet-up</code> mode)
Padding Obfs Mode	<code>xPaddingObfsMode</code>	<code>false</code> (off)	Enables extended padding obfuscation and opens additional fields (see below)
No SSE Header	<code>noSSEHeader</code>	<code>false</code> (off)	Do not send the <code>Content-Type: text/event-stream</code> (SSE) header. Enable it if it interferes with passage through intermediate nodes

The "Mode" field (`mode`)

A drop-down with the values:

Value	Description
<code>auto</code>	Automatic mode selection (default)
<code>packet-up</code>	The upstream flow is split into separate HTTP requests (one packet per request)
<code>stream-up</code>	The upstream flow is transmitted as a single long streaming request
<code>stream-one</code>	A single shared bidirectional streaming request

The choice of mode determines which additional fields become visible.

Fields visible only when `mode = packet-up` :

Field	Key	Default	Description
Max buffered upload	<code>scMaxBufferedPosts</code>	<code>30</code>	The maximum number of simultaneously buffered upstream POST requests

Field	Key	Default	Description
Max upload size (bytes)	scMaxEachPostBytes	1000000	The maximum size of a single upstream POST request (bytes)
Uplink Data Placement	uplinkDataPlacement	"" (Default = body)	Where to place upstream data: body , header , cookie , query
Uplink Data Key	uplinkDataKey	""	The key/header name for uplink data. Appears only if uplinkDataPlacement is set and is not equal to body

Field visible only when `mode = stream-up` :

Field	Key	Default	Description
Stream-Up Server	scStreamUpServerSecs	20-80	The range of hold time for the server streaming connection (in seconds, format min-max)

Padding obfuscation fields (visible when `xPaddingObfsMode = on`)

Field	Key	Default	Description
Padding Key	xPaddingKey	"" (placeholder x_padding)	The key name for the padding
Padding Header	xPaddingHeader	"" (placeholder X-Padding)	The name of the HTTP header in which the padding is carried
Padding Placement	xPaddingPlacement	"" (Default = queryInHeader)	Where to place the padding: queryInHeader , header , cookie , query
Padding Method	xPaddingMethod	"" (Default = repeat-x)	The padding generation method: repeat-x or tokenish

Session and sequence placement (always visible)

Field	Key	Default	Description
Session Placement	sessionPlacement	"" (Default = path)	Where to carry the session identifier: path , header , cookie , query
Session Key	sessionKey	"" (placeholder x_session)	The session key name. Appears only if sessionPlacement is set and is not equal to path
Sequence Placement	seqPlacement	"" (Default = path)	Where to carry the packet sequence number: path , header , cookie , query
Sequence Key	seqKey	"" (placeholder x_seq)	The sequence key name. Appears only if seqPlacement is set and is not equal to path

Recommendations:

- For most setups it is enough to leave **Mode = auto**, set **Path/Host** and (when working through a CDN) coordinate them with the client.
- The placement fields (**Placement / *Key*) and padding obfuscation are needed only for fine-tuning to a specific anti-DPI/CDN scenario; with empty values the xray-core default values indicated in parentheses are used.
- Parameters related to the client/outbound side (for example, retry POST intervals, chunk sizes) are not shown in the inbound form – the listener server ignores them. The XMUX multiplexer, by contrast, is available in the inbound form (see below).
- **Service defaults are no longer seeded.** The panel no longer writes the `scMaxEachPostBytes` and `scMinPostsIntervalMs` service defaults into XHTTP configs – xray-core's internal values are applied instead. This removes a stable DPI fingerprint (the literal `scMinPostsIntervalMs=30`) that traffic could previously be blocked on. For inbounds already saved, values equal to xray-core's defaults are no longer emitted into share links and the subscription (no re-save required); values you set manually are kept.

Example `streamSettings` for XHTTP (auto mode):

```
{
  "network": "xhttp",
  "xhttpSettings": {
    "host": "xhttp.example.com",
    "path": "/yourpath",
    "mode": "auto",
    "xPaddingBytes": "100-1000"
  }
}
```

For most setups these four fields are enough; the session/sequence placement and padding obfuscation fields are left empty – then the xray-core default values are used.

XMUX (connection multiplexing)

The **XMUX** toggle (`enableXmux`) enables a multiplexing layer that spreads parallel requests across a small pool of physical connections. When enabled, six configurable fields appear (stored under `xhttpSettings.xmux`):

Field	Key	Default	Description
Max Concurrency	<code>maxConcurrency</code>	16–32	Maximum concurrent requests per connection (min–max range)
Max Connections	<code>maxConnections</code>	0	Maximum physical connections (0 – unlimited)
Max Reuse Times	<code>cMaxReuseTimes</code>	"" (empty)	How many times a connection may be reused
Max Request Times	<code>hMaxRequestTimes</code>	600–900	Maximum requests per connection (range)

Field	Key	Default	Description
Max Reusable Secs	<code>hMaxReusableSecs</code>	<code>1800-3000</code>	How long a connection stays reusable (seconds, range)
Keep Alive Period	<code>hKeepAlivePeriod</code>	<code>""</code> (empty)	The keep-alive period for holding the connection open

Important. **Max Connections** and **Max Concurrency** cannot be set at the same time – xray-core will reject such a config. By default, when XMUX is enabled the panel pre-fills `Max Concurrency = 16-32`; if you set **Max Connections** (a value greater than `0`), the panel drops the default `Max Concurrency` value to avoid the conflict.

6.8. Hysteria transport (`hysteriaSettings`)

The **Hysteria** transport is not selected in the "Transport" list: it is activated automatically when an inbound is created with the Hysteria protocol, and is hidden for other protocols (when leaving the Hysteria protocol, the network is forcibly reset to `tcp`). The parameters:

Field	Key	Default	Description
Version	<code>version</code>	<code>2</code> (fixed, the field is locked)	The Hysteria version. Only Hysteria 2 is supported
UDP idle timeout (s)	<code>udpIdleTimeout</code>	<code>60</code>	The UDP session idle timeout (seconds). The accepted range is 2–600; xray-core rejects values outside this interval at startup
Masquerade	<code>masquerade</code>	off (absent)	Enables disguising the listener as an HTTP/3 server when probed

When **Masquerade** is enabled, a type selector (`type`) and fields depending on it appear:

- `""` – **default (404 page)**: a standard 404 page is served (no additional fields required).
- **proxy (reverse proxy)**: reverse proxying to an external site.
 - `url` (**Upstream URL**, placeholder `https://www.example.com`) – the target address;
 - `rewriteHost` (**Rewrite Host**, default `false`) – substitute the `Host` header;
 - `insecure` (**Skip TLS verify**, default `false`) – do not verify the upstream's TLS certificate.
- **file (serve directory)**: serving files from a directory.
 - `dir` (**Directory**, placeholder `/var/www/html`).
- **string (fixed body)**: a fixed HTTP response.
 - `statusCode` (**Status code**, default `0`, range 0–599);
 - `content` (**Body**) – the response body;
 - `headers` (**Headers**) – a `name → value` map.

Masquerade allows a Hysteria-based inbound to look like an ordinary HTTP/3 server to active probes, which increases stealth. By default masquerade is off.

Example `hysteriaSettings` with reverse proxying (`masquerade` → `proxy`):

```
{
  "version": 2,
  "udpIdleTimeout": 60,
  "masquerade": {
    "type": "proxy",
    "url": "https://www.example.com",
    "rewriteHost": true,
    "insecure": false
  }
}
```

Here, when probed, the listener serves the response from `https://www.example.com`, disguising itself as an ordinary HTTP/3 site.

6.9. Related parameters

In addition to the network choice, two common blocks that do not depend on a specific transport are available on the same tab (in detail — in the corresponding sections):

- **External Proxy** (`externalProxy`) — a list of external addresses/ports that are substituted into subscription links instead of the panel's own address.
- **Sockopt** (`sockopt`) — low-level socket options (TCP Fast Open, mark, domain strategy, transparent proxying, etc.).
- **FinalMask** (`finalmask`) — a common transport-layer obfuscation mechanism (including legacy mKCP obfuscation) that has replaced the separate "seed"/"header type" fields inside the network sub-forms.

7. Connection Security: TLS, XTLS, and REALITY

Every inbound that supports transmission over a transport stream (VMess, VLESS, Trojan, Shadowsocks, Hysteria) has a **"Security"** tab in the editor. It configures exactly how the transport channel is encrypted and disguised. Three modes are available, toggled by radio buttons:

Mode	UI label	When available
none	None	Always (except Hysteria, where TLS is mandatory)
tls	TLS	For VMess/VLESS/Trojan/Shadowsocks on <code>tcp</code> , <code>ws</code> , <code>http</code> , <code>grpc</code> , <code>httpupgrade</code> , <code>xhttp</code> networks; for Hysteria — always
reality	Reality	Only for VLESS/Trojan on <code>tcp</code> , <code>http</code> , <code>grpc</code> , <code>xhttp</code> networks

The **None** button is not shown if the protocol is Hysteria (TLS is mandatory for it). The **Reality** button appears only for a valid combination of protocol and network (see the table above).

When the mode is changed, the panel fully rebuilds the `streamSettings` block: `tlsSettings` and `realitySettings` from the previous mode are removed and the default values for the selected mode are substituted. In particular, when **Reality** is selected, the panel immediately and automatically: substitutes a random `target` + `serverNames` (SNI) pair from a built-in list of popular domains, generates random `shortIds`, and also makes a request to the server for a fresh X25519 key pair (`privateKey/publicKey`).

7.1. What's the difference: TLS vs XTLS vs REALITY

- **TLS** — classic transport encryption over the TLS 1.2/1.3 protocol. A valid certificate (your own domain + chain) must be present on the server. Traffic looks like regular HTTPS, but an active censor can recognize a distinguishable TLS handshake to your domain; in case of "SNI shooting" or the absence of a trusted certificate, the connection is blocked/returns an error.
- **XTLS (Vision)** — this is not a separate mode in the "Security" list but a *flow* mechanism on top of TLS or Reality. It is enabled on the inbound's client side via the **Flow** field = `xtls-rprx-vision` (or `xtls-rprx-vision-udp443`). Vision is available for VLESS on the `tcp` network with `security = tls` or `security = reality`, and also for VLESS over the `xhttp` transport when VLESS encryption (`vlessenc` / ML-KEM) is enabled — in that case the **Flow** field can likewise be set to `xtls-rprx-vision`, and the value is correctly carried into the `vless://` link (`flow=xtls-rprx-vision`). Vision reduces "double encryption" (TLS-in-TLS) by delivering the payload directly after the handshake, which speeds up transmission and improves disguise. Therefore, the combination of **VLESS + Reality + Flow xtls-rprx-vision** is considered the recommended modern configuration.
- **REALITY** — a disguise mechanism without its own certificate. The server "borrows" the TLS handshake of a real third-party site (`target` / `serverNames`), so to an observer the connection is indistinguishable from accessing that site, and no certificate is needed at all. Authentication is built on an X25519 key pair and a set of `shortIds`. REALITY is resistant to active probing and SNI-based blocking, since the SNI points to a real external domain. The price is stricter configuration requirements (a correct `target` with port, key synchronization with the client).

A short rule of thumb for choosing:

- you have your own domain and a valid certificate, and you need a simple HTTPS appearance → **TLS** (with Vision where possible);
- you have no domain/certificate or you need maximum stealth against DPI → **REALITY** (with Vision for VLESS/TCP).

7.2. "None" mode (`none`)

The transport is transmitted without a TLS wrapper: the `tlsSettings` and `realitySettings` blocks are excluded from `streamSettings`. The mode has no additional fields. It is suitable when:

- the inbound listens only on `127.0.0.1` and serves as a fallback target (by the panel's rule, a child inbound for a fallback must listen on `127.0.0.1` with `security=none`);
- encryption/disguise is provided by an external layer (for example, an Nginx reverse proxy in front of the panel);
- a protocol with its own encryption (Shadowsocks) is used on the internal network.

For inbounds accessible from outside, "None" mode is not recommended.

Example: a `streamSettings` block for TLS on the `tcp` network (VLESS/Trojan/VMess). This is the result after selecting **TLS** mode and filling in the SNI and certificate paths:

```
"streamSettings": {
  "network": "tcp",
  "security": "tls",
  "tlsSettings": {
    "serverName": "vpn.example.com",
    "minVersion": "1.2",
    "maxVersion": "1.3",
    "alpn": ["h2", "http/1.1"],
    "settings": { "fingerprint": "chrome" },
    "certificates": [
      {
        "certificateFile": "/root/cert/vpn.example.com.crt",
        "keyFile": "/root/cert/vpn.example.com.key",
        "ocspStapling": 3600,
        "usage": "encipherment"
      }
    ]
  }
}
```

7.3. TLS mode

Fields of the `tlsSettings` block. The default values are taken from the panel schema.

Main parameters

Field (label)	Default value	Description
SNI (<code>serverName</code>)	<code>""</code> (empty)	Server Name Indication – the domain name presented in the TLS handshake. Must match the certificate's domain. English placeholder hint: "Server Name Indication".
Cipher Suites (<code>cipherSuites</code>)	<code>""</code> → Auto	A list of allowed cipher suites. Empty by default – the choice is left to Xray/Go's discretion (the Auto option). Change it only when you need to explicitly restrict the ciphers.
Min/Max version (<code>minMaxVersion</code>)	min = <code>1.2</code> , max = <code>1.3</code>	The minimum and maximum TLS versions. Available values: <code>1.0</code> , <code>1.1</code> , <code>1.2</code> , <code>1.3</code> . It is recommended to leave <code>1.2</code> – <code>1.3</code> ; lowering the minimum to 1.0/1.1 is undesirable (outdated, insecure versions).
uTLS (<code>settings.fingerprint</code>)	<code>chrome</code> (in the form, a None option = <code>""</code> is available)	The emulated TLS fingerprint of the client hello (uTLS fingerprint), so the handshake looks like that of a popular browser. See the list below. In TLS the first item in the list is None (<code>""</code>), which disables emulation.
ALPN (<code>alpn</code>)	<code>["h2", "http/1.1"]</code>	A list of application-layer protocols negotiated in TLS (multiple selection). Permitted values: <code>h3</code> , <code>h2</code> , <code>http/1.1</code> . By default <code>h2</code> and <code>http/1.1</code> are offered.

Possible **uTLS fingerprint** values (the same for TLS and REALITY): `chrome` , `firefox` , `safari` , `ios` , `android` , `edge` , `360` , `qq` , `random` , `randomized` , `randomizednoalpn` , `unsafe` . In the TLS form an empty **None** option is additionally available (no fingerprint emulation is applied).

Available **Cipher Suites** values (TLS 1.3 and ECDHE suites): `TLS_AES_128_GCM_SHA256` , `TLS_AES_256_GCM_SHA384` , `TLS_CHACHA20_POLY1305_SHA256` , `TLS_ECDHE_ECDSA_WITH_AES_128_CBC_SHA` , `TLS_ECDHE_ECDSA_WITH_AES_256_CBC_SHA` , `TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA` , `TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA` , `TLS_ECDHE_ECDSA_WITH_AES_128_GCM_SHA256` , `TLS_ECDHE_ECDSA_WITH_AES_256_GCM_SHA384` , `TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256` , `TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384` , `TLS_ECDHE_ECDSA_WITH_CHACHA20_POLY1305_SHA256` , `TLS_ECDHE_RSA_WITH_CHACHA20_POLY1305_SHA256` .

TLS toggles

Toggle	Default	Description
Reject unknown SNI (<code>rejectUnknownSni</code>)	off (<code>false</code>)	If enabled, the server closes the connection when the SNI presented by the client does not match the name in the certificate. Increases stealth (the server does not respond to "foreign" requests), but requires an exact SNI match on the client.
Disable System Root (<code>disableSystemRoot</code>)	off (<code>false</code>)	Disables the use of the system store of trusted root certificates.
Session resumption (<code>enableSessionResumption</code>)	off (<code>false</code>)	Enables TLS session resumption (session resumption / session tickets).

Certificates

The **SSL certificate** section (the UI heading is "SSL certificate") is defined as a list: the **+** button adds a new certificate entry, the **- Delete** button removes one (the delete button is available only if there is more than one entry). By default, one empty entry is created when TLS is enabled.

For each entry there is an input-mode toggle (`useFile`):

- **Certificate path** (value `useFile = true`, default) – paths to files on the server are specified:
 - **Public key** (`certificateFile`) – path to the certificate file (`.crt` / `.pem`);
 - **Private key** (`keyFile`) – path to the private key file (`.key`).
- **Certificate content** (value `useFile = false`) – the content is pasted directly into the fields (multiline text areas):
 - **Public key** (`certificate`) – PEM content of the certificate;
 - **Private key** (`key`) – PEM content of the key.

Below the "Certificate path" mode fields there are two buttons:

- **Set panel certificate** – substitutes the paths to the panel's own SSL certificate into the fields. For an inbound on the central panel, its certificate is taken (`POST /panel/setting/all` → `webCertFile` / `webKeyFile`); for an inbound assigned to a node, the node's own certificate (`GET /panel/api/nodes/webCert/{nodeId}`), because the central panel's paths do not exist on the node. If the certificate is not configured, a warning is shown: *"No certificate is configured for the panel. Set it up in Settings first."* (the panel certificate itself is set in the "Settings → Security" section).
- **Clear** – erases both paths.

Additional fields of each certificate entry:

Field	Default	Description
OCSP Stapling (<code>ocspStapling</code>)	<code>3600</code> (sec.)	The OCSP stapling refresh interval in seconds (minimum <code>0</code>).
One-time loading (<code>oneTimeLoading</code>)	<code>off</code> (<code>false</code>)	If enabled, the certificate is read from disk once at startup and is not re-read when the file changes.
Usage option (<code>usage</code>)	<code>encipherment</code>	The certificate's purpose. Permitted: <code>encipherment</code> (encryption – a regular server certificate), <code>verify</code> (verification), <code>issue</code> (issuance – the server signs/issues certificates itself).
Build Chain (<code>buildChain</code>)	<code>off</code> (<code>false</code>)	Shown only when <code>usage = issue</code> . Build up the certificate chain.

There is no separate self-signed certificate button in the inbound editor: the panel does not generate a self-signed certificate on the fly for an inbound. The certificate is either specified by path/content or pulled from the panel settings via the "Set panel certificate" button. Issuing/obtaining the panel's own SSL certificate (including uploading files and binding to a domain) is performed in the **Settings** → **Security** section; there are no ACME/Let's Encrypt endpoints for inbounds here.

ECH and certificate pinning (advanced TLS fields)

Field	Default	Description
ECH key (<code>echServerKeys</code>)	<code>""</code>	Encrypted Client Hello server keys.
ECH config (<code>settings.echConfigList</code>)	<code>""</code>	ECH config list (the client part, included in the link).
Peer certificate SHA-256 (<code>settings.pinnedPeerCertSha256</code>)	<code>[]</code>	SHA-256 hashes of the peer certificate (hex strings, comma-separated). Verbatim hint: <i>"SHA-256 hashes of the peer certificate as a hexadecimal string (e.g. e8e2d3...), comma-separated. Panel-only – not written to the xray server config, but included in invite links so that clients can pin the certificate."</i>

Buttons:

- **Generate random hash** (the refresh icon next to the hash field) – adds a random 32-byte hex hash to the list (generated in the browser).
- **Get a new ECH certificate** – requests a new ECH pair from the server for the current SNI (`POST /panel/api/server/getNewEchCert` , on the server `xray tls ech --serverName <SNI>` is executed); fills the **ECH key** and **ECH config** fields.
- **Clear** – clears both ECH fields.

7.4. REALITY mode

Fields of the `realitySettings` block. REALITY does not use an SSL certificate: instead, it relies on the borrowed TLS handshake of an external domain and an X25519 key pair.

Disguise parameters

Field (label)	Default value	Description
Show (<code>show</code>)	<code>off (false)</code>	REALITY debug output to the Xray logs. Usually left off.
Xver (<code>xver</code>)	<code>0</code>	The version of the PROXY protocol sent to the backend (<code>0</code> – disabled). Minimum <code>0</code> .
uTLS (<code>settings.fingerprint</code>)	<code>chrome</code>	The emulated TLS fingerprint (the same list as in TLS, but without the empty None option).
Target (<code>target</code>)	<code>""</code> (the panel substitutes a random one when enabled)	Required field. The real domain whose TLS handshake REALITY borrows. Verbatim hint: <i>"Required. Must include a port (for example, example.com:443). Without a port, Xray-core does not start."</i> Validation in the panel checks for the presence and correctness of the port; otherwise the errors "REALITY target is required" / "REALITY target must include a port..." / "REALITY target has an invalid port" are shown. The refresh button next to it substitutes a random pair from the built-in list.
SNI (<code>serverNames</code>)	<code>[]</code> (substituted together with the target)	A list of allowed SNIs (multiple input as tags). Must correspond to the domain from Target . The refresh button substitutes the SNI together with a random target.

Field (label)	Default value	Description
Max time difference (ms) (<code>maxTimediff</code>)	<code>0</code>	The maximum allowed clock discrepancy between client and server in milliseconds (<code>0</code> – no limit). Minimum <code>0</code> .
Min client version (<code>minClientVer</code>)	<code>""</code>	The minimum Xray client version (placeholder <code>25.9.11</code>). Empty – no limit.
Max client version (<code>maxClientVer</code>)	<code>""</code>	The maximum Xray client version. Empty – no limit.
Short IDs (<code>shortIds</code>)	<code>[]</code> (generated when enabled)	A list of short identifiers (hex) distinguishing clients. Multiple input as tags; the refresh button generates a random set.
SpiderX (<code>settings.spiderX</code>)	<code>/</code>	The "spider" path (the client part of REALITY), used when emulating access to the external site. Included in the invite link.

Target (`target`) and **SNI** (`serverNames`), when REALITY is enabled and via the refresh button, are filled with a random pair from the panel's built-in list: `www.amazon.com`, `aws.amazon.com`, `www.oracle.com`, `www.nvidia.com`, `www.amd.com`, `www.intel.com`, `www.sony.com` (each with port `:443`). Choose a "heavy", stable third-party HTTPS site that is not located behind your own server.

Example: a `streamSettings` block for REALITY on the `tcp` network (VLESS). No certificate is needed – a borrowed domain and an X25519 key pair are used instead:

```
"streamSettings": {
  "network": "tcp",
  "security": "reality",
  "realitySettings": {
    "show": false,
    "xver": 0,
    "dest": "www.nvidia.com:443",
    "serverNames": ["www.nvidia.com"],
    "privateKey": "YOUR_X25519_PRIVATE_KEY",
    "shortIds": [ "", "6ba85179e30d4fc2" ],
    "settings": {
      "publicKey": "YOUR_X25519_PUBLIC_KEY",
      "fingerprint": "chrome",
      "spiderX": "/"
    }
  }
}
```

Here the panel's **Target** (`target`) field corresponds to `dest` in the final Xray config.

REALITY keys (X25519)

Field	Default	Description
Public key (<code>settings.publicKey</code>)	<code>""</code>	The X25519 public key (the client puts it in its configuration/link).
Private key (<code>privateKey</code>)	<code>""</code>	The X25519 private key (stored only on the server).

Buttons below the keys:

- **Get a new certificate** — requests a new key pair from the server (GET /panel/api/server/getNewX25519Cert ; on the server xray x25519 is executed), fills the **Private key** and **Public key**. The same pair is automatically generated when REALITY mode is first enabled.

Example: obtain an X25519 key pair via the API (outside the form, e.g. for a script). The request returns the private and public keys:

```
curl -s -b cookie.txt https://your-panel:2053/panel/api/server/getNewX25519Cert
# Response:
# {"success":true,"obj":{"privateKey":"...", "publicKey":"..."}}
```

cookie.txt is the session cookie file obtained after logging in via POST /login.

- **Clear** — clears both keys.

Post-quantum ML-DSA-65 signature (mldsa65)

An additional (optional) layer of REALITY post-quantum authentication:

Field	Default	Description
mldsa65 Seed (mldsa65Seed)	""	The server seed of the ML-DSA-65 key.
mldsa65 Verify (settings.mldsa65Verify)	""	The verification value (the client part, included in the link).

Buttons:

- **Get a new Seed** — requests a new pair (GET /panel/api/server/getNewmldsa65 ; on the server xray mldsa65 is executed), fills the **mldsa65 Seed** and **mldsa65 Verify**.
- **Clear** — clears both fields.

7.5. Practical configuration recommendations

1. **VLESS + Reality (recommended)**: create a VLESS inbound on the tcp network, on the "Security" tab select **Reality** — the panel will substitute random target /SNI, shortIds and generate X25519 keys itself. If necessary, click "Get a new certificate" for your own key pair. For VLESS clients, enable **Flow** = xtls-rprx-vision (XTLS Vision) — this gives maximum performance and stealth.

Example: the final VLESS + Reality + Vision client link. This is the invite link the panel issues for such an inbound (key/ID values are illustrative):

```
vless://client-uuid@1.2.3.4:443?
type=tcp&security=reality&pbk=PUBLIC_KEY&fp=chrome&sni=www.nvidia.com&sid=6ba85179e30d4f
c2&spx=%2F&flow=xtls-rprx-vision#my-reality
```

Here pbk is the X25519 public key, sni is the borrowed domain from **Target**, sid is one of the **Short IDs**, and flow=xtls-rprx-vision is the enabled XTLS Vision. 2. **TLS with your own domain**: select **TLS**, fill in **SNI** with the domain name, add a certificate (by file path or content), or click "Set panel certificate" if the domain

and certificate are already configured in "Settings → Security". Leave **Min/Max version** = `1.2 - 1.3` and **uTLS** = `chrome` to disguise as a regular browser. 3. Do not leave **None** mode for inbounds open to the outside – use it only for local fallback targets (`127.0.0.1`) or when TLS is provided by an external proxy. 4. A tip from the interface: for most advanced fields the hint "*It is recommended to leave the default settings*" applies – change them only when you understand the consequences.

8. Clients

A client is a VPN user account: a set of credentials (UUID or password) bound to one or more inbounds, with its own traffic quota, expiry, and simultaneous-connection limit. In this fork the client is a standalone entity (the `clients` table): the same client can be bound to several inbounds at once, sharing a common UUID/password and a common traffic counter. The **Clients** section shows all panel accounts regardless of inbound, with search, filters, sorting, and bulk operations.

8.1. Client fields

Below is a breakdown of every field in the **Add client** / **Edit client** editor.

The client form is split into two tabs: **Basic** (email, inbound binding, limits, expiry, group, comment, reverse tag) and **Config** (UUID/password/auth, Flow, VMess Security). In the field labels the quota is shown as **Traffic limit (GB)**, and the expiry settings as **Duration (days)** and **Auto-renew (days)**; the **Traffic limit (GB)** and **IP limit** fields carry tooltips explaining that `0` means "no limit". When editing an existing client the random-email generator button is hidden, and the IP-log button is placed right next to the **IP limit** field and shows the number of recorded addresses.

Field	JSON key	Default	Description
Email	<code>email</code>	— (required)	Unique client identifier
UUID	<code>id</code>	generated	Identifier for VMess/VLESS
Password	<code>password</code>	generated	Password for Trojan/Shadowsocks
Auth	<code>auth</code>	generated	Password for Hysteria
Flow	<code>flow</code>	empty	Flow control (XTLS), VLESS only
VMess Security	<code>security</code>	<code>auto</code>	VMess encryption method
IP limit	<code>limitIp</code>	<code>0</code> (no limit)	Maximum simultaneous IPs
Total up/down (GB)	<code>totalGB</code>	<code>0</code> (no limit)	Traffic quota
Expiry	<code>expiryTime</code>	<code>0</code> (never)	Expiration date
Auto-renew	<code>reset</code>	<code>0</code> (off)	Traffic reset period, days
Telegram user ID	<code>tgId</code>	<code>0</code> (none)	Numeric Telegram ID
Subscription ID	<code>subId</code>	generated	Subscription identifier
Group	<code>group</code>	empty	Logical grouping label
Comment	<code>comment</code>	empty	Arbitrary note
Enabled	<code>enable</code>	<code>true</code>	Whether the account is active

Email (identifier)

The **Email** field is the primary, required client identifier. Despite the name, it does not have to be a mail address: any text label will do (a username, a number). The value must be **unique** within the panel; an attempt to create a second client with an email already in use is rejected (`email already in use`), except when the `subId` also matches (this is treated as binding the same client).

Email **cannot be left empty** (`client email is required`) and **cannot contain spaces, the characters `/`, `\`, or control characters** ("Email cannot contain spaces, '/', ' ' or control characters"). Email is involved in traffic accounting, the IP log, the online list, and operation names – changing it after the fact is not recommended.

UUID / Password / Auth (credentials)

The specific credential field depends on the protocol of the inbound the client is bound to. Values are filled in automatically if the field is left empty:

- **UUID** (field `id`) – for the **VMess** and **VLESS** protocols. If not set, a random UUID v4 is generated.
- **Password** (field `password`) – for **Trojan** and **Shadowsocks**. For Trojan, a UUID without dashes is generated by default. For Shadowsocks, a Base64 key of the required length is generated depending on the inbound's encryption method: 16 bytes for `2022-blake3-aes-128-gcm`, 32 bytes for `2022-blake3-aes-256-gcm` and `2022-blake3-chacha20-poly1305`; for other methods – a UUID without dashes. If a manually entered key does not match the 2022-blake3 method, it is replaced with a generated one.
- **Auth** (field `auth`) – the password for **Hysteria**. By default a UUID without dashes.

Since a single client can be bound to inbounds of different protocols, a client record may simultaneously contain a UUID, a password, and an auth – each protocol uses its own field.

Example: how a client's credentials look in the settings of different inbounds. The same client is identified by `id` in a VLESS inbound, by `password` in Trojan, and by `password` (a Base64 key) in Shadowsocks:

```
// fragment of settings.clients for a VLESS inbound
{ "id": "b831381d-6324-4d53-ad4f-8cda48b30811", "email": "user-a", "flow": "xtls-rprx-vision" }

// the same client in a Trojan inbound
{ "password": "b831381d63244d53ad4f8cda48b30811", "email": "user-a" }

// the same client in a Shadowsocks inbound (method 2022-blake3-aes-256-gcm)
{ "password": "GPy0aA3f7C00az53eaQ8eqMfRDjmBl0h7v1u3+Z+pHk=", "email": "user-a" }
```

Flow

Flow (field `flow`) is XTLS flow control. It applies **only to VLESS** and only when the inbound is configured for XTLS Vision: VLESS over the **TCP** transport with security **tls** or **reality**. The allowed value is `xtls-rprx-vision` (as well as the historical `xtls-rprx-vision-udp443`); an empty value means no flow.

If the inbound does not support XTLS flow, the configured flow is **silently cleared** when the client is saved: for the same client bound to several inbounds, flow is applied only where it is permitted. Change it only if you are deliberately using VLESS-Vision.

VMess Security

VMess Security (field `security`) is the payload encryption method for VMess. The default value is `auto` (Xray picks the cipher itself). The allowed values are the standard VMess ones: `auto`, `aes-128-gcm`, `chacha20-poly1305`, `none`, `zero`. For other protocols the field is unused.

IP limit (simultaneous connections)

IP limit (field `limitIp`) is the maximum number of **distinct IP addresses** from which the client may be connected simultaneously. The default value is `0`, which means **no limit**. With a positive value, the panel tracks the client's active IPs and, when the limit is exceeded, disables the account via a background job. (Starting with **3.3.1**, IP counting goes through the Xray core's online-stats API and **does not require** the access log; on older core versions the panel falls back to reading the access log, which must then be enabled.) Use it to prevent sharing a single subscription across many devices: for example, `2` allows two devices.

Example values. `limitIp: 0` — no limit; `limitIp: 1` — strictly one device at a time; `limitIp: 3` — up to three distinct IPs. On the fourth active IP the background job disables the client (`enable = false`) until you run **Reset IP limit**.

Related operations: **IP log** shows the list of recorded client IPs (with timestamps, if available); **Reset IP limit** clears the accumulated IP log so the client can connect again without waiting for the records to expire naturally.

Total up/down (GB) — traffic quota

Total up/down (GB) (field `totalGB`) is the combined traffic quota (upload + download). The default value, `0`, means **unlimited**. Once the quota is reached (`up + down >= total`), the client is considered **depleted** and is disabled. In the UI it is usually entered in gigabytes; in the database it is stored in bytes.

In the client list, the **Traffic** column shows a color-coded usage bar: the consumed traffic, the limit label (or the ∞ sign when unlimited), and a hover popover with an upload/download/remaining breakdown. The same compact indicator appears in the client cards in the mobile layout.

Expiry

Expiry (field `expiryTime`) sets the moment the account expires. The field has three modes:

- **Never** — `0`. The client never expires by time.
- **Specific date** — a positive Unix timestamp (in milliseconds). When it is reached (`expiryTime <= now`), the client is considered expired and is disabled. In the UI it is usually set by picking a date or by a duration in days (**Duration**, unit — **Days**).
- **Start after first use** — a **negative** value encoding a duration. As long as the client has not transferred a single byte, the expiry stays negative ("delayed start"). On the very first traffic-accounting tick, the panel converts it to an absolute date: `now + |duration|`. This makes it possible to sell, for example, "30 days from the first connection" without knowing in advance when the client will activate. The conversion is performed once per email, so all bound inbounds get the same expiry.

Example of encoding the expiry. Fixed date March 1, 2026, 00:00 UTC → `expiryTime: 1772323200000` (a positive timestamp in milliseconds). "30 days from first connection" → `expiryTime: -2592000000` (a negative value, $30 \times 24 \times 60 \times 60 \times 1000$); on the first byte of traffic the panel replaces it with `now + 2592000000`. Never → `expiryTime: 0`.

Auto-renew (client traffic reset period)

The **Auto-renew** field (field `reset`) is the automatic renewal/reset period in days. Tooltip: "Automatic renewal after expiry. (0 = disabled) (unit: day)".

- `0` — auto-renew is **disabled** (the default value). On expiry the client simply becomes depleted.
- `> 0` — when the expiry is reached, a background job **resets the traffic counters to zero** (`up = down = 0`), **shifts the expiry forward** by `reset` days (over several periods if needed, until the new expiry is in the future), and **re-enables** the client if necessary. This implements a periodic subscription (for example, monthly). Auto-renew **does not apply to inbounds on nodes** (`node_id IS NOT NULL`).

An important consequence: clients with `reset > 0` are **excluded** from the notion of "depleted" in bulk-delete operations — their traffic/expiry are expected to be zeroed by auto-renew rather than making the account a deletion candidate.

Telegram user ID

Telegram user ID (field `tgId`) is the numeric Telegram identifier of the user, for binding to the panel's built-in Telegram bot (notifications, self-service statistics viewing). Tooltip: "Numeric Telegram user ID (0 = none)". The default value `0` means no binding. This field can be used for filtering (**Has / None**).

Subscription ID (subId)

Subscription ID (field `subId`) is the identifier under which the client is included in a **subscription**. All clients with the same `subId` are served via a single subscription link. If the field is left empty at creation, the panel **automatically generates a random** `subId` (a UUID). The value must be **unique** among clients with a different email (`subId already in use`) and is subject to the same character restrictions as email ("Subscription ID cannot contain spaces, '/', or control characters").

Without a `subId`, the subscription link is unavailable for the client ("This client has no subId, the share link is unavailable.").

Group

Group (field `group`) is a logical label for grouping related clients. Tooltip: "A logical label for grouping related clients (e.g., team, customer, region). Filterable from the toolbar.", placeholder — "e.g., customer-a". The field is optional (empty by default). You can filter the list and run bulk operations by group; to clear a client's label, use the **Ungroup** action.

Comment

Comment (field `comment`) is an arbitrary text note for the administrator (empty by default). The content is included in search and is filterable (**Has** / **None** comment).

Enabled

Enabled (field `enable`) is the account activity flag. By default it is **enabled** (`true`); at creation, even if the flag is not provided, the panel forces it to `true`. A disabled client (`enable = false`) cannot connect and is counted as **deactive** in the summary. The panel disables clients on its own when they have used up their quota, expired, or exceeded the IP limit.

Read-only fields

The client card also shows service fields: **Created** (`created_at`) and **Updated** (`updated_at`) — the timestamps of creation and last change, filled in automatically and not editable. The **Reverse tag** field (`reverse`) is an optional Reverse tag for a simple VLESS reverse proxy ("Optional Reverse tag").

8.2. Binding to inbounds

Every client must be bound to at least one inbound — at least one is required at creation (`at least one inbound is required`). In the editor this is the **Attached inbounds** field with the hint **Select one or more inbounds**.

- **Attach** — add the client to the selected inbounds (same UUID/password and shared traffic). Existing bindings are preserved.
- **Detach** — remove the client from the selected inbounds. The client record itself is preserved (use **Delete** for full removal). Pairs where the client was not bound are silently skipped.

When saving a client bound to several inbounds, fields incompatible with a specific protocol/transport (for example, Flow outside VLESS-Vision) are automatically coerced to permitted values for each inbound.

Above the inbound selection list (in the client form, in bulk client creation, and in the bulk attach/detach dialogs) there are **Select all** and **Clear** buttons. In these lists each inbound is labelled by its remark, if one is set, otherwise by the inbound's tag.

8.3. Client operations

For an individual client (via the **Client info** card or the **Actions** context menu) the following are available:

Viewing info, QR code, and link

- **Client info** — a card with all fields, used/remaining traffic (**Remaining**), expiry, and attached inbounds.

Requesting a client via the API (`GET /panel/api/clients/get/:email`) additionally returns `usedTraffic` next to the `client` and `inboundIds` fields – the traffic actually consumed (upload + download, including node data), which makes it easier to compare consumption against the `totalGB` quota.

- **QR code** and **Link** – the client's configuration link for importing into a client app. It is built from all attached inbounds with a supported protocol (`GET /links/:email`). If there are no suitable links: "No share links – first attach the client to an inbound with a supported protocol".
- **Subscription link** – the subscription URL by `subId` (`GET /subLinks/:subId`). Available only if the client has a `subId` and the subscription service is enabled in **Panel settings** → **Subscription** (otherwise "The subscription service is disabled."). Additionally, a **JSON subscription URL** is provided.

Reset traffic

Reset traffic (`POST /resetTraffic/:email`) zeroes the upload/download counters (`up` , `down`) of a specific client. The quota (`totalGB`) and the expiry are **not affected** – only the used volume is reset. Toast: "Traffic reset". If the client is not bound to any inbound: "First attach this client to an inbound".

The **Reset traffic** button is also available from the client edit form – in the bottom bar, next to **Cancel / Save** (a confirmation is requested before the reset). If the client was disabled because its traffic was depleted, the reset (both single and bulk) automatically **re-enables** it (`enable = true`) and immediately pushes this change to the nodes – there is no longer any need to re-enable the client manually on the master and on each node.

Reset IP limit

Clears the client's accumulated IP log (`POST /clearIps/:email`) to lift a temporary block caused by exceeding the simultaneous-connection limit. Toast: "Log has been cleared".

Delete

Delete (`POST /del/:email`) is the full removal of a client. Confirmation dialog: title "Delete client {email}?", text "The client will be removed from all attached inbounds, and its traffic record will be destroyed. This action cannot be undone.". Deletion removes the client from **all** inbounds and destroys its traffic record. Toast: "Client deleted".

8.4. Bulk operations

In the client list you can select several records (**Select all**, **Clear all**); the counter is "{count} selected". The following are available for the selected clients:

- **Delete ({count})** (`POST /bulkDel`) – bulk deletion. Confirmation: "Delete {count} clients?", "Each selected client is removed from all attached inbounds, and its traffic record is destroyed. This action cannot be undone.". Toast: "Clients deleted: {count}", or, on partial failure, "Deleted: {ok}, failed: {failed}".
- **Edit ({count}) / Adjust** (`POST /bulkAdjust`) – bulk change of expiry and/or quota. Dialog "Edit {count} clients" with the hint "Positive values add, negative values subtract. Clients with unlimited expiry or traffic are skipped for the corresponding field.". Fields: **Add days** and **Add traffic (GB)**. Logic:
 - **Expiry**: clients with unlimited expiry (`expiryTime == 0`) are skipped ("unlimited expiry"); for clients with a date the expiry is shifted by the given number of days; for clients in "after first use" mode

(negative expiry) the waiting duration is adjusted. A reduction that goes beyond the remainder is skipped ("reduction exceeds remaining time/delay window").

- **Traffic:** clients with unlimited traffic (`totalGB == 0`) are skipped ("unlimited traffic"); otherwise the quota is changed by the given volume, not going below zero.
- If neither days nor traffic is specified: "Specify days or traffic before applying.". Toast: "Adjusted: {count}" / "Adjusted: {ok}, skipped: {skipped}".

Example: extend the selected clients by 30 days and add 50 GB. In the **Edit** dialog set **Add days** = `30` , **Add traffic (GB)** = `50` . Conversely, to subtract a week and trim the quota by 10 GB, enter negative values: **Add days** = `-7` , **Add traffic (GB)** = `-10` (clients with unlimited expiry or unlimited traffic are skipped for the corresponding field).

- **Attach ({count}) / Detach ({count})** (`POST /bulkAttach / bulkDetach`) — bulk attach/detach of the selected clients to/from the selected inbounds. Targets are multi-user inbounds only. Detach result: "Detached {detached}, skipped {skipped}."
- **Sub links ({count})** — a summary table of the subscription and JSON-subscription URLs of the selected clients, with a **Copy all** button. If none have a subId: "None of the selected clients have a subscription ID."
- **Add to group** and **Ungroup** — assign and remove a group label.

Resetting traffic and deleting by status

- **Reset all clients' traffic** (`POST /resetAllTraffics`) — zeroes the `up / down` counters of **all** panel clients. Confirmation: "Reset all clients' traffic?" and "The upload/download counters of all clients are reset to zero. Quotas and expiry are not affected. This action cannot be undone.". Toast: "All clients' traffic reset".
- **Delete depleted** (`POST /delDepleted`) — deletes all clients whose **quota is depleted** (`total > 0` and `up + down >= total`) **or whose expiry has passed** (`expiry_time > 0` and `expiry_time <= now`), provided `reset = 0` (clients with auto-renew are left untouched). Confirmation: "Delete depleted clients?", "All clients whose traffic quota is depleted or whose expiry has passed are deleted. This action cannot be undone.". Toast: "Depleted clients deleted: {count}".

8.5. Search, filters, and sorting

Above the list there is a search box ("Search email, comment, sub ID, UUID, password, auth...") — it searches by email, comment, subId, UUID, password, and auth. Results counter: "Showing {shown} of {total}".

The **Client filter** panel lets you select by status (category), protocol, attached inbound, expiry range, used-traffic range, presence of auto-renew (**Has/None**), presence of a Telegram ID and a comment, as well as by group. On panels that have nodes, a **Nodes** multi-select appears: you can narrow the list to clients of the selected nodes; a separate **Local panel** entry selects clients of inbounds not attached to any node (the filter is shown only when nodes exist). Sorting: **Oldest/Newest first**, **Recently updated**, **Recently online**, **Email A→Z / Z→A**, **Most traffic**, **Highest remaining**, **Expiring soonest**.

8.6. Badges and statuses

Status priority: depleted/expired → deactive → expiring soon → active.

- **Online / Offline** — a client with an active connection (present in the current online list) and **enabled**. The online list is updated by separate requests (`/onlines` , `/onlinesByGuid`).

- **Depleted** – the quota is used up (`up + down >= totalGB`) **or** the expiry has passed (`expiryTime <= now`). Such a client is automatically disabled and falls under the **Delete depleted** action.
 - **Expiring / running out** (expiring) – an enabled client with less than a threshold interval left until expiry **or** less than a threshold volume left until quota depletion (the thresholds are set in the panel settings). It does not count if the client is already depleted/disabled.
 - **Deactive** – a client with `enable = false` (disabled manually or by a background job).
 - **Active** – enabled, not depleted, not expired, and still far from the thresholds.
-

9. Client groups

This is a feature of this fork of 3X-UI. The original 3x-ui project (MHSanaei) has no concept of a "client group" — here a separate groups table, a **Groups** page in the panel menu, and the corresponding API methods have been added. If you migrate the configuration to the original 3x-ui, the group label simply will not be taken into account anywhere.

9.1. What a client group is and why you need it

A **group** is a named logical label that can be attached to one or more clients. It does not create a new way of connecting and is neither an inbound nor a node — it is purely an organizational tag that makes it convenient to filter clients and process them in bulk.

The key idea of the client model in this fork: **a client is a top-level entity identified by email** (the `email` field in the `clients` table has a unique index). The same client (one email with the same credentials) can simultaneously be listed in several inbounds and even on several nodes, including with different protocols. The group label is stored **once per client**, so it automatically propagates to all of that client's inbound bindings at once.

The group label is a logical grouping label:

Layer	Where it is stored	Field
Client record (DB)	<code>clients</code> table	<code>group_name</code> (default empty string <code>''</code>)
Groups directory (DB)	<code>client_groups</code> table	<code>name</code> (unique index, non-empty)
Inbound settings (Xray)	JSON <code>settings.clients[].group</code>	duplicated into each client object of each inbound the client belongs to

Why this is useful in practice:

- **One client across several inbounds/nodes.** If a client is "sold" as access to several inbounds at once (for example, different protocols or different nodes), the group lets you manage it as a single whole: reset traffic, delete, or rename the label — with a single operation across all of its inbounds.
- **Bulk operations and filtering.** On the **Clients** page the list can be filtered by group; on the **Groups** page bulk actions over all members of a group are available.
- **Organizing a large fleet of clients.** Labels such as `vip`, `trial`, `team-A` help sort thousands of clients into logical buckets without proliferating separate inbounds.

9.2. How a group relates to clients, inbounds, nodes, and protocols

This is the most important subsection to understand, because synchronizing the label is non-trivial.

A group describes a client, not an inbound. The label lives in the client record (`clients.group_name`). When a client is attached to several inbounds, on any change of group the panel goes through **all** inbounds the client belongs to and sets/clears the `group` field inside their Xray settings (`settings.clients[]`). Technically this

is done as follows: by the client's email, all inbounds the client belongs to are found, then in the JSON settings of each such inbound the client object with that email is edited. Therefore:

- The group **does not depend on the protocol**. One email can be a VLESS client in one inbound and a Hysteria client in another — its group label is still the same and applies to both (the credentials for each protocol are their own and are stored separately).
- The group **spans nodes**. Inbounds belonging to nodes differ from the inbounds of the main panel by the `nodeId` field (for the main panel's inbounds it is `null / 0`). The group label propagates to client objects in inbounds regardless of whether it is a main or a node inbound — as long as a client with that email is present there.

The group label is resistant to synchronization from nodes and to settings rebuilds. This behavior is intentional:

- When a node sends a traffic snapshot, its data does **not** overwrite the local `group_name` and `comment` of the client in the panel DB. The group and comment are considered "panel-local" fields — the node does not manage them.
- When inbound settings are rebuilt, an empty `group` value in the incoming data does **not** reset the already-stored label. The group is managed specifically through the **Groups** page (and not through editing an inbound's Xray settings), so an "empty group" during an ordinary settings rebuild is interpreted as "do not touch" rather than "clear".

The practical takeaway: the only operations that **intentionally clear** the label are deleting a group and explicitly removing a client from a group (see below). An ordinary inbound edit or a background sync with a node will not lose the group.

9.3. The groups directory and "empty" groups

The list of groups on the page is built by merging two sources:

1. **Derived groups** — all non-empty `group_name` values actually occurring among clients, with a count of clients.
2. **Stored groups** — records from the `client_groups` table.

This union has an important effect: a group can exist **without a single client**. Such a group is created by the explicit "Add Group" button (a record in `client_groups`) and is shown in the list with a count of `0`. These records are what count as **empty groups**. The list is always sorted by name case-insensitively.

Summary counters on the page:

Field	What it shows
Total groups	The total number of groups (stored and derived together).
Clients with a group	How many clients have a non-empty group label.
Empty groups	How many groups exist without clients (count of <code>0</code>).
Clients in group	The number of clients in a specific group (a table column).

9.4. Group fields and columns

A group record in the `client_groups` table contains:

Field	Type	Default	Description
<code>Id</code>	int	autoincrement	Primary key of the group record.
<code>Name</code>	string	— (required)	The group name. Unique index, cannot be empty. In the UI it is the Name column.
<code>CreatedAt</code>	int64 (ms)	creation time	The moment the group record was created.
<code>UpdatedAt</code>	int64 (ms)	modification time	The moment of the last modification.

The table on the page displays at least the **Name** and **Clients in group** columns, as well as action buttons (see below).

9.5. Creating a group

The **Add Group** button.

Steps:

1. Click **Add Group**.
2. Enter the group name.
3. Confirm.

Backend behavior (`POST /panel/api/clients/groups/create` , body `{"name": "..."}:`

- The name is trimmed of leading and trailing whitespace. An empty name is rejected with the error "group name is required".
- If a group with that name already exists — the error "group already exists".
- On success a record is created in `client_groups` (initially without clients — this is an empty group).

Success message: **"Group "{name}" created."**

Example: create an empty group via the API. Prepare a set of labels in advance, before populating them with clients:

```
curl -s 'https://panel.example.com:2053/panel/api/clients/groups/create' \
-H 'Content-Type: application/json' \
-b cookie.txt \
-d '{"name": "vip"}'
```

Response on success:

```
{ "success": true, "msg": "Group \"vip\" created.", "obj": null }
```

Calling it again with the same name returns `"success": false` and the message `group already exists`.

Creating an empty group in advance is convenient when you want to prepare a set of labels and then populate them with clients via "Add clients..."

9.6. Renaming a group

The **Rename** button, the dialog title is **"Rename {name}"**.

Behavior (`POST /panel/api/clients/groups/rename` , body `{"oldName": "...", "newName": "..."}:`

- Both names are trimmed of whitespace. An empty old name gives the error "old group name is required", an empty new name "new group name is required".
- If the new name matches the old one — nothing is done (`0` clients affected).
- Otherwise the rename is performed atomically:
 - the record in `client_groups` is renamed;
 - for all clients with `group_name = oldName` the field is updated to `newName` ;
 - in **all inbounds** the affected clients belong to (including node inbounds), the `group` value in the Xray settings is changed from the old one to the new one.
- After the rename the panel marks Xray as requiring a restart and sends a notification about the client change.

Messages:

- Success: **"Group renamed for {count} client(s)."**
- Name conflict in the UI: **"A group named "{name}" already exists."**

9.7. Adding clients to a group

The **Add clients...** button, title — **"Add clients to group "{name}"**".

The verbatim hint in the dialog:

"Select clients to add to this group. Existing inbound bindings are kept; only the group label changes. Clients already in this group are not shown."

If there is no one to add, **"No other clients to add."** is shown.

Behavior (`POST /panel/api/clients/groups/bulkAdd` , body `{"emails": [...], "group": "..."}:`

- The group name is required (otherwise the error "group name is required"); an empty list of emails does nothing.
- If such a group does not yet exist either in `client_groups` or among the derived ones — it will be created automatically.
- For the selected emails the clients get `group_name = group` ; the **bindings of clients to inbounds are not changed** — only the label is affected. Then the `group` field is set in all inbounds of these clients.

- The number of affected client records is returned; Xray is marked for restart.

Success message: **"Added {count} client(s) to {name}."**

Example: label several clients with a group in a single request. Clients are specified by email, and inbound bindings are not changed:

```
curl -s 'https://panel.example.com:2053/panel/api/clients/groups/bulkAdd' \
-H 'Content-Type: application/json' \
-b cookie.txt \
-d '{"emails": ["alice@example.com", "bob@example.com"], "group": "vip"}'
```

If the `vip` group does not exist yet, it is created automatically. After the request these clients get `group_name` = `"vip"` in their record, and in the Xray settings of each of their inbounds the client object gains a `"group": "vip"` field:

```
{ "id": "6f1b...", "email": "alice@example.com", "group": "vip", "enable": true }
```

9.8. Removing clients from a group (without deleting the clients themselves)

The **Remove clients...** button, title — **"Remove clients from group "{name}"**".

The verbatim hint:

"Select members to remove from this group. The clients themselves are kept (use "Delete group clients" for a full deletion)."

Behavior (`POST /panel/api/clients/groups/bulkRemove`, body `{"emails": [...]}`): technically this is the same as "Add to group" with an empty group. For the selected clients `group_name` is cleared, and in their inbounds the `group` field is removed from the Xray settings. The clients themselves and their inbound bindings remain.

Success message: **"Removed {count} client(s) from {name}."**

9.9. Resetting group traffic

The **Reset traffic** button.

Confirmation dialog:

- Title: **"Reset traffic for group {name}?"**
- Text: **"This will zero out up/down for all {count} client(s) in this group."**

Behavior: for all member emails of the group, `up` and `down` in the traffic table are zeroed out and the `enable` field is set to `true` (the client is enabled). The operation is performed in batches within a transaction.

Success message: **"Reset traffic for {count} client(s)."**

9.10. Deleting a group and deleting group clients

The page has **two fundamentally different delete operations** — they are easy to confuse, so the distinction is critical.

9.10.1. Delete group (keep clients)

The **"Delete group (keep clients)"** button.

Dialog:

- Title: **"Delete group {name}?"**
- Text: **"This deletes the group and clears its label from {count} client(s). The clients themselves are not deleted."**

Behavior (`POST /panel/api/clients/groups/delete` , body `{"name": "..."}`): the group record is deleted from `client_groups` , `group_name` is cleared for all of its clients, and the `group` field is removed from their inbounds. **The clients, their connections, and their traffic are kept.** Xray is marked for restart.

Success message: **"Cleared the group from {count} client(s)."**

9.10.2. Delete group clients (full deletion)

The **"Delete group clients"** button.

Dialog:

- Title: **"Delete all clients in {name}?"**
- Text: **"This permanently deletes {count} client(s) along with their traffic records. The group label is also cleared. This cannot be undone."**

This is a destructive operation: it deletes the clients themselves (via a bulk deletion by email, endpoint `POST /panel/api/clients/bulkDel`), including their traffic records, and thereby removes them from all inbounds.

Messages:

- Success: **"Deleted {count} client(s)."**
- Partial result: **"{ok} deleted, {failed} skipped"**

If the group is empty, actions over its members are unavailable — **"This group has no clients yet."** is shown.

9.11. Relationship with the "Clients" page

The group label is visible and used outside the **Groups** page as well:

- The compact client record has a `group` field, so the client list shows group membership.

- The client list (`/panel/api/clients/list/paged`) accepts a `group` filter parameter: you can pass a single name or several names separated by commas. Matching is done on an "OR" basis within the field, case-insensitively. A special case: an empty element in the filter's group list means "clients without a group" (whose `group` is empty).
- The clients page response returns a `groups` array — the full list of names of existing groups, so the UI can build the filter dropdown.

Example: filtering clients by group. This request returns only clients labeled `vip` or `trial` (several names are comma-separated, with "OR" semantics):

```
GET /panel/api/clients/list/paged?group=vip,trial
```

To get clients **without** a group, pass an empty element in the list — for example, the filter value `group=` (empty string) or `group=vip,` (the `vip` label plus clients with no group).

9.12. API endpoints summary

All group routes are mounted under `/panel/api/clients` :

Method and path	Purpose	Request body
GET <code>/panel/api/clients/groups</code>	List of groups with client counts	—
GET <code>/panel/api/clients/groups/:name/emails</code>	Emails of all members of a group (sorted by email)	—
POST <code>/panel/api/clients/groups/create</code>	Create an empty group	<code>{"name"}</code>
POST <code>/panel/api/clients/groups/rename</code>	Rename a group	<code>{"oldName", "newName"}</code>
POST <code>/panel/api/clients/groups/delete</code>	Delete a group, keeping clients (clear the label)	<code>{"name"}</code>
POST <code>/panel/api/clients/groups/bulkAdd</code>	Add clients to a group (by email)	<code>{"emails":[...], "group"}</code>
POST <code>/panel/api/clients/groups/bulkRemove</code>	Remove clients from a group (clear the label)	<code>{"emails":[...]}</code>
POST <code>/panel/api/clients/bulkDel</code>	Full deletion of clients (used by "Delete group clients")	<code>{"emails":[...], "keepTraffic"}</code>

Example: a typical group-lifecycle scenario via the API.

```
# 1. Create the trial label
curl -s ../panel/api/clients/groups/create -d '{"name":"trial"}'

# 2. Attach it to two clients
curl -s ../panel/api/clients/groups/bulkAdd -d '{"emails":
["u1@example.com","u2@example.com"],"group":"trial"}'

# 3. Remove one member from the group (label-only)
curl -s ../panel/api/clients/groups/bulkRemove -d '{"emails":["u2@example.com"]}'

# 4. Delete the group but keep the clients (label is just cleared)
curl -s ../panel/api/clients/groups/delete -d '{"name":"trial"}'
```

Step 4 removes the group record and clears `group_name` from its clients, but the clients themselves, their connections, and their traffic remain. To permanently delete the clients themselves, use `bulkDel` instead.

Operations that change clients' label (`rename` , `delete` , `bulkAdd` , `bulkRemove`) mark Xray as requiring a restart and send a notification about the client change.

9.13. Traffic by group

New in version 3.3.0: in the **Groups** section (the "Clients" page, group management tab) the groups table now shows not only the number of clients in each group, but also the total traffic consumed by the group. The column is labeled **"Traffic used"**.

What the column shows

For each group row, the sum of traffic across all clients in that group is shown — that is, the added-up `up` + `down` (sent + received traffic) of all its members. This gives a quick answer to the question "how much did the whole group download/upload in total" without having to open clients one by one and add it up manually.

Alongside in the groups table are shown:

Column	What it means
Name	The group name
Clients	How many clients are labeled with this group (the column was previously named "Clients in group")
Upload	The total <code>up</code> (sent traffic) across all clients of the group
Download	The total <code>down</code> (received traffic) across all clients of the group
Traffic used	The total <code>up</code> + <code>down</code> across all clients of the group

Sent and received traffic are shown as separate **Upload** and **Download** columns, while the **Traffic used** column shows their sum. The client-count column is now simply named **Clients**.

The summary above the table additionally shows aggregates across all groups — **"Total groups"** and **"Clients with a group"**, and the total traffic is split into two cards: **"Total upload / download"** (with up/down arrows — the sent and received traffic of all groups separately) and **"Total traffic"** (with a pie-chart icon — their combined total).

How it is calculated

The calculation is performed with a single SQL query against the clients table with a `LEFT JOIN` to the traffic accounting table:

- by the group label field (`group_name`) clients are grouped, their count is computed – this is the "Clients in group";
- traffic is taken as the sum of `up + down` from the joined `client_traffics` table. That is, both the sent (`up`) and received (`down`) bytes are summed for each client;
- since the email is unique both in the clients table and in the traffic table, the join does not double-count one client's traffic.

Value specifics:

- **Clients without a traffic record** are counted in the member count but add 0 to the sum, so a freshly created group shows traffic of `0`.
- **Empty groups** (created but without clients) are also present in the list with a zero count and zero traffic: besides groups "derived" from client labels, the explicitly stored groups are mixed into the result, after which the list is sorted by name case-insensitively.
- Clients without a group label (`group_name` empty) are not included in the calculation.

Related actions

From the groups table, actions over the whole group are still available, including **"Reset traffic"** — it zeroes out `up / down` for all clients of the selected group. After such a reset, the "Traffic used" column for that group shows `0`.

10. Subscriptions (Subscription)

A subscription is a mechanism that lets you give a client a single permanent link (URL) through which the VPN client itself downloads and periodically refreshes the full set of configurations. Instead of manually forwarding the user a separate link for each inbound, you give them a single address of the form `https://domain:port/sub/<subId>`. At that address the panel assembles, on the fly, all configurations bound to that client and serves them in the format the client expects. When server settings change (a new address, Reality-key rotation, an added inbound), the client receives the up-to-date configuration on its next automatic refresh, without requiring any action from the user.

The subscription is served by a separate HTTP/HTTPS server inside the panel, which starts independently of the web panel and listens on its own port. This is done for security reasons: the subscription port can be exposed to the outside without exposing the port of the panel itself.

10.1. What subId is and how the link is built

Every client in an inbound has a `subId` field (in the UI — "Subscription ID"). This value is precisely the subscription key: the panel searches across all inbounds for clients whose `subId` matches the requested one, and merges their configurations into a single response.

- If several clients (in the same inbound or in different inbounds) have the same `subId`, their configurations end up in one subscription. This is the standard way to give a single user several servers/protocols at once via one link.

Example: one user — two servers via one link. Suppose there are two inbounds (VLESS on server A and Trojan on server B). To hand the user both configurations through a single link, give both of his clients the same `subId`:

```
Inbound 1 (VLESS):  email = ivan@vpn,  subId = ivan2025
Inbound 2 (Trojan): email = ivan@vpn,  subId = ivan2025
```

Then at `https://sub.example.com:2096/sub/ivan2025` the panel will serve both configurations at once. If you add a third inbound later with the same `subId`, it appears for the user on the next automatic subscription refresh, without sending a new link.

- If a client's `subId` field is empty, you cannot share a public-access link. The UI indicates this with the hint: "This client has no subId, the sharing link is unavailable."
- The `subId` value cannot be set arbitrarily: on save it is checked that it contains no spaces, no `/`, `\`, or control characters. The corresponding validation hint: "Subscription ID cannot contain spaces, '/', or control characters".

The final link is built as `<base>/<subPath>/<subId>` (see the section on subscription server settings and the "Reverse proxy URI" field). If no client is found for the `subId` (the client was deleted, the `subId` does not exist), the server returns HTTP 404 with no body. On an internal error — HTTP 500. VPN clients rely solely on the response code, so the error body is intentionally empty.

Inbound link order in the subscription

Every inbound has a **"Subscription order"** field (`subSortIndex`) – a number from 1 that sets the position of this inbound's links in the subscription output. Lower values come first; ties keep the original creation order (by id). The order applies to all output formats – raw text, the subscription page, JSON, and Clash. It does not affect the order of inbounds in the panel itself.

The field is edited in the inbound form next to the share-address settings and propagates to nodes by the usual sync rules. If at least one inbound has an order other than 1, a compact **"Order"** column appears in the Inbounds list.

10.2. Subscription server settings

All subscription parameters are located in the panel settings under the **"Subscription"** tab. Each parameter is explained below; the internal setting key and the default value are given in parentheses.

The section itself is split into tabs: **"Panel settings"**, **"Information"**, **"Profile"**, **"Certificates"**, **"Happ"**, and **"Clash / Mihomo"**. The subscription title, support URL, profile page, announcement, and theme directory fields are on the "Profile" tab; the Happ and Clash/Mihomo routing rules are on the tabs of the same name; the subscription update interval is on the "Information" tab.

Basic parameters

Field (UI)	Key	Default value	Description
Enable subscription	<code>subEnable</code>	<code>true</code> (enabled)	Starts the separate subscription server. Hint: "Subscription feature with separate configuration". If disabled – the subscription server does not start, and none of the links work.
Listen IP	<code>subListen</code>	empty	The IP address on which the subscription server accepts connections. Hint: "Leave empty by default to listen on all IP addresses".
Subscription port	<code>subPort</code>	<code>2096</code>	The TCP port of the subscription server. Hint: "The port number used to serve the subscription service must not be in use on the server" – the port must be free and must not conflict with the panel or Xray.
URI path	<code>subPath</code>	<code>/sub/</code>	The path at which ordinary subscriptions are served. Hint: "Must start with '/' and end with '/'".
Listening domain	<code>subDomain</code>	empty	The domain through which subscription access is allowed (Host validation). Hint: "Leave empty by default to listen on all domains and IP addresses". If set – requests with a different Host are rejected.

Important for security: the default path `/sub/` (and `/json/` for JSON) is widely known and easily guessed. The panel shows a warning: "The default subscription path `/sub/` is widely known – change it." and a similar one for JSON. It is recommended to set your own non-obvious path.

TLS / certificate

Field (UI)	Key	Default	Description
Path to the subscription certificate public key file	<code>subCertFile</code>	empty	The full path to the certificate file (<code>.crt</code> / <code>fullchain</code>). Hint: "Enter the full path starting with '/'".
Path to the subscription certificate private key file	<code>subKeyFile</code>	empty	The full path to the private key file. Hint: "Enter the full path starting with '/'".

If both paths are set and the certificate loads successfully, the subscription server runs over **HTTPS**. If the fields are empty or the certificate could not be read – the server falls back to **HTTP** (the error is written to the log). The presence of valid TLS also affects how the base URL is built: with port 443 and TLS, and with port 80 without TLS, the port number is omitted from the link.

Update interval

Field (UI)	Key	Default	Description
Subscription update intervals	<code>subUpdates</code>	<code>12</code>	How often (in hours) the client application should re-request the subscription. Hint: "The interval between updates in the client application (in hours)".

The value is passed to the client in the `Profile-Update-Interval` HTTP header; modern clients use it as the configuration auto-update period.

Format and information in the response

Field (UI)	Key	Default	Description
Encode	<code>subEncrypt</code>	<code>true</code>	Hint: "Encrypt the configs returned in the subscription". Technically this is not encryption but Base64 encoding of the entire body of an ordinary subscription (the format that most clients expect). When disabled, the links are served as plain text, one per line.
Show usage info	<code>subShowInfo</code>	<code>true</code>	Hint: "Show the remaining traffic and the expiry date after the config name". When enabled, markers for remaining traffic (<code>📶</code>) and expiry term (e.g. <code>5D, 3H</code>) are appended to the name (remark) of each configuration; for an expired/unavailable client, <code>N/A</code> is shown.
Include Email in remark	<code>subEmailInRemark</code>	<code>true</code>	Hint: "Include the client's email in the subscription profile name.". Adds the client's email to the profile remark.
Remark model and separator character	<code>remarkModel</code>	<code>-ieo</code>	Defines which parts and in what order form the displayed name (remark) of each configuration, as well as the separator character. The letters encode the component parts (e.g. inbound, email, etc.), and the first character is the separator. A "Remark example" is shown next to it.

Profile metadata (response headers)

These strings are passed to the client in HTTP response headers and are displayed in the VPN client as profile metadata. All of them are empty by default.

Field (UI)	Key	Header	Description
Subscription title	subTitle	Profile-Title (in Base64)	"The subscription name that the client sees in the VPN client". For Clash it is also used as the name of the imported profile via Content-Disposition.
Support URL	subSupportUrl	Support-Url	"A link to technical support, displayed in the VPN client".
Profile URL	subProfileUrl	Profile-Web-Page-Url	"A link to your website, displayed in the VPN client". If not set, the actual subscription request URL is substituted.
Announcement	subAnnounce	Announce (in Base64)	"The announcement text displayed in the VPN client".

In addition, each response carries the Subscription-Userinfo header with the client's aggregated traffic data: upload, download, total, and expire (the expiry moment in seconds). The client uses it to show the remaining traffic and the expiry term.

Routing (Happ client only)

Field (UI)	Key	Default	Description
Enable routing	subEnableRouting	false	"A global setting to enable routing in the VPN client. (Happ only)". Passed in the Routing-Enable header.
Routing rules	subRoutingRules	empty	"Global routing rules for the VPN client. (Happ only)". Passed in the Routing header.

Reverse proxy URI

Field (UI)	Key	Default	Description
Reverse proxy URI	subURI	empty	"Change the base URI of the subscription URL for use behind proxy servers".

If the field is empty, the panel builds the base address of the link itself from the subscription domain and port (taking TLS into account). But if the subscription is served through an external reverse proxy/CDN on a different domain or path, this field is set to the final base URI, and all links will be built from it. Analogous separate fields exist for JSON (subJsonURI) and Clash (subClashURI).

If only the common subURI is set while the separate JSON and Clash fields are left empty, the links of those formats on the subscription page inherit the scheme and host from subURI (instead of the sub-server port and http) – so they match the reverse-proxy address.

Example: subscription behind a reverse proxy. The subscription itself listens on 2096, but is exposed externally via nginx/CDN at https://cfg.example.com/u/. So that the links in the response are built from

the external address rather than the internal `domain:2096`, set the final base URI in the "Reverse proxy URI" field:

```
Reverse proxy URI: https://cfg.example.com/u
```

The final link then takes the form `https://cfg.example.com/u/ivan2025`. For the JSON and Clash formats, fill in the separate `subJsonURI` and `subClashURI` fields the same way if needed.

10.3. Output formats

A subscription can be served in three independent formats, each with its own endpoint that can be enabled/disabled separately.

Server address and nodes in the output

The server address in subscription links is substituted using the same share-address strategy as the panel's ordinary links and QR codes: "listen" — a routable bind address, "custom" — the configured user address (`shareAddr`), "node" (the default) — the node's address. For inbounds with no explicitly set strategy, the subscription output is unchanged. This lets a node-hosted inbound bound to a specific public IP advertise a reachable address to clients. The strategy applies to the raw, JSON, and Clash formats.

For an inbound hosted on a node, the node name is automatically appended to the name (remark) in the subscription via "@" (e.g. `my-inbound@node1`), unless the admin already included it. This removes the situation where, in a multi-node subscription, several identically-named entries differed only by address.

If, due to drift between nodes, the same client ended up twice in an inbound's internal JSON, the subscription output automatically removes such duplicates by email in all three formats, so repeated profiles no longer appear in the output.

Ordinary links (SUB) — Base64 / plain text

The base format, the `subPath` endpoint (default `/sub/`). Always enabled (when subscriptions are enabled overall). It returns a list of Xray links (`vless://`, `vmess://`, `trojan://`, `ss://`, etc.) — one per line. When the "Encode" option (`subEncrypt`) is enabled, the entire list is Base64-encoded; when disabled — it is served as plain text. This format is understood by virtually all clients (v2rayNG, V2RayTun, Sing-box, NekoBox, Streisand, Shadowrocket, Happ, and others).

Example: response body with "Encode" disabled. With `subEncrypt = false`, the `/sub/` endpoint serves plain text — one link per line:

```
vless://3c8f...@a.example.com:443?security=reality&...#srvA-ivan
trojan://p4ss@b.example.com:443?security=tl&...#srvB-ivan
```

With `subEncrypt = true` (the default), the same list as a whole is Base64-encoded and served as a single string — this is exactly the form most clients expect.

JSON subscription (sing-box and compatible)

The `subJsonPath` endpoint (default `/json/`), enabled by a separate checkbox.

Field (UI)	Key	Default	Description
JSON subscription	<code>subJsonEnable</code>	false	"Enable/disable the JSON subscription endpoint independently."

Returns the full JSON configuration (the format understood by sing-box and derivative clients – Podkop, OpenWRT sing-box, Karing, NekoBox). Additional parameters are available for this format (the `subFormats` tab):

- **Mux** (`subJsonMux` , empty by default) – JSON multiplexing (Mux) settings that are injected into the outbound of every JSON-subscription stream. "Transmitting multiple independent data streams over a single connection."
- **Final Mask** (`subJsonFinalMask` , empty by default) – "Xray finalmask masks (TCP/UDP) and QUIC settings added to every JSON-subscription stream. Requires a recent version of xray on the client.". Configured through subfields: "Packets" (`packets`), "Length" (`length`), "Interval" (`interval`), "Max split" (`maxSplit`), "Noises" (`noises` : "Type"/ `type` , "Packet"/ `packet` , "Delay (ms)"/ `delayMs` , "Apply to"/ `applyTo` , "+ Noise" button), as well as "Concurrency" (`concurrency`), "xudp concurrency" (`xudpConcurrency`), and "xudp UDP 443" (`xudpUdp443`).
- **Routing rules** (`subJsonRules` , empty by default) – global rules added to the JSON configuration.

Clash / Mihomo subscription (YAML)

The `subClashPath` endpoint (default `/clash/`), enabled by a separate checkbox.

Field (UI)	Key	Default	Description
Clash / Mihomo subscription	<code>subClashEnable</code>	false	Enables generation of a YAML configuration for Clash and Mihomo clients.
Enable routing	<code>subClashEnableRouting</code>	false	"Add global Clash/Mihomo routing rules to the generated YAML subscriptions."
Global routing rules	<code>subClashRules</code>	empty	"Clash/Mihomo rules added at the beginning of each YAML subscription before MATCH,PROXY."

The response is served with the type `application/yaml; charset=utf-8`. If the "Subscription title" (`subTitle`) is set, it is also passed in the `Content-Disposition` header (`attachment; filename*=UTF-8' '<title>`), so that the Clash client names the imported profile with this name.

Note: this build supports exactly three formats – ordinary links (Base64/text), JSON (sing-box-compatible), and Clash/Mihomo (YAML). There is no separate Outline format in the subscription server.

10.4. Subscription information page and QR codes

If you open the subscription link in a browser (or explicitly add the `?html=1` or `?view=html` parameter to the URL, or send the `Accept: text/html` header), the server, instead of the "raw" response, serves a visual **subscription information page** ("Subscription information"). VPN clients still receive the machine response, since they do not request HTML.

The page (a single-page application built with Vite) shows:

- **Subscription information** (a Descriptions block):
 - "Subscription ID" — the `subId` value;
 - "Status" — "Active", "Inactive", or "Unlimited". The "inactive" status is set if the client is disabled, has exhausted the traffic limit, or has expired;
 - "Downloaded" and "Uploaded" — traffic volumes;
 - "Total quota" — the traffic limit, or ∞ if unlimited;
 - "Expiry" — the expiry date, or "No expiry";
 - the remaining traffic and the last-online time.
 - Dates are displayed using the Gregorian or Jalali calendar depending on the panel's "Calendar Type" setting (`datepicker` , default `gregorian`).
- **Subscription links**: for each enabled format — a separate row with a colored tag (green **SUB**, purple **JSON**, gold **CLASH**), a copy button, and a **QR code** button (a pop-up, size 240 px). The rows for JSON and CLASH appear only if the corresponding format is enabled in the settings.
- **Individual links** ("Copy link"): the full list of individual configurations included in the subscription, each with its own protocol tag, copy button, and QR code (for post-quantum links no QR is built).
- **A "Copy all configs" button** (above the list of individual links): with a single click it copies all configuration links to the clipboard at once (each on its own line), without copying them one by one; when done, the notification "All configs copied" is shown.
- **Quick-import buttons for apps** (drop-down menus by platform): for Android — `v2box`, `v2rayNG` (deep-link `v2rayng://install-config?url=...`), `Sing-box`, `V2RayTun`, `NPV Tunnel`, `Happ` (`happ://add/...`); for iOS — `Shadowrocket` (via the `flag=shadowrocket` parameter), `v2box` (`v2box://install-sub?url=...&name=...`), `Streisand` (`streisand://import/...`), `V2RayTun`, `NPV Tunnel`, `Happ`. These buttons either open the deep-link of the required app with the subscription address already filled in, or copy the link to the clipboard.

The information page is served with cache-prevention headers (`Cache-Control: no-cache`), so that the client always sees up-to-date data on traffic and expiry.

10.5. Custom subscription page templates

Starting from 3.3.0, you can replace the standard subscription landing page with your own HTML template. By default the built-in page is served at the subscription address, but if you specify a directory with your own template, the panel will render it and substitute the current client data into it (traffic, expiry, links, etc.).

Important: the panel does **not** ship any ready-made templates. The repository contains only the `sub_templates/` directory with an instruction file `sub_templates/README.md` ; you need to create your own theme yourself.

Where it is enabled

The theme directory is set in the panel settings:

Settings → **Subscription** → the **"Subscription information" section**, the **"Subscription theme directory"** field (`subThemeDir`).

The field description in the UI: "The absolute path to the folder with a custom template (`index.html/sub.html`) for the subscription page (e.g. `/etc/3x-ui/sub_templates/my-theme/`). Leave empty to use the default page."

In the same section nearby are related settings whose values are available in the template:

The "Subscription theme directory" field description includes a **"Template guide"** [↗](#) link to the documentation on creating your own subscription-page themes.

- **"Subscription title"** (`subTitle`) — the name visible to the client;
- **"Support URL"** (`subSupportUrl`) — the link to technical support.

The setting parameter

Parameter	Default value	Purpose
<code>subThemeDir</code>	<code>""</code> (empty)	The absolute path to the directory with your HTML template. Empty = the built-in default page.

How to substitute your own template

1. Create a folder for the theme on the server (anywhere), for example `/etc/3x-ui/sub_templates/my-theme/`.
2. Put inside it an HTML file named `index.html` or `sub.html`.

Example: path to the theme. The final layout on the server and the value of the settings field:

```
/etc/3x-ui/sub_templates/my-theme/
└─ index.html      (or sub.html – it takes priority)
```

```
Settings → Subscription → "Subscription theme directory":
/etc/3x-ui/sub_templates/my-theme/
```

The path must be **absolute** (start with `/`). If the folder contains neither `index.html` nor `sub.html`, the panel serves the built-in page. 3. In the panel, open **Settings** → **Subscription** and enter the **absolute** path to this folder in the "Subscription theme directory" field. 4. Save the settings.

File-selection and rendering behavior:

- If the directory contains `sub.html`, it is used; otherwise `index.html` is taken. That is, `sub.html` takes priority over `index.html`.
- The template is rendered by the standard Go `html/template` engine.
- The parsed template is **cached** and re-read from disk only when the file's modification time changes. Therefore template edits are picked up without restarting the panel, but without the overhead of reading/parsing on every request.
- The response is assembled into a buffer in full and only then served to the client: if the template fails during execution, the partially generated (broken) page will not be sent to the user.

Default behavior and fallback

- The field is empty → the built-in SPA page is served (the data is injected into `window.__SUB_PAGE_DATA__`).
- The path does not exist or is not a directory → the default page is used.
- The directory contains neither `index.html` nor `sub.html` → the warning "subThemeDir set but no usable template found" is written to the log, and the default page is served.
- The template file exists but does not parse → the error "custom template parse failed" is written to the log, and the default page is served.
- An error while executing the template → "custom template execution failed" is written to the log, and the default page is served.

That is, any problem with the custom template does not "break" the subscription – the panel always degrades to the built-in page. All subscription pages (both custom and standard) are served with cache-prevention headers (`Cache-Control: no-cache, no-store, must-revalidate`), so that clients always receive fresh data on traffic and expiry.

Available template variables

A set of subscription-client data is passed into the template context. Access is via `{{ .name }}`:

Variable	Type	Description
<code>{{ .sId }}</code>	string	The subscription ID (UUID).
<code>{{ .enabled }}</code>	bool	Whether the client/subscription is enabled.
<code>{{ .download }}</code>	string	Formatted download volume (e.g. "2.5 GB").
<code>{{ .upload }}</code>	string	Formatted upload volume.
<code>{{ .total }}</code>	string	Formatted total traffic limit.
<code>{{ .used }}</code>	string	Formatted used traffic (download + upload).
<code>{{ .remained }}</code>	string	Formatted remaining traffic.
<code>{{ .expire }}</code>	int64	The expiry – Unix time in seconds (<code>0</code> = no expiry). For a JS <code>Date</code> , multiply by 1000.
<code>{{ .lastOnline }}</code>	int64	The last-online time – Unix time in milliseconds (<code>0</code> = never).

Variable	Type	Description
<code>{{ .downloadByte }}</code>	int64	Download in exact bytes.
<code>{{ .uploadByte }}</code>	int64	Upload in exact bytes.
<code>{{ .totalByte }}</code>	int64	The total limit in exact bytes.
<code>{{ .subUrl }}</code>	string	The URL of the subscription page.
<code>{{ .subJsonUrl }}</code>	string	The URL of the subscription's JSON configuration.
<code>{{ .subClashUrl }}</code>	string	The URL of the Clash/Mihomo configuration.
<code>{{ .subTitle }}</code>	string	The subscription title from settings (may be empty).
<code>{{ .subSupportUrl }}</code>	string	The support URL from settings (may be empty).
<code>{{ .links }}</code>	[]string	The list of config strings (VMess, VLESS, etc.). Iterate with: <code>{{ range .links }} ... {{ end }}</code> .
<code>{{ .emails }}</code>	[]string	The list of emails belonging to the subscription.
<code>{{ .datepicker }}</code>	string	The panel's current calendar format: <code>gregorian</code> or <code>jalali</code> (taken from the "Calendar Type" setting; if empty – <code>gregorian</code>).

A minimal example of a template body that uses some of the variables:

```
<h1>{{ .subTitle }}</h1>
<p>Used: {{ .used }} of {{ .total }} (remaining {{ .remained }})</p>
{{ range .links }}<div>{{ . }}</div>{{ end }}

**Example: expiry date from `expire`.** The `{{ .expire }}` field is Unix time in
**seconds**, so for JavaScript you multiply it by 1000; a value of `0` means "no
expiry":

```html
<script>
 var exp = {{ .expire }};
 document.write(exp === 0
 ? 'No expiry'
 : 'Until ' + new Date(exp * 1000).toLocaleDateString());
</script>
```

Note: `{{ .lastOnline }}` is already in **milliseconds** – do not multiply it by 1000.

---

## ## 11. Xray: routing, outbounds, DNS, and extensions

The `**Xray Settings**` section is the editor for the Xray-core configuration template, from which the panel generates the final `config.json` used to launch the core. The section's template hint reads: `*"The Xray configuration file is generated from the template."*` Unlike inbounds (which are stored separately in the database and substituted into the template when the configuration is assembled), everything else – logs, routing, outbounds, DNS, policy, statistics – is defined here.

> Important: the template value is stored in the database under the key ``xrayTemplateConfig``. On save, the panel runs it through a series of automatic transformations (see [11.10](#1110-saving-restart-and-automatic-transformations)). Any syntactically invalid JSON is rejected with the error `*"xray template config invalid"*`.

### ### 11.1. Editor layout: tabs/modes

The editor offers several modes for displaying the template (filters over the JSON sections):

Mode	What it shows
<code>**Basic**</code>	Basic sections (Log, basic routing, general settings)
<code>**Advanced template**</code>	The full JSON Xray template
<code>**All**</code>	All sections at once

Logical setting groups inside the editor:

- `**General settings**` (hint: `*"These parameters describe general settings"*`).
- `**Log**` (see [11.9](#119-logs-and-statistics-stats-metrics)).
- `**Basic connections**`: blocks and direct routes.
- `**Inbounds**` (hint: `*"Edit the configuration template to connect specific clients"*`).
- `**Outbounds**` (see [11.4](#114-outbounds-outgoing-connections)).
- `**Balancer**` (see [11.5](#115-balancers)).
- `**Routing**` (hint: `*"The priority of each rule matters!"*`, see [11.3](#113-routing-rules-routing)).
- `**DNS / Fake DNS**` (see [11.6](#116-dns)).

### ### 11.2. General settings (General)

#### #### Freedom Protocol Strategy

Field	Label	Description	Default
<code>`FreedomStrategy`</code>	<code>**Freedom protocol strategy setting**</code>	The network output strategy for the direct (freedom) outbound. Hint: <code>*"Set the network output strategy in the Freedom protocol"*</code> . Controls the <code>`domainStrategy`</code> field inside the <code>`settings`</code> of the outbound with protocol <code>`freedom`</code> .   In the reference template, <code>`domainStrategy`</code> for the freedom outbound <code>`direct`</code> is <code>**`AsIs`**</code> (the address is not resolved and is passed through unchanged).	

``domainStrategy`` for freedom (Xray-core values): ``AsIs`` (do not resolve the domain on the server side), and also the ``UseIP`` / ``UseIPv4`` / ``UseIPv6`` family and their "forced" variants ``ForceIP*``, which make the exit server resolve the domain and connect to the resulting IP. Change it to ``UseIPv4`` if the exit server has no IPv6 or if you need to

force IPv4-only egress.

#### #### Freedom Happy Eyeballs (IPv4/IPv6)

Field	Label	Description
`FreedomHappyEyeballs`	**Freedom Happy Eyeballs (IPv4/IPv6)**	Hint: <i>"A dual-stack set for the direct (freedom) outbound – useful on exit servers with both IPv4 and IPv6." Enables the Happy Eyeballs algorithm (simultaneous attempts over both address families) for the freedom outbound.</i>
try delay	(hint)	<i>"Milliseconds before attempting the other address family. 150–250 ms is a good starting point." The delay before switching to the alternative address family. The recommended range is 150–250 ms.</i>

#### #### Overall Routing Strategy

Field	Label	Description	Default
`RoutingStrategy`	**Domain routing setting**	The overall DNS resolution strategy for routing. Hint: <i>"Set the overall DNS resolution routing strategy"*. Controls the `routing.domainStrategy` field. In the reference template, `routing.domainStrategy` = <i>**`AsIs`**.</i></i>	

`routing.domainStrategy` determines how IP-based routing rules are matched against domain requests: *`AsIs`* (domain rules only, no resolution), *`IPIfNonMatch`* (if the domain does not match any rule – resolve it and check the IP rules), *`IPOnDemand`* (resolve immediately upon encountering an IP rule). For IP rules (for example, *`geopip:\*`*) to work with a domain request, *`IPIfNonMatch`* is usually required.

#### #### Outbound Test URL

Field	Label	Description	Default
`outboundTestUrl`	**Outbound test URL**	The URL used to check connectivity when testing an outbound. Hint: <i>"URL for checking outbound connectivity"*. Stored separately from the template, under the key `xrayOutboundTestUrl`.   <i>**`https://www.google.com/generate_204`**</i></i>	

The value is sanitized. During the actual outbound test it is additionally checked as a public URL – this is SSRF protection: a user cannot inject an arbitrary (including internal) URL via the client; the test URL is always taken from the server-side setting. An empty value on save/test is replaced with the default *`generate\_204`*.

#### #### Block BitTorrent

Field	Label	Description
`Torrent`	**Block BitTorrent**	Adds a rule to <i>`routing.rules`</i> that sends traffic with <i>`protocol: ["bittorrent"]`</i> to the <i>`blocked`</i> outbound. In the reference template this rule is present by default.

#### #### Connection Limits

Hint: *"Connection-level policies for level-0 users. Leave the field empty to use Xray's default value." These parameters are written to *`policy.levels.0`*.*

Field	Label	Description	Default



```
| `connIdle` | **Idle timeout** (seconds) | "Closes the connection after it has been idle for the specified number of seconds. Lowering the value frees memory and file descriptors faster on loaded servers (Xray default: 300)."* | empty → Xray default **300** |
| `bufferSize` | **Buffer size** (KB) | "The size of the internal per-connection buffer in KB. Set 0 to minimize memory usage on servers with little RAM (Xray's default value depends on the platform)."* Placeholder: **"auto"**. | empty → platform-dependent; `0` – minimize |

Example (`policy.levels.0`) The fields in this group are written to the level-0 policy. On a loaded server with little RAM you can free resources faster like this:

```json
{
  "policy": {
    "levels": {
      "0": {
        "connIdle": 120,
        "bufferSize": 0
      }
    }
  }
}
```

Here a connection is closed after just 120 s of idle time (instead of the default 300), and `bufferSize: 0` minimizes buffer memory usage. A field left empty in the form simply won't appear in the JSON – and Xray will apply its own default.

11.3. Routing rules (routing)

The `routing.rules` list. **Order is critical** (*"The priority of each rule matters!"*): rules are evaluated top to bottom, and the first match wins. Hint: *"Drag to reorder"*. Order-control buttons: **First, Last, Move up, Move down**.

Each rule has `type: "field"`. Buttons: **Add rule, Edit rule**. Hint for the list fields: *"Comma-separated items"*.

Route Tester

The Routing tab has a **Route Tester** sub-tab that asks the running Xray which outbound would handle a specific connection, without sending any real traffic. Enter a domain or IP, a port, the network (TCP/UDP) and, optionally, an inbound and the sniffed protocol (`http` / `tls` / `quic` / `bittorrent`), then press **Test Route**. The decision is taken straight from the live routing engine.

The result shows the selected outbound and, when a balancer is involved, its balancer tag as well. If no rule matched, the tester reports that the traffic goes to the default outbound (the first one in the `outbounds` list). This is handy for verifying rule order before relying on it.

Rule form fields

Form field	Label (RU)	JSON field	Description
Source	Source	<code>source</code>	Source IP addresses/subnets. Comma-separated list.
Source port	Source port	<code>sourcePort</code>	Source port(s).

Form field	Label (RU)	JSON field	Description
Destination	Destination	<code>domain + ip + port</code>	Target domains, IPs, and ports. Domains support the prefixes <code>domain:</code> , <code>full:</code> , <code>regexp:</code> , <code>keyword:</code> , as well as <code>geosite:*</code> ; IPs support <code>geoip:*</code> and CIDR.
Network	—	<code>network</code>	<code>tcp</code> , <code>udp</code> , or <code>tcp,udp</code> .
Protocol	—	<code>protocol</code>	<code>http</code> , <code>tls</code> , <code>bittorrent</code> (detected via sniffing).
User	User	<code>user</code>	Filter by user e-mail/identifier.
Attributes / Value	Attributes / Value	<code>attrs</code>	HTTP-header attributes to match.
VLESS route	VLESS route	—	Routing by the route field for VLESS.
Inbound tags	Inbound tags	<code>inboundTag</code>	One or more inbound tags to which the rule applies (including the built-in <code>api</code> , and the DNS tag from the DNS settings). In inbound lists it is shown as "tag (remark)" when the inbound has a distinct remark, otherwise just the tag; saved rules still store only the tags.
Outbound tag	Outbound tag / Outbound connection	<code>outboundTag</code>	Where to send matched traffic.
Balancer tag	Balancer tag / Balancer	<code>balancerTag</code>	Hint: <i>"Routes traffic through one of the configured load balancers".</i>

`outboundTag` and `balancerTag` are mutually exclusive: *"You cannot use balancerTag and outboundTag at the same time. If both are used, only outboundTag will take effect."* In a single rule, specify either an outbound tag or a balancer tag.

Built-in rules of the reference template

In the standard `config.json`, the `routing` section contains three rules (in this order):

1. `inboundTag: ["api"] → outboundTag: "api"` — a service rule for the panel's gRPC statistics API.
2. `ip: ["geoip:private"] → outboundTag: "blocked"` — blocks private ranges.
3. `protocol: ["bittorrent"] → outboundTag: "blocked"` — blocks BitTorrent.

The `api → api` rule is always automatically moved to position 0 on save (see [11.10](#)), so that the statistics request is not "swallowed" by a higher-priority catch-all rule.

Rule example. Send all traffic to Russian sites and private networks directly (bypassing the proxy), and route the rest to a balancer. Order matters: the "send directly" rule must sit above the catch-all. In `routing.rules`:

```
{
  "type": "field",
  "domain": ["geosite:category-ru", "domain:example.ru"],
  "ip": ["geoip:ru", "geoip:private"],
  "outboundTag": "direct"
}
```

For the IP rules (`geoip:ru`) to fire for domain requests too, the top-level routing usually needs `routing.domainStrategy: "IPIfNonMatch"` (see [11.2](#)).

Preconfigured routing groups (Basic connections)

In "Basic connections" mode, the panel helps assemble common rules from ready-made lists:

Group	Fields	Hint
Block by protocol/site	—	"Configure this so that clients cannot access certain protocols"
Block by country	Blocked IP addresses, Blocked domains	"These parameters will block traffic depending on the destination country."
Direct connections	Direct IP addresses, Direct domains	"A direct connection means that certain traffic will not be forwarded through another server."
IPv4 rules	—	"These parameters will let clients route to the target domains over IPv4 only"
WARP rules	—	"These options will route traffic to specific destinations through WARP"
NordVPN routing	—	"These options will route traffic to specific destinations through NordVPN."

MTPROTO inbound: routing Telegram traffic through Xray

An MTPROTO inbound has a **"Route through Xray"** toggle (off by default) and an optional **Outbound** picker. When enabled, the panel injects a loopback SOCKS bridge into the Xray config tagged with the inbound's own tag, and mtg dials Telegram through it. The router then governs the Telegram egress: it can be matched by ordinary rules on the Routing tab via the inbound tag, or forced to a chosen outbound or balancer through the **Outbound** field. Leave **Outbound** empty to let the routing rules decide.

11.4. Outbounds (outgoing connections)

The `outbounds` list. Buttons: **Add outbound connection**, **Edit outbound connection**. Hint: *"Edit the configuration template to define this server's outbound connections"*.

The reference template has two mandatory outbounds:

- `protocol: "freedom", tag: "direct"` — direct egress to the internet (with `domainStrategy: "AsIs"` and `finalRules: [{action: "allow"}]`);
- `protocol: "blackhole", tag: "blocked"` — a "black hole" for blocked traffic.

Common outbound form fields

Field	Label (RU)	Description
Tag	Tag (hint: "A unique tag")	Unique identifier of the outbound. Placeholder: "unique-tag". Validation: "Tag is required", "This tag is already used by another outbound".
Protocol	—	The outbound type (see below).
Address / Port	Address / Port	The connection target. Address and port are required.
Send through	Send through	The local IP address of the outgoing interface (<code>sendThrough</code>). Placeholder: "local IP".
Dialer proxy (chain)	—	Hint: "Connect this outbound through another outbound (by tag) to build a proxy chain. Leave empty for a direct connection." Placeholder: "Select an outbound for the chain". Implemented via <code>streamSettings.sockopt.dialerProxy</code> .

Supported outbound protocols

The protocols supported by the form:

- **freedom** — direct egress. Fields `settings.domainStrategy`, `finalRules` (see below), Happy Eyeballs. Not testable ("Outbound has no testable endpoint").
- **blackhole** — discards traffic. Field **Response type**. Not testable.
- **socks**, **http** — a `settings.servers[]` list with `address / port`; field **Auth password**.
- **vmess** — `settings.vnext[]` (`address / port`).
- **vless** — `settings.address / settings.port`.
- **trojan**, **shadowsocks** — `settings.servers[]`.
- **wireguard** — `settings.peers[]` with `endpoint`, plus keys (see [11.7](#)).
- **hysteria** — `settings.address / settings.port` (UDP transport).

Example: a chain through an upstream SOCKS. The `upstream` outbound dials an external SOCKS5 proxy, and `chained` sends its traffic through it (`dialerProxy`), forming a chain. In `outbounds`:

```
[
  {
    "tag": "upstream",
    "protocol": "socks",
    "settings": {
      "servers": [{ "address": "203.0.113.10", "port": 1080 }]
    }
  },
  {
    "tag": "chained",
    "protocol": "freedom",
    "streamSettings": {
      "sockopt": { "dialerProxy": "upstream" }
    }
  }
]
```

Now a routing rule with `outboundTag: "chained"` will egress to the internet through `upstream`.

Mux fields (multiplexing)

Max concurrency, Max connections, Max reuse times, Max request times, Max reusable seconds, Keep alive period. These parameters configure the outbound's mux/XUDP behavior.

Sockopts (socket settings)

The **Sockopts** group: **Keep alive interval, Mark (fwmark), Interface, IPv6 only, Accept proxy protocol, Proxy protocol, TCP user timeout (ms), TCP keep-alive idle (s).** The dialer-proxy chain is also configured here.

Freedom finalRules (overriding the private-IP block)

For a freedom outbound, the **Final rules** group is available:

Field	Label	Description
<code>overrideXrayPrivateIp</code>	Override Xray's default private-IP block	Removes Xray's built-in prohibition on outgoing connections to private IPs.
<code>action</code>	Action	<code>allow</code> (as in the reference template: <code>finalRules: [{action: "allow"}]</code>), <code>redirect</code> (Redirect), or others.
<code>blockDelay</code>	Block delay (ms)	The delay before dropping the connection.
<code>redirect / fragment</code>	Redirect / Fragment	Traffic redirection and fragmentation actions.

Other form fields

- **UDP over TCP** and **UoT version** — for shadowsocks-like protocols.
- **No gRPC header, Uplink chunk size** — gRPC transport parameters.
- TLS/uTLS fields: **Verify peer name, Pinned SHA256, Short ID, Vision testpre**, placeholder "server name".

Testing outbounds

Buttons: **Test, Test all.** States: **Testing connection..., Test succeeded, Test failed, Failed to test the outbound connection.** Result: **Test result**, the latency in milliseconds.

Two modes (hint: "*TCP: a fast dial-only probe. HTTP: a full request through xray.*"):

- **TCP** (`mode=tcp`) — a simple dial to `host:port`, performed in parallel across all endpoints, ~5 s timeout. Checks only TCP reachability and does not validate the proxy protocol. For `freedom / blackhole / the blocked` tag it returns "*Outbound has no testable endpoint*".
- **HTTP** (`mode=http` or empty) — spins up a temporary Xray instance and runs a real HTTP request (probe URL = the server-side `outboundTestUrl`), measuring real latency. The authoritative but expensive mode: it is serialized by a global lock ("*Another outbound test is already running, please wait*"). The per-attempt timeout is 10 s and the result-wait window is 15 s (raised so that healthy outbounds on slow or tunneled

links are not marked "Failed"). On failure the real cause (DNS error, connection refused, deadline exceeded, TLS error, etc.) is written to the panel/Xray log, which the generic timeout messages point to.

UDP protocols (`wireguard` , `hysteria`) and UDP transports (`kcp` , `quic` , `hysteria`) are **always** tested in HTTP mode, even if TCP is requested – a bare UDP dial cannot distinguish a "live" endpoint from a "dead" one. For `wireguard`, the test configuration forcibly sets `noKernelTun: true` .

Batched testing and per-stage breakdown

In HTTP mode, **Test** and **Test all** bring up a single shared temporary Xray instance per batch of outbounds, create a loopback SOCKS inbound with a rule for each one, and send a real HTTP request through each in parallel; **Test all** checks outbounds in batches. Besides success/failure, the result popup shows the HTTP response status and a time breakdown by stage: **Proxy connect** (connecting to the proxy), **TLS via outbound** (the TLS handshake through the outbound), and **First byte** (time to the first byte) – which helps pinpoint at which step the latency or failure occurred.

Outbound traffic statistics

The panel keeps per-tag traffic counters (`up / down / total`). The reset button resets the counters for a specific tag or for all of them (`tag = "-alltags-"`). The **Account info** and **Outbound status** fields display a summary.

11.5. Balancers

The `routing.balancers` list. Buttons: **Add balancer**, **Edit balancer**.

The Balancers tab has live-state columns: **Live Target** shows the balancer's current active target in the running Xray, and **Override** lets you manually override the target choice (the **Auto (strategy)** value returns to the strategy-based selection). A refresh button updates the live state. If the balancer is not yet active in the running Xray, the panel prompts you to save your changes or start Xray first.

Field	Label (RU)	Description
Tag	Tag (hint: "A unique tag")	Unique identifier. Placeholder: <i>"unique balancer tag"</i> . Validation: <i>"Tag is required"</i> , <i>"This tag is already used by another balancer"</i> .
Selectors	Selectors	A list of outbound tags (matched by substring) among which the balancer chooses the egress. At least one must be selected: <i>"Select at least one outbound"</i> .
Fallback	Fallback	A backup outbound tag, if no selector matched.
Strategy	Strategy	The selection algorithm (see below).

Strategy and observation parameters

The strategy (`strategy.type`) determines how the balancer chooses an outbound from the selectors. Xray-core values: `random` (random), `roundRobin` (round-robin), `leastPing` (lowest latency by observatory results), `leastLoad` (lowest load). For `leastLoad / leastPing` , the parameters from `strategy.settings` are used:

Field	Label	Description
expected	Expected	Placeholder: <i>"optimal number of nodes"</i> — the target number of live nodes.
maxRtt	Max RTT	The upper bound of acceptable RTT when selecting candidates.
tolerance	Tolerance	The tolerance when comparing latency/load.
baselines	Baselines	Latency threshold values for grouping nodes.
costs	Costs	Weight coefficients (cost) for individual tags.

Strategy examples. The `strategy` block lives inside a balancer (in JSON, next to `tag` and `selector`):

```
"strategy": { "type": "random" }      // random choice among the selectors
"strategy": { "type": "roundRobin" }  // round-robin, one after another
"strategy": { "type": "leastPing" }   // lowest latency (needs an observatory)
```

For `leastLoad`, the parameters go into `settings`:

```
"strategy": {
  "type": "leastLoad",
  "settings": {
    "expected": 2,
    "maxRTT": "1s",
    "tolerance": 0.05,
    "baselines": ["500ms", "1s", "2s"],
    "costs": [
      { "regex": false, "match": "proxy-premium", "value": 0.1 },
      { "regex": true, "match": "^proxy-cheap-.+$", "value": 5 }
    ]
  }
}
```

How it plays out (worked example). Suppose the observatory measured these latencies per egress: `A = 250 ms`, `B = 280 ms`, `C = 700 ms`, `D = 1500 ms`. With the settings above, selection goes as follows:

1. **maxRTT: "1s"** — egresses slower than 1 s are dropped: `D (1500 ms)` is out. `A`, `B`, `C` remain.
2. **baselines + expected** — egresses are grouped by latency thresholds, and the **smallest** threshold that contains at least `expected` egresses is chosen. The `500ms` threshold already contains `A` and `B` — that's 2 (= `expected`), so the group { `A`, `B` } is selected. `C (700 ms)` doesn't make the cut while there are enough fast ones (it's a "hot standby").
3. **tolerance: 0.05** — within the selected group, egresses whose latencies differ by no more than 5% are treated as equal and the load is split evenly between them. `A (250)` and `B (280)` differ by ~12% (> 5%), so all else being equal the faster `A` is preferred; if the difference were within 5%, traffic would go through both `A` and `B`.
4. **costs** — adjust the "cost" of individual egresses before comparison: a smaller `value` makes an egress more attractive, a larger one less so. In the example `proxy-premium` gets `0.1` (becomes "cheaper" and is chosen more eagerly), while all `proxy-cheap-*` (by regular expression, `regex: true`) get `5` (become "more expensive" and are used last). This lets you softly prioritize egresses without excluding them outright.

Result: traffic goes mostly through **A** (split evenly with **B** when their latencies are close), **C** stays as a standby, and **D** is excluded until its RTT drops below `maxRTT`.

Observatory: `observatory` and `burstObservatory` (measurements for `leastPing` / `leastLoad`)

The `leastPing` and `leastLoad` strategies don't measure anything themselves — they need latency and availability data for each outbound. This is collected by an **observatory**: it periodically "pings" each watched outbound and records its response time and availability. The same data is shown on the **Observatory** tab (statuses **Alive** / **Down**, **Last seen**, **Last try**).

There is no dedicated form for the observatory in the panel — you add the block **by hand** in the Xray configuration editor, at the top level of the config (next to `routing` and `outbounds`), and then **restart Xray**.

Two variants are available:

- **observatory** — simple: `subjectSelector` + `probeURL` + `probeInterval`.
- **burstObservatory** — extended, with fine-grained ping control via `pingConfig`; convenient for several egresses.

Example `burstObservatory` block:

```
{
  "subjectSelector": ["WS-SE", "WS-FR", "WS-PL"],
  "pingConfig": {
    "destination": "https://www.google.com/generate_204",
    "interval": "1m",
    "connectivity": "http://connectivitycheck.platform.hicloud.com/generate_204",
    "timeout": "5s",
    "sampling": 2
  }
}
```

Field meanings:

Field	What it sets
<code>subjectSelector</code>	A list of outbound tag prefixes to watch. Xray takes every outbound whose tag starts with one of these strings. In the example it watches the <code>WS-SE...</code> , <code>WS-FR...</code> , <code>WS-PL...</code> egresses. These tags must match what is chosen in the balancer's Selectors .
<code>pingConfig.destination</code>	The URL requested through each outbound to measure latency. Use a lightweight page that replies <code>204</code> with no body — e.g. <code>https://www.google.com/generate_204</code> . The time until the reply is the measured latency.
<code>pingConfig.interval</code>	How often to ping each outbound. A duration string: <code>"1m"</code> — once a minute, also <code>"30s"</code> , <code>"5m"</code> , etc. More often = fresher data but more background traffic.
<code>pingConfig.connectivity</code>	(optional) A URL to check the server's own baseline connectivity . If it is unreachable, the problem is the server's network, and the observatory does not mark outbounds as down (protection against false positives during a local outage). Usually also a <code>204</code> endpoint.
<code>pingConfig.timeout</code>	How long to wait for a single ping before treating the attempt as failed (e.g. <code>"5s"</code>).

Field	What it sets
<code>pingConfig.sampling</code>	How many recent measurements to keep and average per outbound. <code>2</code> — use the last two pings (smooths out random spikes).

How to wire it together:

1. In the Xray editor, add a `burstObservatory` block with the desired `subjectSelector`.
2. Create a balancer: **Strategy** = `leastPing`, and in **Selectors** list the same outbound tags (`WS-SE`, `WS-FR`, `WS-PL`).
3. Route traffic to it with a routing rule (the **Balancer tag** field, see [11.3](#)).
4. Restart Xray. The **Observatory** tab will show the egress statuses, and the balancer will start picking the fastest live one.

In a single rule you cannot set both `balancerTag` and `outboundTag` — only `outboundTag` will take effect.

11.6. DNS

The `dns` section. Enabling: **Enable DNS** (hint: *"Enable the built-in DNS server"*).

General DNS parameters

Field	Label (RU)	JSON	Description / hint
<code>tag</code>	DNS tag name	<code>dns.tag</code>	"This tag will be available as an inbound tag in routing rules." Allows routing the DNS requests themselves via <code>inboundTag</code> .
<code>clientIp</code>	Client IP	<code>dns.clientIp</code>	"Used to notify the server of the specified IP location during DNS requests" (EDNS Client Subnet).
<code>strategy</code>	Query strategy	<code>dns.queryStrategy</code>	"The overall domain-name resolution strategy". Values: <code>UseIP</code> , <code>UseIPv4</code> , <code>UseIPv6</code> .
<code>disableCache</code>	Disable cache	<code>dns.disableCache</code>	"Disables DNS caching".
<code>disableFallback</code>	Disable fallback DNS	<code>dns.disableFallback</code>	"Disables fallback DNS requests".
<code>disableFallbackIfMatch</code>	Disable fallback DNS on match	<code>dns.disableFallbackIfMatch</code>	"Disables fallback DNS requests when the DNS server's domain list matches".

Field	Label (RU)	JSON	Description / hint
<code>enableParallelQuery</code>	Enable parallel queries	—	"Enable parallel DNS requests to multiple servers for faster resolution".
<code>useSystemHosts</code>	Use system Hosts	<code>dns.useSystemHosts</code>	"Use the hosts file from the installed system".

Example dns block. Requests for Google domains are resolved via Cloudflare's DoH server, everything else via 1.1.1.1 ; for Google requests only non-private IPs are expected. At the top level of the config:

```
"dns": {
  "tag": "dns-inbound",
  "queryStrategy": "UseIPv4",
  "servers": [
    {
      "address": "https://cloudflare-dns.com/dns-query",
      "domains": ["geosite:google"],
      "expectIPs": ["geoip:!private"]
    },
    "1.1.1.1"
  ]
}
```

The bare server string ("1.1.1.1") with no fields is the default server for all other domains. The `dns-inbound` tag can then be used as an `inboundTag` in routing rules to send the DNS requests themselves through a specific outbound.

Stale-entry cache

Field	Label	Description
<code>serveStale</code>	Serve stale	"Return stale results from the cache while refreshing in the background".
<code>serveExpiredTTL</code>	Stale TTL	"The validity period (in seconds) of stale cache entries; 0 = unlimited".

DNS servers (the `dns.servers` list)

Buttons: **Add DNS** ("Add Server"), **Edit DNS**, **Clear all** (confirmation: "All DNS servers will be removed from the list. This action cannot be undone."). Presets: **Use preset**, the **DNS presets** dialog, including the **Family** preset.

Pressing **Edit DNS** on a DNS-server entry (and likewise on a Fake DNS entry) opens the edit dialog with the server's saved values rather than the defaults.

DNS server fields:

Field	Label	Description
<code>address</code>	—	The DNS address (IP, DoH URL, <code>localhost</code> , <code>fakedns</code> ,etc.).
<code>domains</code>	Domains	The list of domains for which this server is used.

Field	Label	Description
<code>expectIPs</code>	Expected IPs	Accept a response only if the IP falls within the list.
<code>unexpectedIPs</code>	Unexpected IPs	Discard responses with the specified IPs.
<code>skipFallback</code>	Skip fallback	Do not use this server as a fallback.
<code>finalQuery</code>	Final query	Marks the server as final in the chain.
<code>timeoutMs</code>	Timeout (ms)	The request timeout to the server.

Hosts (static entries)

The **Hosts** group (`dns.hosts`). Button **Add Host**; empty state **No hosts defined**. Fields: domain (placeholder: "Domain (e.g. domain:example.com)") and values (placeholder: "IP or domain – type and press Enter").

DNS logs

See 11.9: the **DNS logs** flag (`dnsLog`) in the logging section.

11.7. Fake DNS

The `fakedns` section. Buttons: **Add Fake DNS**, **Edit Fake DNS**.

Field	Label	Description
<code>ipPool</code>	IP pool subnet	The CIDR range from which fake IPs are issued (for example, <code>198.18.0.0/15</code>).
<code>poolSize</code>	Pool size	How many addresses to keep in the ring pool.

Fake DNS is used together with sniffing on the inbound: the core issues a fake IP to the client, remembers the domain↔IP mapping, and restores the domain during routing. For Fake DNS to work, a DNS server with the address `fakedns` must be added to the DNS server list.

Example: Fake DNS + DNS server pairing. First define the fake-address pool, then add a `fakedns` DNS server so that domain requests get an IP from that pool:

```
"fakedns": [
  { "ipPool": "198.18.0.0/15", "poolSize": 65535 }
],
"dns": {
  "servers": [
    { "address": "fakedns", "domains": ["geosite:geolocation-!cn"] },
    "1.1.1.1"
  ]
}
```

Additionally, the inbound must have sniffing enabled with `destOverride: ["fakedns"]`, otherwise the core has no way to recover the real domain.

11.8. WireGuard / WARP / NordVPN

WireGuard fields (`wireguard`)

Field	Label	Description
<code>secretKey</code>	Secret key	The private key of the local interface.
<code>publicKey</code>	Public key	The peer's public key.
<code>psk</code>	Shared key	The PreShared Key (optional).
<code>allowedIPs</code>	Allowed IP addresses	The ranges routed into the tunnel.
<code>endpoint</code>	Endpoint	The peer's <code>host:port</code> .
<code>domainStrategy</code>	Domain strategy	The resolution strategy for the WireGuard outbound.

Cloudflare WARP (`warp`)

The integration uses the API `https://api.cloudflareclient.com/v0a4005` (client-version `a-6.30-3596`).
Controller actions (`/xray/warp/:action`): `config` , `reg` , `license` , `data` , `del` .

Step by step:

1. **Create WARP account** → `reg` : the panel generates/accepts a private (`privateKey`) and public (`publicKey`) key, registers the device with Cloudflare, and saves `access_token` , `device_id` , `license_key` , `private_key` (as well as `client_id`) in the `warp` setting.
2. **WARP / WARP+ license key** → `license` : setting the 26-character WARP+ key (placeholder: "26-character WARP+ key"). On error: "Failed to set the WARP license." If the config has not been fetched yet: "Fetch the WARP config first."
3. **Account info: Device name, Device model, Device enabled, Account type, Role, WARP+ data, Quota, Usage.**
4. **Add outbound** – creates a WireGuard outbound with the obtained keys and the Cloudflare endpoint.
5. **Delete account** → `del` : clears the saved WARP data.

NordVPN (`nord` / `nordvpn`)

The integration uses NordLynx (= WireGuard). Controller actions (`/xray/nord/:action`): `countries` , `servers` , `reg` , `setKey` , `data` , `del` .

Step by step:

1. **Access token** → `reg` : the panel requests NordLynx credentials from `api.nordvpn.com` and extracts `nordlynx_private_key` . It saves `private_key` and `token` in the `nord` setting. The alternative is `setKey` : enter the **Private key** directly (it cannot be empty).
2. **Country** → `countries` loads the list of countries; **City** (or **All cities**).
3. **Server** → `servers` loads the servers of the selected country (`countryId` is validated as a number – injection protection). Filter: only servers with a **Load** > 7% are shown. If there are no servers: "No servers found for the selected country". If the server does not report a NordLynx public key: "The selected server does not report a NordLynx public key."

4. Creating/updating the outbound: toasts *"NordVPN outbound added"* / *"NordVPN outbound updated"*.

IPv4 priority and userspace TUN

The WireGuard outbounds generated by the WARP and NordVPN wizards use `domainStrategy: "ForceIPv4v6"` (IPv4 priority with IPv6 fallback on v6-only hosts) instead of `ForceIP` — this fixes the handshake "hanging" on hosts with half-configured IPv6, where the AAAA record of the Cloudflare endpoint would be picked. They also enable userspace TUN (`noKernelTun: true`) instead of kernel TUN: the latter needs privileges and fwmark routing and fails silently on many VPS setups, whereas the panel's built-in connectivity probe always tests through userspace TUN — so real traffic and the probe now follow the same path. This affects only newly added or reset outbounds; already-saved templates keep their settings.

11.9. Reverse proxy and TUN

Reverse (reverse proxy)

The `reverse` section of the Xray configuration. In the outbound form there is a switch to the **Reverse proxy** type. Buttons: **Add reverse proxy**, **Edit reverse proxy**.

Field	Label	Description
Type	Type	Bridge or Portal — the two roles of the Xray reverse proxy.
Domain	Domain	A service label domain for the bridge↔portal pair.
Tag / Connection	Tag / Connection	Tags for pairing the bridge and the portal.
Reverse Tag	Reverse proxy tag	Hint: <i>"The outbound tag for a simple VLESS reverse proxy. Leave empty to disable."</i> Placeholder: <i>"outbound tag (empty = disabled)"</i> . Implements a simplified VLESS reverse.

The outbound form also contains reverse-flow fields: **Reverse sniffing**, **Workers**, **Reserved**, **Min upload interval (ms)**, **Max upload size (bytes)**.

TUN (tun)

Field	Label	Description	Default
name	—	<i>"The name of the TUN interface."</i>	xray0
mtu	—	<i>"Maximum transmission unit. The maximum size of data packets."</i>	1500
<code>userLevel</code>	User level	<i>"All connections established through this inbound will use this user level."</i>	0

11.10. Logs and statistics (Stats, metrics)

Log (log)

Hint: "Logs can slow down the server. Enable only the log types you need, and only when necessary!" The `log` section of the reference template: `access: "none"`, `error: ""`, `loglevel: "warning"`, `dnsLog: false`, `maskAddress: ""`.

Field	Label	JSON	Description	Default
<code>logLevel</code>	Log level	<code>loglevel</code>	"The log level for error logs..." Values: <code>debug</code> , <code>info</code> , <code>warning</code> , <code>error</code> , <code>none</code> .	warning
<code>accessLog</code>	Access logs	<code>access</code>	"The path to the access log file. The special value 'none' disables access logs."	none
<code>errorLog</code>	Error logs	<code>error</code>	"The path to the error log file. The special value 'none' disables error logs."	"" (default)
<code>dnsLog</code>	DNS logs	<code>dnsLog</code>	"Enable DNS query logs"	false
<code>maskAddress</code>	Address masking	<code>maskAddress</code>	"When enabled, the real IP address is replaced with a masked one in the logs."	"" (off)

Statistics (stats / policy)

The **Statistics** group. It enables the counters in `policy.system` and `policy.levels`. In the reference template: `statsInboundUplink: true`, `statsInboundDownlink: true`, `statsOutboundUplink: false`, `statsOutboundDownlink: false`; for level `0` — `statsUserUplink: true`, `statsUserDownlink: true`.

Field	Label	Description	Default
<code>statsInboundUplink</code>	Inbound uplink statistics	"Enables statistics collection for the outgoing traffic of all inbound proxies."	true
<code>statsInboundDownlink</code>	Inbound downlink statistics	"Enables statistics collection for the incoming traffic of all inbound proxies."	true
<code>statsOutboundUplink</code>	Outbound uplink statistics	"Enables statistics collection for the outgoing traffic of all outbound proxies."	false
<code>statsOutboundDownlink</code>	Outbound downlink statistics	"Enables statistics collection for the incoming traffic of all outbound proxies."	false

Per-client and per-inbound statistics (uplink/downlink) are the basis for displaying traffic in the dashboard and to clients; disabling them is not recommended. Outbound statistics are off by default and are needed only if you track traffic by outbound tags.

Metrics

The reference template contains a `metrics` section (`listen: "127.0.0.1:11111"` , `tag: "metrics_out"`) and the corresponding `metrics_out` API. The panel uses this listener to collect metrics and observatory snapshots: it parses `metrics.listen` from the template, polls `/debug/vars` , and aggregates the latency history by tag. If you change the address/port of `metrics.listen` , the panel will query the new address; removing the `metrics` section disables observatory graph collection.

Testing an outbound in HTTP mode spins up a **separate temporary** Xray instance with its own `metrics` listener on a random port – this is not the same listener as in the main config.

11.11. Saving, restart, and automatic transformations

Buttons

Button	Action
Save	POST <code>/xray/update</code> : validates and saves the template + <code>outboundTestUrl</code> .
Restart Xray	Reloads the service with the saved configuration. Confirmation: <i>"Restart xray?"</i> / <i>"Reloads the xray service with the saved configuration."</i>

Toasts: success – *"Xray restarted successfully"*, *"Xray stopped successfully"*; errors – *"An error occurred while restarting Xray."*, *"An error occurred while stopping Xray."* The **Xray restart output** dialog shows the core's diagnostic output.

Hot-applying changes (without a full restart)

Changes to inbounds, outbounds, and routing rules are applied live: when you press **Save**, the panel computes the diff between the old and new config and applies only the changed parts through the Xray gRPC API (HandlerService/RoutingService), without restarting the process. A full restart is performed automatically only when sections with no hot-reload API (`log` , `dns` , `policy` , `observatory` , etc.) change. Because of this, the Xray page no longer has a separate "Restart" button – **Save** applies the changes itself. A core restart, when needed, still happens automatically (see also the auto-reload on subscription updates and WARP rotation).

Restoring the default template

The endpoint `GET /xray/getDefaultJsonConfig` returns the reference template (`config.json` , embedded in the binary). It can be used to reset the configuration to factory defaults.

Automatic transformations on save

When saving the Xray settings, the panel performs (in this order):

1. **Unwrapping** — strips wrappers of the form `{ "xraySetting": <config>, "inboundTags": ..., "outboundTestUrl": ... }` if they accidentally ended up in the value (otherwise the layers would accumulate on every save). Up to 8 layers are stripped.
2. **Config check** — the JSON is parsed into the Xray configuration structure; on error — rejected with *"xray template config invalid"*.
3. **Stats-rule enforcement** — the rule `inboundTag: ["api"] → outboundTag: "api"` is forcibly moved to position 0 in `routing.rules` (or added if absent). This guarantees that the panel's gRPC statistics request is not intercepted by a higher-priority catch-all rule (otherwise clients may appear offline with zero traffic while the proxy is working).

Because of item 3, do not try to remove or move the `api → api` rule — the panel will return it to its place on the next save anyway. This is service infrastructure for statistics, not a user route.

11.12. Outbounds from a subscription (with auto-update)

Starting with version 3.3.0, the panel can import outbounds directly from a subscription URL — the same format that VPN providers serve to client applications. Subscriptions are re-read regularly in the background, so the set of outbounds on the server is kept up to date without manually editing the configuration template.

In the Russian interface the section is called **"Outbound subscriptions"**, with the description: "Import outbounds from remote subscription URLs (vmess/vless/trojan/ss/...). Tags remain unchanged for use in balancers and routing rules. Updates are performed automatically." The section is located on the Xray page, above the outbound configuration panel.

How it works

Subscriptions are stored separately from the Xray configuration template. The template is **never overwritten**: outbounds obtained from subscriptions are added to the final configuration on the fly each time the Xray config is generated.

Adding a subscription

The "Add subscription" form offers the following fields:

Field	Key	Default	Purpose
Subscription URL	<code>url</code>	— (required)	The subscription address. Placeholder: "https://... (base64 list of links)". Only HTTP(S) is accepted; the address is checked for safety.
Remark	<code>remark</code>	empty	An arbitrary label (placeholder "e.g. HK nodes").

Field	Key	Default	Purpose
Tag prefix	<code>tagPrefix</code>	<code>subN-</code>	The prefix that the tags of imported outbounds start with. If left empty, the panel picks the smallest free number of the form <code>sub1-</code> , <code>sub2-</code> , etc., on its own.
Update interval	<code>updateInterval</code>	600 seconds (10 minutes)	How often the subscription is re-read. In the UI it is set in hours/minutes.
Enabled	<code>enabled</code>	yes (<code>true</code>)	Only enabled subscriptions are included in the config and updated automatically.
Allow private addresses	<code>allowPrivate</code>	no (<code>false</code>)	Allows URLs to localhost, the LAN, and private IPs. Disabled by default for SSRF protection – enable it only for a trusted local source.
Before manual outbounds	<code>prepend</code>	no (<code>false</code>)	If enabled, this subscription's outbounds are placed before the template's manual outbounds, and one of them may become the default outbound. Otherwise they are added after .

The **"Preview"** button (`POST /outbound-subs/parse`) lets you download and parse the URL before saving and see which outbounds and tags will result; nothing is written to the database in the process. If nothing is recognized at the URL, "No outbounds found at this URL." is shown.

The order of multiple subscriptions within the overall outbound list is set by priority (`priority`) and changed with the up/down arrows (`POST /outbound-subs/:id/move`).

Which subscription formats are accepted

The body of the response from the URL is processed as follows:

- The content is first tried as **base64** (the standard and URL-safe variants, with padding auto-completion and removal of spaces/line breaks). If it is base64, it is decoded; otherwise it is taken as is.
- Then the body is split into lines. Each non-empty line that does not start with `#` is parsed as a link. Unrecognized lines (comments, unsupported protocols) are silently skipped.
- Supported link schemes: `vmess://`, `vless://`, `trojan://`, `ss://` (Shadowsocks), `hysteria2://` / `hy2://`, `wireguard://` / `wg://`.

In other words, an ordinary subscription of the form "a base64-encoded list of links", as used by most providers, will work.

Stable tags

Each link is assigned a stable "identity" (the URI core without the remark fragment; for vmess – the internal JSON without the `ps` field). The "identity → tag" mapping is preserved, and on the next update the same server receives the same tag, even if the remark or secondary parameters changed. This is done deliberately so that balancers and routing rules continue to work after updates:

- An exact tag in a balancer/rule will continue to point to the same server.
- A prefix/wildcard selector (for example, `hk-*`) automatically picks up new servers that the subscription returns later – this is the recommended way to "subscribe to a pool".

- If a server disappears from the subscription, its tag simply drops out of the final outbounds array; if the balancer has a `fallbackTag`, Xray uses it.
- If the provider changed the server's UUID/host/credentials, the identity changes — this is treated as a new outbound with a new tag.

Within a single fetch, tags are deduplicated with a `-N` suffix.

How auto-update works

- The background subscription-refresh job runs on a schedule **every 5 minutes**.
- On each run it iterates over all enabled subscriptions and updates only those whose own interval has elapsed: a subscription is updated if it has never been updated yet, or if at least its `updateInterval` has passed since the last update. Thus the job checks subscriptions often, but each individual subscription is re-read no more frequently than its `updateInterval` (10 minutes by default). In the UI this is reflected by the corresponding hint.
- The update: the URL is re-checked for safety as a public URL (private addresses are blocked unless the subscription has `allowPrivate` set), the request goes through the panel's proxy client with the header `User-Agent: 3x-ui-outbound-sub/1.0`. The redirect chain is limited to 10 hops, and each hop is also checked for privateness (SSRF protection). HTTP 200 is expected; otherwise an error is recorded.
- After a successful parse, the result is saved, the last-update time is set, and the error is cleared. On error its text is visible in the UI as "Last error", and the previously obtained outbounds remain in effect.
- If at least one subscription actually updated, the job marks Xray for restart and sends a UI invalidation so the interface pulls in the new outbounds. The actual Xray reload happens on the next 30-second manager cycle.

A manual update of a single subscription is the **"Refresh now"** button (`POST /outbound-subs/:id/refresh`); it also marks Xray for restart. Adding, editing, or deleting a subscription likewise sets the Xray restart flag (on deletion its outbounds drop out of the config on the next reload). The UI hints: "After adding or updating, restart Xray (or wait for the next auto-reload) for the outbounds to become active."

How it gets into the Xray config

Each time the Xray configuration is generated, the active subscription outbounds are split into two groups — `prepend` (the "Before manual outbounds" flag) and the rest — and stitched together with the template: `[prepend subscriptions] + [template outbounds] + [the rest of the subscriptions]`. Within each group, subscriptions are ordered by priority. The manual outbounds from the template are not touched; if the template's outbounds array fails to parse for some reason, the subscription outbounds are not mixed in (so as not to lose the manual ones).

The imported outbounds are additionally shown in the outbounds panel itself, in a separate block **"From outbound subscriptions (read-only)"** — they cannot be edited there; management is only via the "Outbound subscriptions" section.

11.13. IP rotation in WARP

In 3X-UI you can set up a WARP outbound — an outgoing WireGuard connection to Cloudflare WARP (the tag `warp` in the Xray config). The panel itself registers a device account on Cloudflare's servers, obtains the

WireGuard keys and addresses, and inserts them into the outbound with the tag `warp`. Through such an outbound, traffic exits to the internet under a Cloudflare WARP IP address. New in version 3.3.0 is the ability to change this outgoing IP manually or on a schedule, without manually recreating the WARP account.

The controls are in the **Xray** section, in the WARP card (after pressing "Create WARP account" and obtaining the config; before that the actions are unavailable – the panel will hint "Fetch the WARP config first").

What happens when the IP is changed

The **"Change IP"** button starts the IP change. The logic:

1. A new WireGuard key pair is generated.
2. With the new key, the WARP device is re-registered on Cloudflare's servers (a new `device_id`, `access_token`, addresses, and peer data).
3. The new data is written to the WARP outbound of the Xray config: `secretKey`, `address` (v4 /32 and v6 /128), `reserved` (from `client_id`), as well as the peer's `publicKey` and `endpoint` are updated.
4. If a WARP+ license key (at least 26 characters long) was previously set, it is automatically reinstalled on the new account. On failure this is only a warning in the logs – the IP change is not canceled.
5. After a successful change, Xray is marked as requiring a restart, so the new outbound takes effect.

On success the interface shows "WARP IP address changed successfully!".

Automatic rotation on a schedule

The WARP card has a **"Automatic IP address update"** toggle and an **"Interval (days)"** field. The hint: "0 – disable. Automatically changes the IP address."

Parameter	Value
DB setting	<code>warpUpdateInterval</code> (integer, ≥ 0)
Default value	0 (auto-rotation disabled)
Unit	days
0	disables the automatic change
> 0	change the IP every N days

Writing the interval saves `warpUpdateInterval`, and when the value is greater than 0 it resets the "last update time" to the current moment – otherwise the scheduler would change the IP on the very next tick.

The schedule is executed by a background job that runs once an hour – that is, the panel checks once an hour whether it is time to rotate. The check algorithm:

- if the interval ≤ 0 – it does nothing;
- if the "last update time" equals 0 (for example, the interval was set by editing the DB directly) – this is the first run: the job only records a baseline timestamp and does NOT change the IP immediately;
- if at least `interval × 24 × 3600` seconds have passed since the last update – the same IP change is performed, the timestamp is updated, and an Xray restart is scheduled.

An important detail: a manual change via the "Change IP" button also resets the last-update timestamp. Therefore, after a manual rotation, the automatic interval count starts over and a scheduled change will not fire immediately afterward.

"Through the panel proxy"

Changed in 3.3.1. The separate "Panel network proxy" (`panelProxy`) setting has been removed. The panel's own outgoing traffic (including WARP API requests) now goes through the selected **panel traffic outbound** — an Xray outbound or balancer (see ["What's new in 3.3.1"](#) and section 13). The description below applies to versions before 3.3.1.

All requests to the Cloudflare WARP API (registration, fetching the config, setting the license, changing the IP) go not directly but through the panel's HTTP client with a 15-second timeout. This client respects the **"Panel network proxy"** setting (`panelProxy`) from the panel settings.

From the setting's description: the proxy routes the panel's own outgoing requests (geo-database updates, Xray/panel version checks, Telegram, and now WARP calls too) — to bypass server-side filtering. Addresses of the form `socks5://` or `http(s)://` are accepted, for example a local SOCKS inbound of Xray itself. If the field is empty or the proxy is set incorrectly, a direct connection is used (behavior does not break).

The benefit for WARP: if the server cannot directly reach `api.cloudflareclient.com`, registration and rotation used to fail. Now, by specifying a working proxy in `panelProxy` (including Xray's own inbound), you can ensure the WARP API is reachable and that both the manual button and the scheduled rotation work.

When this is useful

- Regularly changing the outgoing IP for an outbound that goes through WARP — reduces the risk of blocking and tracking by a single address.
- "Refreshing" the IP manually if the current Cloudflare address has landed on blacklists or is slow.
- Servers that have no direct access to the Cloudflare WARP API: routing the requests through `panelProxy` makes registration and rotation work.

12. Nodes (multi-panel, master/slave)

The **Nodes** section turns an ordinary 3X-UI installation into a **central (master) panel** that remotely monitors and manages other (child) 3X-UI panels. Each node is a separate 3X-UI installation on its own server; the master reaches it through its own HTTP API, polls its state, and synchronizes the inbounds and clients assigned to it. This is precisely the **multi-panel** capability: instead of logging into each panel separately, you see all servers in a single list and manage them centrally.

An important principle: **a node is not an agent — it is a full-fledged 3X-UI panel**. The master "installs" nothing on it — it merely connects to its API by token. Removing a node from the list stops only the monitoring; the remote panel itself is not affected (tooltip: "This will stop monitoring the node. The remote panel itself will not be affected").

12.1. Summary at the top of the list

Aggregate counters are shown above the node table:

Field	Description
Total nodes	Total number of nodes in the list.
Online	How many nodes have status <code>online</code> .
Offline	How many nodes have status <code>offline</code> .
Average latency	Averaged latency (ping) to the nodes, in milliseconds.

12.2. Adding and editing a node

The **Add node** and **Edit node** buttons open a form with the node's fields.

The **Name**, **Address**, **Port**, and **API Token** fields are required (tooltip: "Name, address, port, and API token are required").

When you click "Save" (both when adding and when editing), the panel **first checks the node's reachability** with a 6-second timeout. If the node does not respond, the record is not saved and an error is shown. In other words, you cannot add a node that is knowingly unreachable.

Form fields

Field (EN)	Default	Allowed values	Description
Name	— (required)	non-empty string, unique	The node's internal name. The name column is unique — two nodes with the same name cannot be created. Placeholder hint: <code>napr. de-frankfurt-1</code> . On save, leading/trailing spaces are trimmed.
Remark	empty	any string	An optional note/description for the node. Has no effect on operation.

Field (EN)	Default	Allowed values	Description
Scheme	<code>https</code>	<code>http</code> / <code>https</code>	The protocol used to connect to the remote panel. If left empty or set to an invalid value, normalization sets <code>https</code> . If the node responds over plain HTTP while the scheme is <code>https</code> , the panel returns a clear hint: "the server speaks HTTP, not HTTPS; set the node scheme to http".
Address	— (required)	host or IP	The address of the remote panel. Placeholder: <code>panel.example.com</code> or <code>1.2.3.4</code> . The address is normalized; by default private/local addresses are forbidden to protect against SSRF — see "Allow private address".
Port	— (required)	integer 1–65535	The web panel port of the remote node. Values out of range are rejected ("node port must be 1-65535").
Base path	<code>/</code>	path string	The base path (web base path) of the remote panel, if one is set. It is normalized: it is guaranteed to begin and end with <code>/</code> (an empty value → <code>/</code>). When polling, the panel appends <code>panel/api/server/status</code> to it.
API Token	— (required)	the remote panel's token	The bearer token for accessing the node's API. It is passed in the <code>Authorization: Bearer <token></code> header. Placeholder: "Token from the remote panel's Settings page". Hint: "The remote panel shows its API token under Settings → API Token". That is, the token must be created on the node itself (Settings → API Token) and pasted here.
Enabled	<code>true</code>	yes/no	Enables monitoring and synchronization of the node. Disabled nodes are not polled by background jobs (heartbeat and traffic-sync skip them) and do not take part in bulk panel updates.
Allow private address	<code>false</code>	yes/no	Removes the SSRF protection and allows connecting to the node at a private/local address. Hint: "Enable only for nodes on a private network or VPN". Enable this only when the node is genuinely on a private network or reachable through a VPN.

Obtaining and regenerating the token on the node side

The token is obtained on the remote panel under **Settings** → **API Token**. It can also be reissued there: the **Regenerate token** button with a warning: "Regenerating invalidates the current token. Any central panel using it will lose access until updated. Continue?". After regeneration, the old token stops working in the master panel — it must be updated in the node form.

Inbound import (choosing which inbounds to synchronize)

The node form has an **Inbound import** setting (`inboundSyncMode`) with two modes: **All inbounds** (`all` , the default) and **Selected** (`selected`). By default the master synchronizes to the node every inbound that has this node selected; existing nodes keep working in "All inbounds" mode.

In **Selected** mode a multi-select of inbound tags appears below the field. Click **Load inbounds** — using the entered (not-yet-saved) connection parameters, the master asks the node for the list of its inbounds (endpoint `POST /panel/api/nodes/inbounds`) and shows their tags; check the ones you want. The panel then synchronizes and deploys only the checked tags to the node, while the other inbounds that exist directly on the node are left untouched — the master neither deletes nor manages them.

Example: list a node's inbounds for selective import. The body carries the not-yet-saved connection parameters; the response holds the tags of the inbounds available on the node:

```
POST /panel/api/nodes/inbounds
Content-Type: application/json

{ "name": "de-fra-1", "scheme": "https", "address": "node1.example.com",
  "port": 2053, "basePath": "/", "apiToken": "abcdef..." }
```

12.3. TLS verification (for https nodes)

This group of fields specifies how the master verifies the node's HTTPS certificate. These settings **apply only to the https scheme**; for http nodes they are ignored.

TLS verification is a dropdown with the hint: "How the panel verifies the node's HTTPS certificate. Pin or Skip — for self-signed certificates (https nodes only)".

Mode (EN)	Value	Default	Description
Verify (standard CA)	verify	yes (default)	Normal verification of the certificate chain against a trusted CA. Suitable for nodes with a public / Let's Encrypt certificate. Also used for all http nodes.
Pin certificate (SHA-256)	pin	—	The CA chain is not verified, but the SHA-256 of the node's leaf certificate is compared against the stored fingerprint (constant-time comparison). Retains MITM protection for self-signed certificates. Requires the fingerprint field to be filled in.
Skip verification	skip	—	Certificate verification is disabled entirely. Warning: "Skipping verification removes protection against man-in-the-middle attacks — the API token can be intercepted. Pinning the certificate is preferable".

If any value other than skip or pin is selected, normalization forces verify.

Certificate pinning

When **Pin certificate** is selected, the following appear:

- **SHA-256 of the pinned certificate** — an input field. It accepts a fingerprint in **base64** (the pinnedPeerCertSha256 format from Xray) or in **hex** with or without colons (the openssl -fingerprint style). Hint: "The node's certificate SHA-256 in base64 or hex. Click 'Fetch' to read it from the node now". Placeholder: "SHA-256 in base64 or hex". When pin is selected, an empty or invalid fingerprint causes a validation error on save.
- The **Fetch** button — connects to the node over HTTPS without verifying the certificate and reads the SHA-256 of the current leaf certificate (endpoint POST /certFingerprint), filling it into the field. On success — "The node's current certificate has been fetched"; on failure — "Failed to fetch the certificate". Available only for https nodes.

Example: the same fingerprint in two formats. The field accepts either form — both denote one certificate:

```
# base64 (the pinnedPeerCertSha256 format from Xray)
607TNg3l2k0pq8R1sT2uV3wX4yZ5a6B7c8D9e0F1g2=

# hex with colons (openssl x509 -fingerprint -sha256 style)
E8:E2:D3:60:DE:5D:9A:4D:29:AB:CF:11:B2:7C:34:...
```

If the fingerprint is not yet known, click **Fetch** — the master reads it from the node over HTTPS and fills the field in for you.

12.4. What is shown for each node

The table columns and the fields on the node card (the observed state, populated on every heartbeat poll):

Field (EN)	Description
Status	<code>online</code> / <code>offline</code> / <code>unknown</code> — see below.
CPU	CPU load of the remote server, as a percentage.
Memory	RAM usage as a percentage (computed as <code>current/total*100</code>).
Uptime	The server's continuous uptime (in seconds).
Latency	The node's response time to the last poll (ms).
Last ping	The time of the last successful heartbeat (unix seconds; <code>0</code> = "never"; a recent value is shown as "just now").
Xray version	The version of Xray-core running on the node.
Panel version	The 3X-UI version on the node — compared against the latest for the update indicator.
(inbounds)	How many inbounds are physically hosted on this node.
(clients)	The number of clients on the node's inbounds.
(online)	How many of the node's clients are currently online.
(depleted)	How many of the node's clients are disabled/expired/have exhausted their traffic limit.

The inbounds/clients/online counters are attributed to a node by its stable GUID (`panelGuid`) rather than by local id — so that a client on a sub-node is counted under the sub-node and not under the intermediate node through which it syncs.

Node statuses

Status	EN	When it is set
<code>online</code>	Online	The node responded with <code>success=true</code> to the <code>panel/api/server/status</code> poll.
<code>offline</code>	Offline	The node did not respond, returned an HTTP error, <code>success=false</code> , or an unrecognizable response.
<code>unknown</code>	Unknown	The initial value, until the node has been polled at least once.

On an unsuccessful poll, the error text is saved and shown in clear wording, which helps diagnose the cause of "offline".

12.5. Actions on a node

- **Test connection** (`POST /test`) — in the node form, tests connectivity using the entered (not-yet-saved) parameters with a 6 s timeout. Result: "Connection is OK ({ms} ms)" or "Failed to connect". Handy for debugging the address/port/token/TLS before saving.
- **Probe now** (the "Probe now" button, `POST /probe/:id`) — an unscheduled poll of an already saved node; it immediately updates the status and metrics (CPU/memory/uptime/latency/versions) and records a heartbeat. On failure — "Probe failed".

Example: test and probe a node through the master's API. "Test connection" tries the not-yet-saved parameters from the form:

```
POST /panel/api/nodes/test
Content-Type: application/json

{ "scheme": "https", "address": "de-frankfurt-1.example.com", "port": 2053,
  "basePath": "/", "apiToken": "eyJhbGciOi...", "tlsMode": "verify" }
```

An unscheduled poll of an already saved node with id 7:

```
POST /panel/api/nodes/probe/7
```

- **Update panel** (`POST /updatePanel` with the body `{ids: [...]}`) — triggers the node's built-in self-updater: the node downloads the latest 3X-UI release and restarts on it. The **Update selected ({count})** button does this for several checked nodes at once. An indicator is shown next to a node: **Update available** or **Up to date**, based on comparing the node's panel version against the latest.

Example: update several nodes in one request. The body carries the ids of the checked nodes; only enabled, `online` nodes are updated, the rest come back as skipped.

```
POST /panel/api/nodes/updatePanel
Content-Type: application/json

{ "ids": [3, 7, 12] }
```

A response like "Update started on 2 nodes, 1 failed": node 12, for instance, may have been offline and was therefore skipped.

- Confirmation: "Update {count} nodes to the latest version? Each selected node will download the latest release and restart. Only enabled, online nodes are updated".
- **Only enabled nodes with status `online` are updated.** A disabled node is marked in the results as "node is disabled", an offline one as "node is offline". Result: "Update started on {ok} nodes, {failed} failed". If no eligible node is selected — "Select at least one enabled, online node".

- **Set Cert from Panel** (an auxiliary action, `GET /webCert/:id`) — when creating an inbound on the node, it lets you fill in the paths to the node's **own** web-TLS certificate (rather than the central panel's), so that the files actually exist on the node. Requires the node to be enabled and reachable.
- **Delete node** (`POST /del/:id`) — confirmation: "Delete node "{name}"? This will stop monitoring the node. The remote panel itself will not be affected". Deletes the node record and its accumulated traffic statistics; the remote panel keeps running as usual.

12.6. Metric history

The history button/chart calls `GET /history/:id/:metric/:bucket`. The available metrics are **cpu** and **mem** — they accumulate on every successful heartbeat. The aggregation interval size (`bucket` , in seconds) is restricted to an allowlist:

Example: a history request. The CPU-load chart for node 7 aggregated over 60-second intervals (up to 60 points are returned):

```
GET /panel/api/nodes/history/7/cpu/60
```

For memory and "real-time" mode (2 s) use `.../7/mem/60` and `.../7/cpu/2` respectively. Values outside the allowlist are rejected ("invalid metric" / "invalid bucket").

Bucket (s)	Purpose
2	Real-time mode
30	30 s intervals
60	1 min intervals
120	2 min intervals
180	3 min intervals
300	5 min intervals

Up to 60 points are returned. An invalid metric or bucket is rejected ("invalid metric" / "invalid bucket").

12.7. How inbounds and clients are synchronized

An inbound "belongs" to a node through the `node_id` field (the node is selected in the inbound editor):

Example: the token in the node form. The token is taken on the child panel (Settings → API Token) and pasted into the master's **API Token** field. On every poll the master sends it in the header:

```
GET https://panel.example.com:2053/panel/api/server/status
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.abc123...
```

If the child panel has a **base path** (web base path) set, e.g. `/secret/`, the master prepends it before `panel/api/server/status` → `https://panel.example.com:2053/secret/panel/api/server/status`.

1. **Configuration deployment (reconcile).** On any change to an inbound/client bound to a node, the node is marked "dirty". For each enabled node **with status online** that has pending changes, a background job deploys the node's inbounds (by `node_id`) to the node and then clears the dirty flag. A node that is disabled, offline, or "dirty" is considered "pending" — its deployment is deferred until connectivity is restored.
2. **Traffic collection.** The same job requests a traffic snapshot from the node and merges it into the local statistics. Based on the merged traffic, a check for limit/term exhaustion is performed and clients are disabled if necessary; the node's "depleted" counter reflects exactly this. If the node is unreachable, its online clients are cleared.

For a client attached to several panels at once, the master also pushes each node the client's **aggregated, cross-panel** traffic in the same job (stored in a separate table on the node, keyed by the master's GUID; overwritten on every push, so a master-side reset propagates too). On the node, the client's traffic is shown as the larger of the two values — local or pushed — and when the overall quota is exceeded the client is disabled **locally on the node itself** (via the same restart-Xray-on-disable flow, which drops already-established connections). This eliminates the case where a node saw only its own share of the traffic, under-counted usage, and kept serving a client who had already exhausted the overall limit. On a traffic reset, auto-renew, or client delete, the pushed counters are cleared.

3. **Heartbeat.** A separate background job periodically polls all **enabled** nodes (with a concurrency limit) via `panel/api/server/status`, updates the status/metrics/versions, and, if there are web clients connected, broadcasts the updated node tree over WebSocket.

12.8. Node chains (sub-nodes / transitive nodes)

The topology need not be flat: a node can itself be a master for its own nodes. Such downstream panels appear in your list as **Sub-nodes** — these are **read-only projections** obtained from the direct node.

- Hint: "Read-only: a subordinate node reachable through {parent}. Manage it from {parent}'s own panel". That is, a sub-node cannot be edited, deleted, or updated here — all operations on it are performed from the panel of its direct parent.
- A sub-node's identity is determined by its GUID; thanks to this, online clients and inbounds are counted under the physical node that actually hosts them, even in a chain `Node1 → Node2 → Node3` (the master "reaches" one level deeper through each direct node).
- If a direct node becomes unreachable, its sub-node cache is cleared, and the sub-nodes disappear from the tree until connectivity is restored.

12.9. Nodes: new in 3.3.0

In version 3.3.0 the **Nodes** section received three notable improvements: correct attribution of traffic and online clients in multi-hop topologies, synchronization of client IPs between nodes, and a separate status indicator for the case where a node's panel is alive but its Xray core has crashed. The words inbound/outbound are no longer translated below.

1. Multi-hop: correct traffic attribution along the sub-node chain

Previously the counters (number of inbounds, online clients, depleted clients) were computed at the level of the "direct" node. If you had a chain like `Master → Node1 → Node2 → Node3`, everything physically living on `Node2 / Node3` was mistakenly attributed to `Node1`, through which it reached the master. In 3.3.0, attribution follows the real source.

How this is arranged:

- **Sub-nodes become visible as separate rows.** Each panel publishes the list of its direct nodes; only nodes with a known `Guid` are included – a stable identity is needed in order to attribute a node one "hop" upward. The master periodically (from the heartbeat job) pulls these lists and caches them, then adds the "transitive" sub-nodes to the direct nodes.
- **Transitive nodes are read-only.** In the UI they are marked as a **"Sub-node"** with the hint: *"Read-only: a subordinate node reachable through {parent}. Manage it from {parent}'s own panel."* Such a row has no management buttons – the node is managed from the panel of its immediate parent.
- **Hierarchy via GUID.** A direct node's `ParentGuid` is the GUID of the master itself; a transitive node's is the GUID of its parent node. This is how the tree is built.
- **The source of truth for the counters is `origin_node_guid` on the inbound.** This is the `panelGuid` of the node that physically holds this inbound. It is set when the inbound is synced from the node and **preserved as-is across subsequent hops**, so a deeply nested inbound is attributed to the real node rather than to an intermediate one. The inbound-count, online, and depleted counters are recomputed by this GUID. The key-selection logic:

Inbound state	What it is attributed to
<code>origin_node_guid</code> is set	this GUID (the real source node)
empty, but <code>node_id</code> is set	the node's synthetic GUID (an old build that has not yet reported its <code>panelGuid</code>)
empty and <code>node_id</code> is empty	the master's own GUID (an inbound on the local Xray)

Online clients are likewise grouped by GUID, so each node row shows only those who are actually connected to it.

What the user sees: in a flat topology (nodes directly under the master) nothing changes – the counters by GUID and by `id` coincide. But as soon as a node chain appears, "Sub-node" rows show up in the list, and each node's inbound/online/depleted numbers now reflect its own load rather than the sum of everything that passed through it in transit.

2. Synchronization of client IPs from `access.log` between nodes

The IP limit (`limitIp` on a client) relies on the addresses that Xray writes to its `access.log`. Previously each node saw only the connections to itself, so the "no more than N IPs per client" restriction did not work in a cluster: a client could connect to different nodes and bypass the limit. In 3.3.0, the observed IPs are synchronized across the whole cluster.

How this works:

- On each node, a background job parses the `access.log`, extracting per line the IP, the client's email, and the timestamp, and stores them in a local table (one record per email, with IPs stored as a JSON array `{ip, timestamp}`). The local addresses `127.0.0.1` and `::1` are discarded.
- The synchronization **every 10 seconds** performs a two-way exchange for each enabled, online node: it pulls IPs from the node and merges them into the local table, and then hands the node the master's consolidated picture.
- The merge combines old and incoming observations **without double-counting** a single IP seen on multiple nodes, and **without resurrecting stale** records: the same staleness threshold as in the local job is applied — **30 minutes**. The freshest timestamp is kept for each IP. Records from other nodes receive a new local id (the id spaces of the nodes are independent); a concurrent insert of the same email is protected against duplicates.
- When the limit is computed, an IP is considered "live" if it was either seen in the current local scan or has a very fresh timestamp from the synchronized database (**within 2 minutes**). This is precisely what makes the limit work at the scale of the entire cluster, even if the address was seen on another node. When the limit is exceeded, the oldest "live" IPs are sent to the fail2ban log and the connections are forcibly dropped (remove/re-add the client via the Xray API).

What the user sees: the IP-count restriction now applies to the whole cluster rather than to each node separately; in the panel, a client shows the IPs seen on any node (within the 30-minute window). There is no separate button/setting for this — the synchronization runs automatically in the background, provided the node has `access.log` enabled and reachable (the limit itself also requires Fail2Ban on the node).

3. A separate status indicator: the node's panel is online but Xray has crashed

Previously a node's status was essentially "online / offline". If the node's panel responded, the node was considered online — even when the Xray core on it was not running and clients could not actually connect. In 3.3.0, the health of the panel and the health of the Xray core are separated.

How this is arranged:

- When polling a node, the master takes the `xray.state` and `xray.errorMsg` fields from the remote `/panel/api/server/status` response and saves them on the node. These fields are populated even on a successful panel ping when the core is unhealthy — precisely in order to distinguish panel availability from the Xray state.
- The `xray.state` values: `"running"` (running), `"stop"` (stopped), `"error"` (error).
- These values are translated into node statuses. New ones were added to the familiar ones:

Status key	English label	When it is shown
<code>online</code>	"Online"	the panel responds, Xray is running (<code>running</code>)
<code>offline</code>	"Offline"	the panel is unreachable / the ping did not succeed
<code>unknown</code>	"Unknown"	the state has not yet been determined

Status key	English label	When it is shown
<code>xrayError</code>	"Xray error"	the panel is online, but the Xray core is in the <code>error</code> state (there is an <code>errorMsg</code>)
<code>xrayStopped</code>	"Stopped"	the panel is online, but Xray is stopped (<code>stop</code>)

- For such a state, the UI uses a **separate purple indicator** (a color distinct from the green "online" and the red "offline"). Purple signals directly: the node can be reached, but the problem is in the Xray core itself, not in the network or in the panel.

What the user sees: instead of a misleading "green" when the core has crashed, the node is highlighted in **purple** with the status "**Xray error**" or "**Stopped**". This immediately shows that what needs fixing is Xray on the node (restart the core, check `errorMsg`), rather than troubleshooting the node's own availability. The same `xrayState` / `xrayError` is also propagated into the transitive sub-nodes (see point 1), so the incorrect core state is visible all along the chain.

13. Panel Settings

The "Settings" section (page title — **Panel Settings**) controls the behavior of the 3X-UI web panel itself: which address and port it listens on, how it is protected, how it interacts with the Telegram bot and external services, and in which time zone it runs scheduled tasks. Each parameter is stored in the database `settings` table as a key-value pair; if a value is absent from the DB, the default value is applied.

Important — applying changes. Any change on this page must be saved with the **Save** button, and then the panel must be restarted for the changes to take effect. The literal hint: "Every change made here needs to be saved. Please restart the panel to apply changes." When saving, the notification "Settings changed" is shown.

13.1. Saving and restarting the panel

Element	Purpose
Save	Writes all form fields to the DB (<code>POST /panel/setting/update</code>). Before writing, the values pass validation — invalid addresses, ports, or paths will be rejected, and the panel will return an error.
Restart Panel	Restarts the panel web server (<code>POST /panel/setting/restartPanel</code>). The restart happens with a 3-second delay. Hint: "Are you sure you want to restart the panel? If you cannot access the panel after restarting, please view the panel log info on the server." On success — "The panel was successfully restarted."
Reset to Default	Deletes all settings saved in the DB, after which the panel uses the default values. Administrator credentials are not reset by this operation.

The restart is performed by sending the `SIGHUP` signal to the panel process (or via a registered restart hook). On Windows, automatic restart via signal is not supported. **Changes to listening parameters (IP, port, path, domain, certificates, time zone) are applied only after the panel is restarted.**

13.2. General settings ("Panel" tab / *General*)

Interface language (*Language*)

The language of the panel web interface. The available languages are: `en-US` (English), `ru-RU` (Russian), `zh-CN`, `zh-TW`, `fa-IR`, `ar-EG`, `es-ES`, `id-ID`, `ja-JP`, `pt-BR`, `tr-TR`, `uk-UA`, `vi-VN`. This is a display setting and does not affect how Xray works.

Calendar type (*Calendar Type*)

- **Key:** `datepicker`
- **Default value:** `gregorian` (Gregorian).
- **Purpose:** the calendar type used in date selection (for example, when setting client expiry dates). Hint: "Scheduled tasks will run based on this calendar." The alternative value is the Persian (Jalali) calendar, which is in demand among the panel's Iranian audience.

Pagination size (*Pagination Size*)

- **Key:** `pageSize`
- **Default value:** `25`
- **Allowed values:** an integer from `0` to `1000`.
- **Purpose:** the number of rows per page in tables (connection/inbound lists). Hint: "Define page size for inbounds table. (0 = disable)" – with `0`, pagination is disabled, and all records are shown as a single list.
- **No panel restart required** (display setting).

Restart Xray after auto disable (*Restart Xray After Auto Disable*)

- **Key:** `restartXrayOnClientDisable`
- **Default value:** `true`
- **Purpose:** when a client is automatically disabled (due to expiration or reaching the traffic limit), Xray is restarted to tear down that client's already-established connections. Hint: "When a client is automatically disabled due to expiration or traffic limit, restart Xray.". The feature itself is unchanged – the toggle simply lives on the "Panel" (*General*) tab alongside the other general settings.

Remark model and separation character (*Remark Model & Separation Character*)

- **Key:** `remarkModel`
- **Default value:** `-ieo`
- **Purpose:** defines how the configuration name (remark) is formed in the subscription. The string consists of the **first character** – the separator, followed by a **sequence of order letters**:
 - `i` – inbound remark;
 - `e` – client email;
 - `o` – extra label (*extra*).

With the default value `-ieo`, the separator is `-`, and the order of the parts is: inbound → email → extra (for example, `MyInbound-user@mail-extra`). Empty parts are skipped. The "Sample Remark" field in the interface shows a preview of the generated name. Including the email in the name additionally depends on the "Include Email in Name" parameter in the subscription settings (subscription section).

Example: how the `remarkModel` value shapes the configuration name. Suppose the inbound is named `VLESS-Reality`, the client email is `alex@vpn`, and the extra label is `RU`. Then:

Field value	Resulting name (remark)
<code>-ieo</code> (default)	<code>VLESS-Reality-alex@vpn-RU</code>
<code>_ie</code>	<code>VLESS-Reality_alex@vpn</code>
<code>-ei</code>	<code>alex@vpn-VLESS-Reality</code>
<code>o</code> (space separator, label only)	<code>RU</code>

The first character of the string is always the separator; the remaining letters define which parts go into the name and in what order.

13.3. Panel access: IP, port, path, domain, certificate

This group defines the panel's network entry point. **All changes here are applied only after the panel is restarted.**

Field	Key	Default value	Description
Listen IP (<i>Listen IP</i>)	<code>webListen</code>	<code>""</code> (empty)	The IP on which the web panel listens. Empty = listen on all IPs. Hint: "The IP address for the web panel. (leave blank to listen on all IPs)". If specified, it must be a valid IP address (otherwise saving is rejected).
Listen Domain (<i>Listen Domain</i>)	<code>webDomain</code>	<code>""</code> (empty)	The panel's domain name for validating the request by domain. Empty = accept connections from any domains and IPs. Hint: "The domain name for the web panel. (leave blank to listen on all domains and IPs)"
Listen Port (<i>Listen Port</i>)	<code>webPort</code>	<code>2053</code>	The port on which the panel runs. Hint: "The port number for the web panel. (must be an unused port)". Allowed <code>1–65535</code> . The port must be free; the panel and the subscription service cannot use the same <code>IP:port</code> pair at the same time.
URI Path (<i>URI Path</i>)	<code>webBasePath</code>	<code>/</code>	The panel's base URL path (<code>basePath</code>). Hint: "The URI path for the web panel. (begins with '/' and concludes with '/')". When saving, the panel automatically adds a leading and trailing <code>/</code> if they are missing. Disallowed characters in the path are rejected.

Panel certificate (TLS / HTTPS)

Field	Key	Default value	Description
Public Key Path (<i>Public Key Path</i>)	<code>webCertFile</code>	<code>""</code>	The full path to the certificate (chain) file. Hint: "The public key file path for the web panel. (begins with '/')".
Private Key Path (<i>Private Key Path</i>)	<code>webKeyFile</code>	<code>""</code>	The full path to the private key file. Hint: "The private key file path for the web panel. (begins with '/')".

If **at least one** of the certificate/key paths is specified, the panel attempts to load the certificate + key pair when saving; on an error (a non-existent file, a mismatch between key and certificate) saving is rejected. When both correct paths are specified, the panel switches to HTTPS. Both fields empty = the panel works over plain HTTP.

Security warnings (*Security warnings*). The panel shows a "Your panel may be exposed:" block with warnings if it detects an insecure configuration:

- working over plain HTTP – "Panel is served over plain HTTP – set up TLS for production.";
- the default port 2053 – "Default port 2053 is well-known – change it to a random port.";
- the default base path `/` – "Default base path `/` is well-known – change it to a random path.";

- the default subscription path `/sub/` and JSON subscription `/json/` — "Default subscription path `/sub/`" is well-known — change it." "Default JSON subscription path `/json/`" is well-known — change it." These are recommendations, not blocks.

13.4. Session, panel proxy, and trusted proxies ("Proxy and Server" tab / *Proxy and Server*)

Session duration (*Session Duration*)

- **Key:** `sessionMaxAge`
- **Default value:** `360` (minutes, i.e. 6 hours).
- **Allowed values:** from `1` to `525600` minutes (1 year).
- **Purpose:** how long the administrator stays logged in without signing in again. The unit is the **minute**. Hint: "The duration for which you can stay logged in. (unit: minute)".

Panel traffic outbound (*Panel Traffic Outbound*)

- **Key:** `panelOutbound`
- **Default value:** `""` (empty = direct connection).
- **Purpose:** selects the Xray **outbound** through which the panel sends its **own requests** — panel/Xray version checks and downloads, requests to Telegram, the normal geo-file update — to bypass server-side filtering of GitHub/Telegram. The field is a **dropdown picker**: it lists outbounds from the Xray configuration template, outbounds derived from outbound subscriptions, and routing **balancers** (as a separate group). `blackhole` outbounds are excluded from the list — routing a download into a black hole is pointless. The literal hint: "Routes the panel's own requests — panel/Xray version checks and downloads, Telegram, and the normal geo-file update — through this Xray outbound to bypass server-side filtering of GitHub/Telegram. A loopback bridge inbound is added to the running config automatically and applied live. The Xray-native Geodata Auto-Update is not affected; it has its own download outbound. Leave empty for a direct connection."

How it works. When an outbound is selected, the panel itself adds a service loopback inbound (a SOCKS bridge with the tag `panel-egress`) to the running config plus a routing rule that steers the panel's own HTTP traffic to the chosen outbound. If a balancer is selected, `balancerTag` is emitted in the rule and the panel's traffic load-balances across its members. The bridge and rule are applied **live**, without a full panel restart. Leave the field empty for a direct connection. Xray's native geo-data Auto-Update is **not affected** by this setting — it has its own outbound inside the Xray router.

- **Format:** `socks5://` (or `socks5h://`) or `http(s)://`, with authorization of the form `socks5://user:pass@host:port` if needed. The supported schemes are strictly: `socks5`, `socks5h`, `http`, `https` — other schemes are considered invalid, and the panel then falls back to a direct connection. A typical example is Xray's own local SOCKS inbound.
- The literal hint: "Routes the panel's own outbound requests (geo updates, Xray/panel version checks, Telegram) through this proxy to bypass server-side filtering of GitHub/Telegram. Accepts `socks5://` or `http(s)://`, e.g. a local Xray SOCKS inbound. Leave empty for a direct connection."

- An invalid proxy does not cause a save error — the panel simply uses a direct connection and writes a warning to the log.

Field value example. If a local Xray SOCKS inbound is already running on the server on port `10808`, route the panel's own requests through it:

```
socks5://127.0.0.1:10808
```

For an external HTTP proxy with authorization:

```
http://user:pass@proxy.example.com:8080
```

After saving and restarting, the panel will fetch geo-database updates, check versions, and reach Telegram through the specified proxy.

Trusted proxy CIDRs (*Trusted proxy CIDRs*)

- **Key:** `trustedProxyCIDRs`
- **Default value:** `127.0.0.1/32,::1/128` (localhost only).
- **Format:** a comma-separated list of IP addresses or CIDR subnets (for example `10.0.0.0/8, 192.168.1.5`). Each element is validated as an IP or CIDR — an invalid value is rejected when saving.
- **Purpose:** lists the sources that are allowed to set the `X-Forwarded-Host`, `X-Forwarded-Proto` headers and the client's real IP header. The literal hint: "Comma-separated IPs/CIDRs allowed to set forwarded host, proto, and client IP headers." It needs to be configured if the panel runs behind a reverse proxy (nginx, Caddy, etc.), so that client IPs and the scheme are determined correctly.

Example: the panel behind a reverse proxy. If nginx runs on the same host and proxies requests to the panel, trust only localhost (the default value):

```
127.0.0.1/32,::1/128
```

If the proxy sits on a separate server in the internal `10.0.0.0/8` network, add its subnet, otherwise the panel will ignore the headers it forwards and will see the proxy's IP instead of the real client:

```
127.0.0.1/32,::1/128,10.0.0.0/8
```

A matching nginx block that forwards the real IP and the scheme:

```
proxy_set_header X-Real-IP      $remote_addr;
proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
proxy_set_header X-Forwarded-Proto $scheme;
proxy_set_header X-Forwarded-Host $host;
```

13.5. Telegram bot ("Telegram Bot" tab / *Telegram Bot*)

Enable Telegram Bot (*Enable Telegram Bot*)

- **Key:** `tgBotEnable`
- **Type/default:** boolean, `false`.
- **Purpose:** enables the Telegram bot. Hint: "Enables the Telegram bot."

Telegram token (*Telegram Token*)

- **Key:** `tgBotToken`
- **Default:** `""`.
- **Purpose:** the bot token. Hint: "The Telegram bot token obtained from '@BotFather'."
- **Security specifics:** the token is among the secret values. It is not returned in the panel's response when reading settings (the field is cleared, only a "configured/not configured" flag is provided). If the field is left empty when saving, the previously saved token is **kept** (not overwritten).

Telegram bot language (*Telegram Bot Language*)

- **Key:** `tgLang`
- **Default:** `en-US`.
- **Purpose:** the language of the bot's messages (independent of the web interface language). The list of available languages matches the panel's languages.

Bot admin User ID (*Admin Chat ID*)

- **Key:** `tgBotChatId`
- **Default:** `""`.
- **Format:** one or more numeric Telegram User IDs **separated by commas**.
- **Purpose:** the recipients of notifications and the administrators who are allowed to manage the panel via the bot. Hint: "The Telegram Admin Chat ID(s). (comma-separated)(get it here `@userinfobot`) or (use '/id' command in the bot)".

Notification time (*Notification Time*)

- **Key:** `tgRunTime`
- **Default:** `@daily` (once per day).
- **Format:** a string in **Crontab** format (both standard cron expressions and abbreviations of the form `@daily`, `@hourly`, `@every 1h` are supported). Hint: "The Telegram bot notification time set for periodic reports. (use the crontab time format)". Controls the bot's periodic reports.

Field value examples.

Value	When the bot sends a report
<code>@daily</code>	once a day at midnight (default)

Value	When the bot sends a report
@hourly	every hour
@every 6h	every 6 hours
0 9 * * *	daily at 09:00
30 8 * * 1	every Monday at 08:30

The time is interpreted in the zone from the "Time Zone" setting (section 13.6).

SOCKS proxy (*SOCKS Proxy*)

- **Key:** tgBotProxy
- **Default:** ""
- **Purpose:** a SOCKS5 proxy, used separately for the bot's connection to Telegram. Hint: "Enables SOCKS5 proxy for connecting to Telegram. (adjust settings as per guide)". It applies specifically to the bot's traffic (different from the general "Panel Network Proxy" in section 13.4).

Telegram API Server (*Telegram API Server*)

- **Key:** tgBotAPIServer
- **Default:** "" (use the standard server api.telegram.org).
- **Format:** a http(s)://... URL; when saving, it passes a URL validity check – an invalid address is rejected. Hint: "The Telegram API server to use. Leave blank to use the default server.". Needed for a self-hosted Telegram Bot API server.

Bot notifications (the "Notifications" group / *Notifications*)

Field	Key	Default	Description
Database Backup (<i>Database Backup</i>)	tgBotBackup	false	Send the DB backup file to Telegram together with the report. Hint: "Send a database backup file with a report."
Login Notification (<i>Login Notification</i>)	tgBotLoginNotify	true	Notify on a panel login attempt. Hint: "Get notified about the username, IP address, and time whenever someone attempts to log into your web panel."
Expiration Date Notification (<i>Expiration Date Notification</i>)	expireDiff	0	How many days before a client's expiry date to send a notification. 0 – disabled. Allowed ≥ 0 . Hint: "Get notified about expiration date when reaching this threshold. (unit: day)".
Traffic Cap Notification (<i>Traffic Cap Notification</i>)	trafficDiff	0	The remaining-traffic threshold for the notification. Hint: "Get notified about traffic cap when reaching this threshold. (unit: GB)". Allowed 0–100.
CPU Load Notification (<i>CPU Load Notification</i>)	tgCpu	80	Notify administrators if the CPU load exceeds the threshold (in %). Allowed 0–100. Hint: "Get notified if CPU load exceeds this threshold. (unit: %)".

13.6. Date and time ("Date and Time" tab / *Date and Time*)

Time zone (*Time Zone*)

- **Key:** `timeLocation`
- **Default value:** `Local` (the server's system time zone).
- **Format:** a zone name from the IANA tz database (for example, `Europe/Moscow`, `UTC`, `Asia/Tehran`).
- **Purpose:** the time zone in which the panel runs scheduled tasks (bot reports, traffic resets/checks, expirations). Hint: "Scheduled tasks will run based on this time zone..".
- **Validation:** when saving, the zone is checked – a non-existent zone is rejected. If an invalid value later ends up in the DB, the panel falls back to `Local` at runtime, and if that too is unavailable – to `UTC`.

13.7. External traffic and Xray behavior ("External Traffic" tab / *External Traffic*)

Field	Key	Default	Description
External Traffic Inform (<i>External Traffic Inform</i>)	<code>externalTrafficInformEnable</code>	<code>false</code>	Notify an external API on every traffic update. Hint: "Inform external API on every traffic update..".
External Traffic Inform URI (<i>External Traffic Inform URI</i>)	<code>externalTrafficInformURI</code>	<code>""</code>	The URL to which the panel sends traffic updates. Passes a URL validity check when saving. Hint: "Traffic updates are sent to this URI..".
Restart Xray After Auto Disable (<i>Restart Xray After Auto Disable</i>)	<code>restartXrayOnClientDisable</code>	<code>true</code>	Restart Xray when a client is automatically disabled due to expiration or exceeding the traffic limit. Hint: "When a client is automatically disabled due to expiration or traffic limit, restart Xray.". The toggle is on the "Panel" (General) tab – see section 13.2; it is listed here for completeness.

13.8. Other: Xray configuration template and test URL

Xray configuration template (*xrayTemplateConfig*)

- **Key:** `xrayTemplateConfig`
- **Default:** the embedded JSON template, shipped with the build.
- **Purpose:** the base JSON template for the Xray-core configuration, on top of which the panel builds the inbounds/outbounds. This value is **not returned** in the normal output of all settings and is edited on a separate Xray configuration page, not in the general list of panel settings fields. The standard default template is available via `GET /panel/setting/getDefaultJsonConfig`.

Outbound test URL (*xrayOutboundTestUrl*)

- **Key:** `xrayOutboundTestUrl`
- **Default:** `https://www.google.com/generate_204`

- **Purpose:** the URL used when testing the operability of outbound connections. When set, it is sanitized as an HTTP(S) URL.

13.9. Administrator account and API tokens

These parameters are on the adjacent tab ("Account" / *Authentication*) and are covered in detail in the security section; here is a brief summary of the keys.

- **Changing credentials** (the "Current Username", "Current Password", "New Username", "New Password" fields) is saved with a separate request `POST /panel/setting/updateUser`. The correct current username and password are required; the new username and password must not be empty. Messages: "You have successfully changed the credentials of the administrator." / "Wrong username or password".
- **Two-factor authentication (2FA)** — the keys `twoFactorEnable` (default `false`) and the secret `twoFactorToken`. The token is a secret: when 2FA is enabled, an empty field when saving does not overwrite the existing token. On the **first** enabling of 2FA, the panel invalidates the current sessions (the "login epoch" is raised).
- **API tokens** are managed by separate endpoints (`/panel/setting/apiTokens...`): list, create (`apiTokens/create`), delete, enable/disable. The token itself is shown **only once, at creation**, and is not stored in readable form: "Copy this token now. For security it is not stored in readable form and will not be shown again."

Details on 2FA, passwords, LDAP synchronization, and subscription formats (JSON/Clash, fragmentation, noises, mux) are moved to the corresponding separate sections of the manual.

13.10. API changes in 3.3.0 (important for integrations)

In version 3.3.0, the structure of the server API paths changed. If you have external integrations (scripts, bots, central panels, CI jobs) that access the panel over HTTP, they **need to be fixed**, otherwise they will stop working.

 **BREAKING CHANGE:** the `/panel/setting/*` and `/panel/xray/*` endpoints moved under `/panel/api`

Previously, management of panel settings and the Xray configuration lived separately, under the paths `/panel/setting/*` and `/panel/xray/*`. Now both sets are registered inside the common API group `/panel/api`. The old paths are **completely removed** — a request to them will return 404.

Why this was done: the entire `/panel/api` group goes through the unified access check, that is, these endpoints now accept the same `Authorization: Bearer <token>` header as the rest of the API. An API token is full administrator access, and in this way the entire API surface became uniform.

What did NOT change: the web interface pages (SPA routes) `/panel/settings` and `/panel/xray` stayed in place — this is only about the server API endpoints.

Path mapping table (old → new)

The prefix for all paths below — `api/` was simply added after `/panel/`.

Was (≤ 3.2.x)	Now (3.3.0)	Method
/panel/setting/all	/panel/api/setting/all	POST
/panel/setting/defaultSettings	/panel/api/setting/defaultSettings	POST
/panel/setting/update	/panel/api/setting/update	POST
/panel/setting/updateUser	/panel/api/setting/updateUser	POST
/panel/setting/restartPanel	/panel/api/setting/restartPanel	POST
/panel/setting/getDefaultJsonConfig	/panel/api/setting/ getDefaultJsonConfig	GET
/panel/setting/apiTokens	/panel/api/setting/apiTokens	GET
/panel/setting/apiTokens/create	/panel/api/setting/apiTokens/create	POST
/panel/setting/apiTokens/delete/:id	/panel/api/setting/apiTokens/ delete/:id	POST
/panel/setting/apiTokens/ setEnabled/:id	/panel/api/setting/apiTokens/ setEnabled/:id	POST
/panel/xray/	/panel/api/xray/	POST
/panel/xray/update	/panel/api/xray/update	POST
/panel/xray/getDefaultJsonConfig	/panel/api/xray/getDefaultJsonConfig	GET
/panel/xray/getXrayResult	/panel/api/xray/getXrayResult	GET
/panel/xray/getOutboundsTraffic	/panel/api/xray/getOutboundsTraffic	GET
/panel/xray/resetOutboundsTraffic	/panel/api/xray/resetOutboundsTraffic	POST
/panel/xray/testOutbound	/panel/api/xray/testOutbound	POST
/panel/xray/warp/:action	/panel/api/xray/warp/:action	POST
/panel/xray/nord/:action	/panel/api/xray/nord/:action	POST
/panel/xray/outbound-subs (and / outbound-subs/*)	/panel/api/xray/outbound-subs (and / outbound-subs/*)	GET/POST/ DELETE

The sub-path names themselves, the request bodies, and the response formats did not change — **only the prefix** changed.

How to fix existing integrations

1. Find all occurrences of `/panel/setting/` and `/panel/xray/` in your scripts/configs.
2. Replace the prefix: add `api/` right after `/panel/` (for example, `/panel/setting/all` → `/panel/api/setting/all`).
3. The request bodies, parameters, and response format do not need to be edited — only the URL changes.
4. Since settings and the Xray configuration are now under `/panel/api`, they can (and should) be accessed with the same API token `Authorization: Bearer <token>` as `/panel/api/inbounds/*` and the other endpoints. Don't forget the CSRF middleware, which is enabled for the entire `/panel/api` group.

Example: reading all settings via the API. Before ($\leq 3.2.x$):

```
curl -sk -X POST "https://panel.example.com:2053/MyPath/panel/setting/all" \
-H "Authorization: Bearer <token>"
```

Now (3.3.0) — `api/` is added after `/panel/`:

```
curl -sk -X POST "https://panel.example.com:2053/MyPath/panel/api/setting/all" \
-H "Authorization: Bearer <token>"
```

Likewise restarting the panel: `POST /panel/api/setting/restartPanel`. The old path `/panel/setting/restartPanel` now returns 404.

Typed API: schemas and documentation (Swagger / OpenAPI)

In 3.3.0, the OpenAPI specification became fully typed. Previously, typed responses were described by an empty object `{}`; now the components and schemas (`components.schemas`) are generated directly from the data models. Thanks to this:

- Swagger UI shows real data models, rather than faceless stubs.
- External generators (`openapi-generator` , etc.) can build ready-made clients in the desired language from the specification.
- A `$ref` to a concrete model is attached to each typed response, and response examples are included.

Where to look for the API documentation:

- **The built-in Swagger page.** In the panel menu — the **"API Documentation"** item (the SPA route `/panel/api-docs`). Here all endpoints are listed interactively, with descriptions, request bodies, and response examples.
- **The raw OpenAPI 3.0 specification** is served at `/panel/api/openapi.json` . This URL can be fed directly into Postman, Insomnia, or `openapi-generator` . The specification is embedded in the binary at build time; when the panel runs under a non-standard `webBasePath` , the `servers` field in the specification is automatically rewritten to the current base path, so that the "Try it out" button and external generators hit the correct prefix.

14. Telegram Bot

The 3X-UI panel includes a built-in Telegram bot through which you can receive notifications about the server and client status, as well as manage individual clients directly from the messenger. The bot works using long polling (continuous polling of Telegram), so it does not require an external domain or an open port — outbound access to the Telegram servers is sufficient.

The bot distinguishes between two types of interlocutors:

- **Administrator** — a user whose Telegram User ID is specified in the bot settings (the "Bot Admin User ID" field). Has access to all features: server statistics, backup, client management, and restarting Xray.
- **Client** — any other user whose Telegram User ID is linked to a specific inbound client (the client's `tgId` field). Sees only information about their own subscriptions.

Example: linking a client to Telegram. For a user to receive statistics about their own subscription, their numeric Telegram User ID is written into the client's `tgId` field. In the client's JSON settings this looks like:

```
{
  "email": "ivan",
  "id": "6f1e6b1a-0c3d-4f2a-9b7e-1a2b3c4d5e6f",
  "tgId": "123456789",
  "enable": true,
  "limitIp": 2,
  "totalGB": 53687091200,
  "expiryTime": 0
}
```

After that, the user with User ID `123456789` can ask the bot `/usage ivan` and see their statistics. The administrator can set the same ID via the  "Set Telegram User" button on the client card — there is no need to edit the JSON by hand.

14.1. Enabling and configuring the bot

All bot parameters are set in the panel under **Settings** → **Telegram Bot**. After changing the settings, you need to save them and restart the panel — the bot is initialized when the web server starts.

Field (UI)	Setting key	Default value	Description
Enable Telegram bot	<code>tgBotEnable</code>	<code>false</code>	The main toggle. Hint: "Access to panel functions through the Telegram bot". While disabled, the bot does not start and notification jobs are not scheduled.
Telegram Token	<code>tgBotToken</code>	(empty)	The bot token. Hint: "You need to obtain the token from the Telegram bot manager @botfather". Without a non-empty token the bot does not start.
SOCKS Proxy	<code>tgBotProxy</code>	(empty)	A proxy for connecting to Telegram. Hint: "If you need a Socks5 proxy to connect to Telegram, configure its parameters according to the guide".

Field (UI)	Setting key	Default value	Description
Telegram API Server	<code>tgBotAPIServer</code>	(empty)	An alternative Telegram API server. Hint: "The Telegram API server to use. Leave empty to use the default server".
Bot Admin User ID	<code>tgBotChatId</code>	(empty)	One or more Telegram User IDs of administrators, separated by commas. Hint: "To obtain a User ID, use <code>@userinfobot</code> or the <code>/id</code> command in the bot".
Notification frequency for administrators from the bot	<code>tgRunTime</code>	<code>@daily</code>	The schedule for the periodic report in crontab format. Hint: "Specify the notification interval in Crontab format".
Database backup	<code>tgBotBackup</code>	<code>false</code>	Hint: "Send a notification with the database backup file". Attaches the backup to the periodic report.
Login notification	<code>tgBotLoginNotify</code>	<code>true</code>	Hint: "Displays the username, IP address, and time when someone attempts to log in to your panel".
CPU load threshold for notification	<code>tgCpu</code>	<code>80</code>	The CPU load threshold as a percentage (validation 0–100). Hint: "Notify Telegram administrators if the CPU load exceeds this threshold (value: %)". When set to 0, the CPU check is disabled.
Telegram Bot Language	—	—	The language in which the bot composes all its messages.

Obtaining a token via @BotFather

1. Open a chat with **@BotFather** in Telegram.
2. Send the `/newbot` command and follow the instructions (the bot name and a unique `username` ending in `bot`).
3. BotFather will issue a token like `123456789:AA...` . Copy it into the **Telegram Token** field.

Obtaining the administrator User ID

The User ID is the numeric identifier of the account (not the username). You can find it in two ways:

- Message the **@userinfobot** bot.
- Start an already configured bot and send it the `/id` command — it will return your ID.

Enter the obtained number into the **Bot Admin User ID** field. To assign several administrators, list their IDs separated by commas (for example, `11111111,22222222`). Each ID is validated as an integer; an invalid value will cause the bot to fail to start.

Example: value of the "Bot Admin User ID" field. A single administrator is just a number:

```
123456789
```

Two administrators separated by a comma (spaces are optional):

123456789,987654321

Each value must be an integer. An entry like `@username` or `123 456` (with a space inside the number) will not work – the bot will fail to start.

Proxy

The `socks5://`, `http://`, and `https://` schemes are supported. If the proxy field is left empty, the bot attempts to use the panel's shared proxy (if one is set and its scheme is supported). A URL with an unsupported scheme or invalid syntax is ignored – the bot connects directly. A proxy is useful when direct access to the Telegram API from the server is blocked.

Telegram API server

By default, the bot uses the official Telegram API. In the **Telegram API Server** field you can specify the address of your own Bot API server (`telegram-bot-api`). The URL is checked for safety; a blocked or invalid address is discarded, and the default server is used.

14.2. Main menu and buttons

The menu is invoked with the `/start` command. The buttons are an inline keyboard attached to the message; the set of buttons depends on whether you are an administrator or a client.

Administrator menu

Button	Action
 Sorted Traffic Usage Report	Lists all clients sorted by traffic, with each one's consumption; "extra" emails without data are marked " No result!".
 Server Usage	A server summary (see section 14.5). The " Refresh" button redraws the data.
Reset All Traffic	Resets the traffic counters of all clients. Asks for confirmation ("Are you sure? 🤔"), then for each client displays " Success" or " Failed", and at the end " Traffic reset process finished for all clients."
 Get DB Backup	Sends the database file and <code>config.json</code> (see section 14.6).
 Get Ban Logs	Sends the log files of addresses banned by the IP limit.
 Get Inbounds	A summary of all inbounds: Remark, port, traffic, number of clients, expiry date.
 Deplete Soon	A list of inbounds and clients whose traffic or term will soon be exhausted (see section 14.5).
 Commands	Displays help on administrator commands.
 Online Clients	The number and list of clients that are online; tapping an email opens the client card. The " Refresh" button.
 All Clients	Opens an inbound selection, then a list of its clients – for viewing/management.
 Add Client	Launches the add-client wizard (select inbound → draft → confirmation).

Button	Action
Subscription settings / individual links / QR code	Selecting an inbound and a client to get the subscription link, individual links, or QR codes.

Client menu

A client has access to a limited set of buttons:

Button	Action
Get Usage	Shows data for all subscriptions linked to the client's Telegram User ID.
Commands	Displays help on client commands.
Subscription settings	Selecting one's own client → subscription link.
Individual links	Selecting one's own client → individual links.
QR code	Selecting one's own client → QR codes.

If the user has no client with their Telegram User ID, the bot replies: "❌ Your configuration was not found! Please ask the administrator to use your Telegram User ID in the configuration. 🆔 Your User ID: ...". This ID must be passed to the administrator so that they enter it into the client's field.

14.3. Bot commands

Four commands are registered for the bot, visible in Telegram's "/" menu:

Command	Description (from the menu)	Access	What it does
/start	Show the main menu	everyone	A greeting; for the administrator it additionally shows "👋 Welcome to management bot." and the main menu.
/help	Bot help	everyone	Displays a general greeting and a prompt to choose a menu item.
/status	Check bot status	everyone	Replies "✅ Bot is OK!".
/id	Show your Telegram ID	everyone	Returns "🆔 Your ID: ...". Convenient for obtaining your own User ID.

In addition to the registered ones, three more argument commands are handled (they are not shown in the "/" menu, but they work):

- **/usage [Email]** — searches for a client by email.
 - For an **administrator** it shows the full client card (with management buttons).
 - For a **client** it shows only their own subscription with the specified email (based on the Telegram User ID link). Without an argument, the bot asks for an email: "❗ Please provide a text to search!".
- **/inbound [inbound name]** — administrator only. Searches for an inbound by Remark and displays its parameters with the statistics of all clients. Without an argument (or for a client) — "❗ Unknown command".

- **/restart** — administrator only. Restarts the Xray Core. Possible responses: "  Operation successful!", "Xray Core is not running." (if the core is not running), "  Error in operation.". Any arguments after **/restart** result in an unknown command message with the **/restart** hint.

In group chats, a command of the form **/command@botusername** is handled only if the username matches the name of the current bot.

Administrator help (the "Commands" button):

 To restart Xray Core: **/restart**
 To search for a client email: **/usage [Email]**
 To search for inbounds (with client stats): **/inbound [Remark]**
 Telegram Chat ID: **/id**

Client help:

\$
 To view information about your subscription: **/usage [Email]**
 Telegram Chat ID: **/id**

14.4. Client management (administrator only)

After opening a client card (via "All Clients", "Online Clients", "Deplete Soon", or **/usage**), the administrator sees information about the client (email, linked inbounds, "Active" status, connection status, expiry date, traffic consumption) and inline management buttons:

Button	Purpose
 Refresh	Re-reads the client card.
 Reset Traffic	Zeroes the client's traffic counter. Requires the confirmation "  Confirm Reset Traffic?".
 Traffic Limit	Sets a traffic limit. Preset values:  Unlimited (0), 1/5/10/20/30/40/50/60/80/100/150/200 GB, or "  Custom"  entering a number on a built-in numeric keypad (buttons 0–9, "  " – reset to 0, "  " – erase the last digit, "  Confirm: N"). The value is specified in gigabytes.
 Change Expiry Date	Preset options:  Unlimited, "  Custom", add 7/10/14/20 days, 1/3/6/12 months. A positive number extends the term (adds days to the current expiry date, or to "now" if the term has already expired); 0 removes the term limit.
 IP Log	Shows the client's recorded IP addresses (with  timestamps, if available). From the log, "  Refresh" and "  Clear IPs" are available (with the confirmation "  Confirm Clear IPs?").
 IP Limit	A limit on simultaneous IPs. Options:  Unlimited (0), 1–10, or "  Custom" (numeric keypad).
 Set Telegram User	Shows the client's currently linked Telegram User ID; allows clearing the link ("  Remove Telegram User" with confirmation). Linking a new user is done via the Telegram system contact picker.
 Enable / Disable	Enables or disables the client. Requires the confirmation "  Confirm Enable/Disable User?".

All operations that change the configuration (traffic/IP limit, expiry date, linking/unlinking a Telegram user, enable/disable) flag Xray for a restart when necessary, so that the changes take effect. After a successful operation, the bot displays a confirmation of the form "✅ : ..." and shows the card again.

Any digit input in the wizards is limited to a value < 999999.

14.5. Notifications and reports

Notifications are sent to all administrators (all User IDs from `tgBotChatId`).

Panel login notification

Controlled by the **Login notification** checkbox (`tgBotLoginNotify` , enabled by default). On every attempt to log in to the web panel, administrators receive a message:

- On success: "✅ Logged in to the panel successfully." + host, username, IP, time.
- On failure: "❌ Login attempt to the panel failed." + host, **reason** (for example, "2FA Failed" for an incorrect second factor), username, IP, time.

CPU load exceeded

If the `tgCpu` threshold > 0, a check of the average CPU load over a minute is run every 10 seconds. When the threshold is exceeded, administrators receive: "❌ CPU Load N% exceeds the threshold of M%".

Periodic report (scheduled)

Scheduled by the cron expression from the **Notification frequency** field (`tgRunTime` , `@daily` by default). If the value is empty or invalid, `@daily` is used. The report includes:

Schedule builder

The **Notification frequency for administrators from the bot** field is set through a schedule builder rather than by typing a string by hand. First, pick a mode from the dropdown:

- **@every — repeat at an interval** — a number field and a unit selector (**Seconds** / **Minutes** / **Hours**) appear; the result is composed into an expression like `@every 6h`.
- **@hourly — every hour**, **@daily — every day at 00:00**, **@weekly — every week**, **@monthly — every month** — ready-made presets saved as the corresponding macro (`@hourly` , `@daily` , `@weekly` , `@monthly`).
- **Custom (crontab)** — a field for your own crontab expression. The panel's scheduler runs with seconds enabled, so a custom expression has **6 fields**: second, minute, hour, day of month, month, day of week (for example, `0 30 8 * * *` — every day at 08:30:00). When you switch to this mode, the field is seeded with the crontab equivalent of the current selection, giving you a real expression to start from.

Example: values of the "Notification frequency" field (`tgRunTime`). Both shorthand aliases and the full crontab format are supported:

Value	When it fires
@daily	once a day at midnight (the default)
@hourly	every hour
@every 6h	every 6 hours
0 9 * * *	every day at 09:00
0 9 * * 1	every Monday at 09:00
0 */12 * * *	every 12 hours (at 00:00 and 12:00)

The crontab field order is: minute, hour, day of month, month, day of week.

1. The line "[?] Scheduled Reports: " and the current date/time.
2. The **server status** (see below).
3. The "Deplete Soon" block for inbounds and clients.
4. Personal notifications to clients with a linked Telegram User ID — each non-admin client receives a list of their subscriptions whose traffic/term will soon be exhausted (including disabled ones).
5. If **Database backup** (`tgBotBackup`) is enabled — a database backup to the administrators.

The **server status** contains: host, the 3X-UI and Xray versions, IPv4/IPv6, uptime (in days), the average load (Load1/2/3), RAM (current/total), the number of online clients, TCP/UDP connection counters, total network traffic (↑/↓), and the Xray status.

"Deplete Soon" shows:

- by inbound: the number of disabled ones and the number "depleting soon", followed by a listing of such inbounds (Remark, port, traffic, expiry date);
- by client: the same, plus client cards and buttons with their emails (tapping opens the client card).

The "depleting soon" thresholds are taken from the panel's general settings: the traffic margin (in GB) and the term margin (in days). An inbound/client is considered "depleting" if less than the threshold remains until the traffic limit OR less than the threshold remains until the expiry date.

14.6. Backup and logs

- **DB backup** (the "  Get DB Backup" button or the checkbox in the periodic report): the bot sends the backup time, the database file (`x-ui.db` , or `x-ui.dump` for PostgreSQL), and the Xray configuration file `config.json` .
- **Ban logs** (the "  Get Ban Logs" button): sends the current and previous log files of IP addresses banned for exceeding the IP limit. Empty files are not sent.

14.7. Operational details

- **Long messages** are split into parts (a threshold of ~2000 characters), with the inline keyboard attached to the last part.
- **Concurrency**: commands and button presses are processed concurrently (a pool of up to 10 simultaneous handlers).

- **Send reliability:** on connection errors, messages are resent with an exponential delay (1s/2s/4s, up to 3 attempts).
 - **Caching:** the "Server Usage" data is cached so that frequent "Refresh" taps do not load the system.
 - **Bot restart:** when the settings are saved and the panel is restarted, the previous polling loop is correctly stopped and a new one is started — only one update-receiving instance runs at a time.
-

15. Geo databases (geoip / geosite and custom)

Geo databases are binary `.dat` files that Xray-core uses to route and filter traffic by country membership (IP ranges) or by domain category. The panel can download and update both the standard set of geo files and arbitrary user-defined sources specified by URL. All files are stored in the `bin` directory next to the Xray binary (the default path is `bin`, overridden by the `XUI_BIN_FOLDER` environment variable).

15.1. What geoip.dat and geosite.dat are

- **geoip.dat** — a database mapping "IP address → country/region code". It is used in routing rules as `geoip:<code>`, for example `geoip:ru`, `geoip:cn`, as well as for special labels such as `geoip:private` (private/local networks). In essence it answers the question "which country is this IP located in".
- **geosite.dat** — a database mapping "domain → category/list". It is used as `geosite:<category>`, for example `geosite:category-ads-all` (advertising domains), `geosite:google`, `geosite:ru`. In essence these are grouped lists of domains.

These files are needed to build rules such as "all traffic to Russian IPs/domains goes directly, everything else goes through an outbound" and similar. The rules themselves are defined in the Xray routing section; the geo databases merely supply the data for them. Without up-to-date geo files, rules that reference `geoip:` / `geosite:` will not work or will rely on outdated lists.

Example: a "Russian domains and IPs go directly" rule. Such a rule in the routing section sends all traffic to Russian resources to the outbound tagged `direct`:

```
{
  "type": "field",
  "domain": ["geosite:category-ru"],
  "ip": ["geoip:ru"],
  "outboundTag": "direct"
}
```

15.2. Standard geo files and their update

The panel contains a fixed allowlist of six standard files with hard-coded download sources. The update is performed via `POST /panel/api/server/updateGeofile/:fileName` (or without a file name — to update all of them at once).

Example: updating a single file and all of them via the API. Update only `geoip_RU.dat`:

```
curl -X POST 'https://panel.example.com:2053/panel/api/server/updateGeofile/
geoip_RU.dat' \
-H 'Cookie: 3x-ui=<session-cookie>'
```

Update all six standard files in a single request (no file name):

```
curl -X POST 'https://panel.example.com:2053/panel/api/server/updateGeofile' \
-H 'Cookie: 3x-ui=<session-cookie>'
```

A successful response:

```
{ "success": true, "msg": "Geofile updated successfully", "obj": null }
```

File name	Source (releases repository)
geoip.dat	github.com/Loyalsoldier/v2ray-rules-dat (geoip.dat)
geosite.dat	github.com/Loyalsoldier/v2ray-rules-dat (geosite.dat)
geoip_IR.dat	github.com/chocolate4u/Iran-v2ray-rules (geoip.dat)
geosite_IR.dat	github.com/chocolate4u/Iran-v2ray-rules (geosite.dat)
geoip_RU.dat	github.com/runetfreedom/russia-v2ray-rules-dat (geoip.dat)
geosite_RU.dat	github.com/runetfreedom/russia-v2ray-rules-dat (geosite.dat)

Specifics of updating the standard files:

- **Button to update a single file.** Before the download a confirmation is shown: *Do you really want to update the geofile?* with the explanation *This will update the #filename# file*. On success a notification pops up: *Geofile updated successfully*.
- **The "Update all" button** (*Update all*) downloads all six files. Confirmation: *This will update all geofiles*.
- **Conditional download.** If the local file already exists, the `If-Modified-Since` header with the file's modification time is added to the request. A `304 Not Modified` server response means the file has not changed – it is not downloaded again, only the file's timestamp is updated.
- **File name safety.** Only names from the allowlist are accepted; the name is checked to contain no `..`, no path separators `/` or `\`, no absolute paths, and must match the pattern `^[a-zA-Z0-9._-]+\\.dat$`. Any name outside the list is rejected with the error "Invalid geofile name".
- **Xray restart.** After the geo files are downloaded, Xray-core is restarted so that it re-reads the updated databases. If the restart fails, a corresponding line is added to the error message.

15.3. Geodata auto-update via Xray (Geodata Auto-Update)

Additional `.dat` sources at an arbitrary URL are added not by the panel itself but through Xray-core's native `geodata` section. The corresponding screen lives in the Xray Updates modal (Dashboard → Xray updates, `xrayUpdates`) as the "Geodata Auto-Update" tab. Here the panel only edits the `geodata` key in the Xray config template; downloading, validating, and hot-reloading the files is handled by the Xray core itself.

At the top of the section a hint is shown: *Xray downloads these files on schedule and hot-reloads them without a restart. URLs must be HTTPS. Each file must already exist in the bin folder once before Xray can update it.*

Section fields

- **Schedule (cron)** (*Schedule (cron)*) – a 5-field cron string; the default value is `0 4 * * *` (daily at 04:00). On save the string is checked to contain exactly 5 fields, otherwise the error `*Cron must have 5 fields, e.g. 0 4 * * *` is shown.
- **Download through outbound (optional)** (*Download through outbound (optional)*) – a dropdown with the tags of available outbounds (plus subscription outbounds) through which Xray will download the files;

outbounds with the `blackhole` protocol are excluded. The field may be left empty – then a direct connection is used. This choice is independent of the outbound used for the panel's own requests (see §11): geodata auto-update has its own separate download outbound.

- **File list** – each row defines a "URL + File name" pair (*File name*). The URL must start with `https://` (otherwise *Each file needs an HTTPS URL.*). The file name must be plain, with no paths or separators – only the characters `^[A-Za-z0-9._-]+$` (otherwise *File names must be plain names like geosite_custom.dat (no paths).*). When a URL is entered, the panel tries to fill in the file name automatically from the last path segment. The "Add file" button (*Add file*) adds a row, and the trash button removes it.

If the list is empty, a hint is shown: *No files configured. Reference files in routing rules as ext:geosite_custom.dat:category.*

Saving

The "Save & Restart Xray" button (*Save & Restart Xray*) shows a confirmation *Save geodata settings?* with the explanation *This updates the Xray config template and restarts Xray.* After saving, the `geodata` key is written into the config template (`POST /panel/api/xray/update`) and Xray is restarted (`POST /panel/api/server/restartXrayService`). If the file list is empty, the `geodata` key is removed from the template.

Important specifics:

- **The file must already exist in `bin`.** Xray only updates `.dat` files that are already present in the `bin` folder at startup. So a new custom file is first placed into `bin` manually (or at least an empty/outdated version is created there under the desired name), and only then does Xray keep it up to date on schedule.
- **Hot reload.** After a scheduled download Xray re-reads the updated databases without a full restart of the process.
- **Compatibility.** Previously downloaded geo files (both standard and custom) keep working in routing rules with the `ext:` syntax unchanged.

If the list is empty, a hint is displayed: *No custom geo sources yet – click Add to create one.*

Table columns and source fields

Field (UI)	JSON	Default value	Description
Type (<i>Type</i>)	<code>type</code>	– (required)	Resource type: only <code>geosite</code> or <code>geoip</code> . Determines the name of the resulting file.
Alias (<i>Alias</i>)	<code>alias</code>	– (required)	A short identifier of the source. The file name is built from it and the type.
URL (<i>URL</i>)	<code>url</code>	– (required)	A direct link to the <code>.dat</code> file (http/https).
Enabled (<i>Enabled</i>)	–	–	Flag indicating whether the source is active in the list.
Last updated (<i>Last updated</i>)	<code>lastUpdatedAt</code>	<code>0</code>	Time of the last successful update (Unix time; <code>0</code> – not updated yet).

Field (UI)	JSON	Default value	Description
Routing (ext:...) (<i>Routing (ext:...)</i>)	—	—	A ready-made string for routing rules: <code>ext:<file.dat>:tag</code> .
Actions (<i>Actions</i>)	—	—	The "Edit", "Delete", "Update now" buttons.

Additionally, the database stores service fields: `localPath` (the actual path to the file in the `bin` directory), `lastModified` (the value of the `Last-Modified` header from the server, used for conditional download), `createdAt` and `updatedAt`.

File naming

The name of the resulting file is generated automatically from the type and the alias:

- type `geoip` → `geoip_<alias>.dat`;
- type `geosite` → `geosite_<alias>.dat`.

For example, a source with type `geosite` and alias `myads` will create the file `geosite_myads.dat`.

Example: adding a source via the API. Add your own list of advertising domains as a `geosite` resource with the alias `myads`:

```
curl -X POST 'https://panel.example.com:2053/panel/api/server/customGeo/add' \
-H 'Cookie: 3x-ui=<session-cookie>' \
-H 'Content-Type: application/json' \
-d '{
  "type": "geosite",
  "alias": "myads",
  "url": "https://example.com/lists/myads.dat"
}'
```

The panel downloads the file into the `bin` directory as `geosite_myads.dat`, saves the record, and restarts Xray.

Buttons and actions

- **Add** (*Add*) — opens the "Add custom geo" form (*Add custom geo*). The save button is "Save" (*Save*). API: `POST /add`.
- **Edit** (*Edit*) — the "Edit custom geo" form (*Edit custom geo*). API: `POST /update/:id`. When the type or alias is changed, the old file is deleted and a new one is downloaded again.
- **Delete** (*Delete*) — confirmation *Delete this custom geo source?* Deletes the record from the database and the `.dat` file itself. API: `POST /delete/:id`. On success: *Custom geo file "" deleted*.
- **Update now** (*Update now*) — re-downloads a specific source and updates its timestamp. API: `POST /download/:id`. On success: *Geofile "" updated*.
- **Update all** — updates all custom sources at once. API: `POST /update-all`. On full success: *All custom geo sources updated*. If at least one source fails to update, the operation is considered partially unsuccessful with the message *One or more custom geo sources failed to update*, and the response lists the successful and unsuccessful sources.

After any of these actions (add, edit, delete, update, update all when there are successes), Xray-core is restarted.

Step by step: adding a source

1. Click "Add".
2. In the "Type" field select `geosite` or `geoip`.
3. In the "Alias" field enter an identifier (only lowercase Latin letters, digits, `-` and `_`; placeholder hint: `a-z 0-9 _ -`).
4. In the "URL" field specify a direct link to the `.dat` file (it must start with `http://` or `https://`).
5. Click "Save". The panel will immediately download the file into the `bin` directory, save the record, and restart Xray.

15.4. Validation and constraints

Strict checks are performed when a source is created and edited. Error messages:

Condition	Message
Type is not <code>geosite</code> / <code>geoip</code>	<i>Type must be geosite or geoip</i>
Empty alias	<i>Alias is required</i>
Invalid characters in the alias (not <code>^[a-z0-9_-]+\$</code>)	<i>Alias must match allowed characters</i>
Alias is reserved	<i>This alias is reserved</i>
Empty URL	<i>URL is required</i>
URL does not parse	<i>URL is invalid</i>
Scheme is not <code>http</code> / <code>https</code>	<i>URL must use http or https</i>
Empty/invalid host, or blocked by SSRF protection	<i>URL host is invalid</i>
Duplicate "type + alias"	<i>This alias is already used for this type</i>
Source not found	<i>Custom geo source not found</i>
Download error	<i>Download failed</i>

Hints in the form (client-side validation): *Alias may only contain lowercase letters, digits, - and _ and URL must start with `http://` or `https://`.*

Additional technical constraints:

- **Reserved aliases.** You cannot use aliases that conflict with the standard files. Reserved (comparison is case-insensitive, a hyphen is treated as an underscore): `geoip`, `geosite`, `geoip_ir`, `geosite_ir`, `geoip_ru`, `geosite_ru`. For example, `geosite-ru` will be rejected as `geosite_ru`.
- **SSRF protection.** The URL host is resolved into an IP, and if it points to a private/internal address, the download is blocked (the user sees *URL host is invalid*). This prevents using the panel to reach internal services.
- **Path traversal protection.** The final file path must reside inside the `bin` directory (with symlinks resolved); an attempt to escape it is rejected.

- **Minimum file size.** A downloaded file is considered valid only if it is at least 64 bytes; a file that is too small is rejected with a download error.
- **Proxy and conditional download.** If a proxy is configured in the panel settings, the download goes through it; otherwise a direct connection with an SSRF-safe transport is used. As with the standard files, `If-Modified-Since / 304 Not Modified` is applied (an unchanged file is not downloaded again). The download timeout is 10 minutes, and the URL availability probe (HEAD, and a partial GET when needed) is 12 seconds.

15.5. Auto-check at panel startup

At startup the panel iterates over all custom sources and, for each one, checks the presence and integrity of the local file (the file is missing, is a directory, or is smaller than 64 bytes). If the file is missing or corrupted, the source is probed and a re-download is attempted. This guarantees that after a reinstallation or loss of the `bin` directory the custom geo files will be restored automatically.

15.6. Using geo databases in routing rules

In Xray routing rules geo databases are used in fields such as `domain / ip` via prefixes:

- **geoip:** for IP databases — `geoip:<code>`. Examples: `geoip:ru`, `geoip:cn`, `geoip:private`. Taken from `geoip.dat` (or `geoip_RU.dat`, etc., if the rule points to a specific file).
- **geosite:** for domain databases — `geosite:<category>`. Examples: `geosite:category-ads-all`, `geosite:google`, `geosite:ru`. Taken from `geosite.dat`.

Example: blocking ads via geosite. A rule that sends all advertising domains into a black hole (assuming an outbound tagged `blocked` with the `blackhole` protocol):

```
{
  "type": "field",
  "domain": ["geosite:category-ads-all"],
  "outboundTag": "blocked"
}
```

For **custom** files the external-file syntax `ext:` is used. The hint in the UI: *In routing rules use the value column as `ext:file.dat:tag` (replace tag)*. Format:

```
ext:<file_name.dat>:<tag>
```

where `<file_name.dat>` is `geoip_<alias>.dat` or `geosite_<alias>.dat`, and `<tag>` is a specific list/category inside the file. In the "Routing (ext:...)" column the panel suggests a ready-made template like `ext:geosite_myads.dat:tag` — you just need to replace `tag` with the desired tag. The name of such a file is set in the "Geodata Auto-Update" section (see §15.3) in the "File name" field — for example `geosite_custom.dat`; rules reference it as `ext:geosite_custom.dat:category`.

Example: a rule based on a custom file. If a source of type `geosite` with the alias `myads` has been added, and inside the `.dat` file the list is labeled with the tag `ads`, the routing rule looks like this:

```
{
  "type": "field",
  "domain": ["ext:geosite_myads.dat:ads"],
  "outboundTag": "blocked"
}
```

For an IP source (type `geoip`, alias `mycorp`, tag `office`) the field would be `"ip": ["ext:geoip_mycorp.dat:office"]`.

16. Operations: Backups, Logs, Updates, CLI

This section covers the day-to-day maintenance of the panel: creating and restoring database backups, viewing the panel and Xray logs, restarting and stopping services, updating Xray and the panel itself, periodic tasks (cron), and uninstalling the panel. Some operations are performed from the web interface (tabs on the "Dashboard" and "Panel Settings" pages), and some from the `x-ui` console menu on the server.

16.1. Database Backup and Restore

All panel data (inbounds, clients, groups, nodes, settings) is stored in a single database. Backup management is available on the **"Dashboard"** page, on the **"Backup"** tab; the block title is **"Backup & Restore"**.

The panel supports two database engines, and the backup behavior depends on which one is used:

- **SQLite** (the default option) — data lives in the `x-ui.db` file.
- **PostgreSQL** — if the panel is configured for PostgreSQL, the block displays a hint:

"This panel runs on PostgreSQL. 'Backup' downloads a pg_dump archive (.dump), and 'Restore' uploads it back via pg_restore. The PostgreSQL client tools (pg_dump and pg_restore) must be installed on the server."

Export (creating a copy)

The **"Export Database"** button (English: Back Up) downloads the backup file to your device.

DB engine	File name	What happens on the server
SQLite	<code>x-ui.db</code>	First a WAL checkpoint is performed so that the file contains the latest records, then the file is read in full and served for download
PostgreSQL	<code>x-ui.dump</code>	<code>pg_dump</code> is run, and the archive is served for download

Hints in the interface:

- SQLite: "Click to download a .db file containing a backup of your current database to your device."
- PostgreSQL: "Click to download a PostgreSQL dump (.dump) of the current database to your device."

Technically, the export is a `GET /panel/api/server/getDb` request. The attachment name is set by the server (`Content-Disposition`) depending on the engine.

Example: downloading a backup via the API. The same export can be fetched from the console — for instance, for an automatic backup script. An authenticated session (login cookie) is required:

```
# 1) Log in and save the session cookie
curl -s -c cookies.txt \
  -d 'username=admin&password=admin' \
  https://panel.example.com:2053/panel/login

# 2) Download the database file (the name is set by the server: x-ui.db or x-ui.dump)
curl -s -b cookies.txt -OJ \
  https://panel.example.com:2053/panel/api/server/getDb
```

If the panel is served under a base path (Web Base Path), add it to the URL: `...:2053/<base_path>/panel/api/server/getDb`.

Import (restore)

The **"Import Database"** button (English: Restore) opens a file picker and uploads the file to the server for restoration (POST `/panel/api/server/importDB` , form field `db`).

Hints in the interface:

- SQLite: "Click to select and upload a .db file from your device to restore the database from a backup."
- PostgreSQL: "Click to select and upload a .dump file to restore the PostgreSQL database. This will replace all current data."

The import process for SQLite (it is important to understand that it is atomic and rolls back):

1. The uploaded file is checked for its format — it must be a valid SQLite database; otherwise an "Invalid db file format" error is returned.
2. The file is saved to a temporary `x-ui.db.temp` and undergoes an integrity check.
3. **Xray is stopped** before the database is replaced.
4. The current database is renamed to a backup `x-ui.db.backup` (fallback).
5. The temporary file is moved into place as the working database, schema initialization and migrations are performed, then the inbound migration.
6. **If any step fails** — a rollback is performed: the previous database is restored from `x-ui.db.backup` , and Xray is restarted on the old data.
7. On success the fallback file is deleted, and **Xray is automatically restarted** on the restored data.

Interface messages depending on the outcome:

Result	Text
Success	"Database imported successfully"
Import error	"An error occurred while importing the database"
File read error	"An error occurred while reading the database"

A restore completely replaces the current data. Since Xray is briefly stopped during the process, existing client connections are interrupted for the duration of the import.

Migration file between engines (SQLite ⇌ PostgreSQL)

Separately from the regular backup, there is a **"Download Migration"** function (`Download Migration`, the `GET /panel/api/server/getMigration` request). It produces a portable file for switching to a different DB engine:

Current engine	What is downloaded	File name	Purpose
SQLite	A portable SQL dump (text)	<code>x-ui.dump</code>	Seed PostgreSQL with your data
PostgreSQL	A SQLite database assembled from the PostgreSQL data	<code>x-ui.db</code>	Move the panel back to SQLite

Hints:

- On SQLite: "Click to download a portable .dump export (SQL text) of your SQLite database."
- On PostgreSQL: "Click to download a SQLite database (.db) assembled from your PostgreSQL data and ready to run the panel on SQLite."

The `.db ⇌ .dump` conversion for SQLite can also be performed from the CLI with the `x-ui migratedB [file]` command (see section 16.7).

Backup via the Telegram bot

If a Telegram bot is configured (see the section on notifications), it can send a backup copy directly to the administrator's chat. A backup via Telegram includes **two files**: the database itself (`x-ui.db`, or `x-ui.dump` on PostgreSQL) and the Xray configuration `config.json`. The message is preceded by the line "[?] Backup time:"

There are two ways to get a backup in Telegram:

1. **On request.** The  **DB Backup** button in the bot's menu – the bot immediately sends the files to the current chat.
2. **Automatically with a report.** The bot settings include a **"Database Backup"** toggle (`Database Backup`) with the description "Send a database backup file with a report". When it is enabled, on every periodic report distribution the bot sends a backup copy to all administrators after the report. The report distribution period is set by the bot's cron schedule (see section 16.6). The bot pauses between files and between administrators so as not to exceed Telegram's limits.

A backup via the bot is sent only if the bot is running; on PostgreSQL it also requires `pg_dump` to be present on the server.

16.2. Viewing Logs

The panel has two independent log viewers, both opened from the **"Logs"** tab on the Dashboard. Each window can refresh (the "refresh" icon in the header) and download what is displayed into an `x-ui.log` file (the button with the download icon).

Panel logs (application / syslog)

The panel log window (`POST /panel/api/server/logs/{count}`). Controls:

Control	Default value	Description
Number of lines	20	Drop-down list: 10 / 20 / 50 / 100 / 500
Level	Info	Minimum level: Debug / Info / Notice / Warning / Error
SysLog (checkbox)	off	Where to take logs from: the application buffer or the system journal

The behavior depends on the **SysLog** checkbox:

- **Off (default):** logs are taken from the panel's internal ring buffer, filtered by the selected level. Records are displayed with a level (DEBUG / INFO / NOTICE / WARNING / ERROR) and a source: X-UI: — messages from the panel itself, XRAY: — forwarded Xray messages.
- **On:** the panel runs `journalctl -u x-ui --no-pager -n <count> -p <level>` on the server, i.e., it shows the system journal of the `x-ui` service. The allowed number of lines is from 1 to 10000; the level accepts syslog values (`emerg/0` , `alert/1` , `crit/2` , `err/3` , `warning/4` , `notice/5` , `info/6` , `debug/7`). On Windows the SysLog mode is not supported — a warning will be shown that you need to clear the checkbox and use the application logs. If `systemd` /the service is unavailable, an error message about failing to run `journalctl` appears.

Example: the same journal from the server console. When the panel is unreachable (e.g., it fails to start), the system journal can be read directly — this is exactly the command the panel runs in SysLog mode:

```
# last 100 lines at the warning level and above
journalctl -u x-ui --no-pager -n 100 -p warning

# follow the journal in real time
journalctl -u x-ui -f
```

The level in this window filters the **output**. The minimum level that is written to the console/syslog at all is determined by the panel's logging level (an environment variable, `Info` by default; the panel always writes to the file at the `DEBUG` level).

Xray logs (access log)

A separate window for the Xray access log (`POST /panel/api/server/xraylogs/{count}`). It parses the lines of the Xray access log and displays them as a table: **Date, From, To, Inbound, Outbound, Email**.

Control	Default value	Description
Number of lines	20	10 / 20 / 50 / 100 / 500
Filter	empty	Substring text search (applied by pressing Enter)
Direct (checkbox)	on	Show direct connections (traffic through the freedom outbound)
Blocked (checkbox)	on	Show blocked connections (traffic to the blackhole outbound)
Proxy (checkbox)	on	Show proxied traffic

The event type is determined automatically by the tag of the outbound connection in the log line: a match with the freedom tags → "DIRECT" (green), blackhole → "BLOCKED" (red), everything else → "PROXY" (blue). Lines `api -> api` and empty lines are skipped.

For this window to show records, Xray must have its **access log** enabled with a file path (not `none`) – see below. If the access log is disabled or the file is unavailable, the window will be empty ("No Record...").

16.3. Xray Logging Level and Configuration

The logging parameters of Xray itself are set on the "**Xray Configurations**" page in the "**Log**" block (Log) with a warning:

"Logs can slow down the server. Enable only the log types you need, and only when necessary!"

Field	Translation	Default value	Description
Log Level (<code>logLevel</code>)	Log Level	<code>warning</code>	The level of detail for the Xray error log. Allowed values: <code>debug</code> , <code>info</code> , <code>notice</code> , <code>warning</code> , <code>error</code> . Hint: "The log level for error logs, indicating the information that needs to be recorded."
Access Log (<code>accessLog</code>)	Access Log	<code>none</code>	The path to the access log file. The special value <code>none</code> disables access logs. Hint: "The path to the access log file. The special value 'none' disables access logs."
Error Log (<code>errorLog</code>)	Error Log	empty (default path)	The path to the error log file; <code>none</code> disables them. Hint: "The path to the error log file. The special value 'none' disables error logs."
DNS Log (<code>dnsLog</code>)	DNS Log	<code>false</code> (off)	Enable logging of DNS queries. Hint: "Enable DNS query logs."
Mask Address (<code>maskAddress</code>)	Mask Address	empty (off)	When enabled, the real IP address is automatically replaced with a masking address in the logs. Hint: "When enabled, the real IP address is replaced with a masking address in the logs."

Since by default "**Access Log**" = `none` , the "Xray logs" window (section 16.2) is initially empty. To make it work, set an access log path here and restart Xray.

Note that an empty access log affects only this window. The online-client list on the Dashboard and the IP-count limit in the client form do **not** depend on the access log – the panel detects online clients and counts their IP addresses through the Xray core's online-stats API (connection statistics). On older core versions that lack this API, the panel automatically falls back to the previous method (reading the access log), in which case an access log path here is still required for the IP limit.

Example: a log block that makes the "Xray logs" window show records. In the Xray JSON configuration it looks like this:

```
{
  "log": {
    "loglevel": "warning",
    "access": "./access.log",
    "error": "",
    "dnsLog": false,
    "maskAddress": ""
  }
}
```

The key change is replacing `"access": "none"` with a file path (e.g., `"./access.log"`). After saving, restart Xray, and the table in the "Xray logs" window will fill with rows.

16.4. Managing Xray: Stopping and Restarting

The state of Xray is managed from the Xray card on the Dashboard. The current state is displayed as one of the following values: **Running** (Running), **Stopped** (Stopped), **Unknown** (Unknown), **Error** (Error). On error, a tooltip "Error while starting Xray" is available.

Button	Translation	Endpoint	Action
Stop	Stop	POST /panel/api/server/stopXrayService	Stops the Xray process. On success – a warning notification "Xray service has been stopped".
Restart	Restart	POST /panel/api/server/restartXrayService	Restarts (or starts) Xray, applying the current configuration. On success – the notification "Xray service has been restarted successfully".

After either operation, the panel broadcasts the new state over WebSocket, so the status on the Dashboard updates without reloading the page. If the operation fails, the Xray state becomes "Error", and the error text appears in the notification.

In addition to manual restarting, the panel itself checks whether Xray needs to be restarted (a background task every 30 s) and whether the process has crashed (a check every second) – see section 16.6.

16.5. Restarting and Updating the Panel

Restarting the panel

The **"Panel Settings"** page has a **"Restart Panel"** action (`Restart Panel` , `POST /panel/api/setting/restartPanel`). On confirmation, the panel restarts **after 3 seconds**.

Messages:

- Confirmation: "Are you sure you want to restart the panel? Confirm, and the restart will happen in 3 seconds. If the panel becomes unreachable, check the server log."
- Success: "Panel restarted successfully".

Technically, on Linux the restart is performed by sending the `SIGHUP` signal to the panel process (or via a registered hook). On Windows, sending `SIGHUP` is not supported.

Panel self-update (Update Panel)

The Dashboard offers an **"Update Panel"** function (`Update Panel`) – updating 3X-UI to the latest release directly from the web interface.

Before updating, the panel compares versions (`GET /panel/api/server/getPanelUpdateInfo`), requesting the latest 3x-ui release from GitHub:

Field	Translation
Current panel version	Current panel version
Latest panel version	Latest panel version
Panel is up to date / "Up to date"	Panel is up to date / Up to date – shown if there is no new version

Starting the update – `POST /panel/api/server/updatePanel` . The confirmation dialog:

- "Do you really want to update the panel?"
- "This will update 3X-UI to version #version# and restart the panel service."

After starting – a pop-up message "Panel update started" (`Panel update started`); on a failed version check – "Panel update check failed" (`Panel update check failed`).

What happens on the server: self-update is supported **only on Linux** (on other operating systems it returns the error "panel web update is supported only on Linux installations"). The panel downloads the official `update.sh` script from GitHub (`raw.githubusercontent.com/MHSanaei/3x-ui/main/update.sh`) and runs it in a separate process: preferably via `systemd-run` in a separate unit (`x-ui-web-update-<timestamp>`), and in the absence of `systemd` – as a separate detached process. Upon completion, the script updates the components and restarts the panel service. Running it requires `bash` .

If the script regenerated a new random web panel base path (Web Base Path) during the update, the `x-ui` service is restarted automatically so that the new path takes effect immediately. (Without the restart, the server

would keep serving the old path while the UI shows the new one, leaving the new address unreachable until a manual restart.)

On nodes, the panel of this same 3x-ui is updated centrally via `POST /panel/api/nodes/updatePanel` — see the section on nodes.

Updating and switching the Xray-core version

Here on the Dashboard you can also manage the Xray-core version separately from the panel.

- **Xray Updates** (`Xray Updates`) / **Version** (`Version`) — a drop-down list of available versions. Hints: "Select the desired version" and the warning "Important: older versions may not support the current settings".
- Installing/changing the version — `POST /panel/api/server/installXray/{version}`. Dialog: "Switch the Xray version" / "Are you sure you want to change the Xray version?". On success — "Xray updated successfully".

Example: switching the Xray-core version via an API request. The version is given as the release tag from XTLS/Xray-core (with a `v` prefix). For instance, to switch to `v1.8.24` :

```
curl -s -b cookies.txt -X POST \
  https://panel.example.com:2053/panel/api/server/installXray/v1.8.24
```

(`cookies.txt` is the cookie file from the example in section 16.1.) After installation, Xray restarts automatically on the selected version.

On the server, when the version is changed, Xray is first stopped, the archive of the desired version is downloaded from GitHub (XTLS/Xray-core), the binary is extracted and replaced, after which Xray is restarted with a check of the control sizes of the archive/binary.

16.6. Periodic Tasks (cron)

The panel registers a number of background tasks at startup. Their schedules are fixed (not configurable in the UI, except for the Telegram report schedule and the LDAP synchronization). Below are the tasks related to operations.

Task	Schedule	Purpose
Xray health check	every 1 s	Verifies that the Xray process is running
Check whether Xray needs a restart	every 30 s	Restart if the configuration is marked as changed
Xray traffic collection	every 5 s (starting 5 s after launch)	Accounting of inbound/client traffic
Client IP check	every 10 s	IP limit control based on the log
Heartbeat and node traffic synchronization	every 5 s	Exchange with nodes

Task	Schedule	Purpose
Log cleanup	<code>daily (@daily)</code>	Clears the IP-limit logs and the persistent access log, rotating the current log into <code>*.prev.log</code>
Periodic traffic reset	<code>@hourly , @daily , @weekly , @monthly</code>	Resets the traffic counters of those inbounds (and their clients) that have a corresponding auto-reset period set
Telegram report	set in the bot settings (<code>@daily</code> by default)	Distributing the report to administrators; with the option enabled – with a database backup attached (section 16.1)
Telegram hash-store reset	every 2 m	Only when the bot is enabled
CPU load monitoring for Telegram	every 10 s	Only if a CPU threshold > 0 is set

Additionally:

- **Periodic traffic reset** triggers only for those inbounds that have a corresponding auto-reset mode selected (hourly/daily/weekly/monthly). The task resets the traffic of the inbound itself and of all its clients.
- **Expiration and depletion checks.** Disabling clients on expiration and on traffic limit depletion is performed as part of traffic accounting: clients with an expired `expiry_time` or a depleted volume are flagged and disabled, and if necessary the next term is calculated (for cyclic limits and the "count from first use" mode). On the Dashboard and in the lists this is reflected by the statuses "Expired"/"Depleted"/"Depleting soon".
- **Automatic backup to Telegram** is a side effect of the report task; there is no separate cron schedule for the backup alone. Therefore, the frequency of the auto-backup equals the frequency of the bot's report.

16.7. Console Menu and CLI (`x-ui`)

On the server, the panel is managed with the `x-ui` command. With no arguments, the interactive "3X-UI Panel Management Script" menu opens; with an argument, a specific subcommand is executed. The menu items related to operations:

# in menu	Item	Action
1	Install	Installing the panel (downloads and runs <code>install.sh</code>)
2	Update	Updating all x-ui components to the latest version without data loss; afterward – an automatic restart
3	Update Menu	Updating only the <code>x-ui</code> menu script itself
4	Legacy Version	Installing a specified (older) version of the panel by the entered number (e.g., <code>2.4.0</code>)
5	Uninstall	Complete removal of the panel and Xray (see 16.8)
6	Reset Username & Password	Resetting the administrator login/password
7	Reset Web Base Path	Resetting the web panel's base path
8	Reset Settings	Resetting the settings to the default values

# in menu	Item	Action
9	Change Port	Changing the panel's port
10	View Current Settings	Viewing the current settings
11–13	Start / Stop / Restart	Starting, stopping, restarting the panel service
14	Restart Xray	Restarting Xray only
15	Check Status	The current status of the service
16	Logs Management	Viewing and clearing logs (see below)
17–18	Enable / Disable Autostart	Enabling/disabling autostart of the service at OS startup
25	Update Geo Files	Updating the geo files (GeoIP/GeoSite)
27	PostgreSQL Management	Managing PostgreSQL

Log management in the CLI (item 16)

In the "Logs Management" submenu:

- **Debug Log** — streaming view of the service journal: `journalctl -u x-ui -e --no-pager -f -p debug` (on Alpine — a `grep` over `/var/log/messages`).
- **Clear All logs** — clearing the system journal: `journalctl --rotate + journalctl --vacuum-time=1s`, after which the service is restarted. (Not available on Alpine.)

Direct `x-ui` subcommands

All available subcommands:

Command	Description
<code>x-ui</code>	Open the administration menu
<code>x-ui start</code>	Start the panel
<code>x-ui stop</code>	Stop the panel
<code>x-ui restart</code>	Restart the panel
<code>x-ui restart-xray</code>	Restart Xray
<code>x-ui status</code>	The current status
<code>x-ui settings</code>	Show the current settings
<code>x-ui enable</code>	Enable autostart at OS startup
<code>x-ui disable</code>	Disable autostart
<code>x-ui log</code>	View logs
<code>x-ui banlog</code>	View Fail2ban ban logs

Command	Description
<code>x-ui update</code>	Update the panel
<code>x-ui update-all-geofiles</code>	Update all geo files (followed by a restart)
<code>x-ui migrateDB [file]</code>	Convert the database <code>.db</code> $\Rightarrow$ <code>.dump</code> (SQLite)
<code>x-ui legacy</code>	Install a legacy version
<code>x-ui install</code>	Install the panel
<code>x-ui uninstall</code>	Uninstall the panel

The `x-ui update` command downloads and runs the official `update.sh` (the same one as the web update from section 16.5), asking for confirmation: "This function will update all x-ui components to the latest version, and the data will not be lost." Upon completion, the panel automatically restarts.

Retrieving an API token via the CLI

The CLI API-token command (the `x-ui` menu item/command) does not display a previously issued token. API tokens are stored only as hashes, so an existing token cannot be retrieved in plaintext. If tokens are already configured, the command reports how many there are, advises managing them in the panel (**Settings** $\rightarrow$ **API Tokens**, see the section on API tokens), and immediately generates a **new fallback token** with a name of the form `cli-fallback-<timestamp>` and prints it, so the CLI stays useful without opening the interface.

16.8. Uninstalling the Panel

Uninstallation is performed from the CLI – menu item **5 (Uninstall)** or the command `x-ui uninstall`. Before uninstalling, confirmation is requested (defaulting to "no"): "Are you sure you want to uninstall the panel? xray will also be uninstalled!".

On confirmation, the script:

1. Stops the service and disables its autostart (`systemctl stop/disable x-ui`, or on Alpine – `rc-service / rc-update`), removes the service unit file, and reloads the systemd configuration.
2. Removes the data and application directories (`/etc/x-ui/`, the installation directory) and the service env file (`/etc/default/x-ui`, `/etc/conf.d/x-ui`, or `/etc/sysconfig/x-ui` – depending on the distribution).
3. Removes the `x-ui` script itself and prints the message "Uninstalled Successfully.", as well as the command for reinstalling.

Uninstallation is irreversible: along with the panel, Xray and all data (including the database) are removed. If the data may be needed, export the database in advance (section 16.1).

16.9. The `x-ui migrateDB` Command

Starting with version 3.3.0, the management script `x-ui.sh` gained a `migrateDB` subcommand — a wrapper around the built-in `x-ui` binary (`x-ui migrate-db`) for converting the panel's SQLite database between two formats: the binary `.db` and the portable text dump `.dump` (plain SQL text).

What the command does

The command works in two directions, and the direction is determined **automatically** by the input file:

Direction	What it is called	What happens
<code>.db → .dump</code>	dump (export)	the binary SQLite database is exported to a text SQL file
<code>.dump → .db</code>	restore	a binary SQLite database is reassembled from the text SQL file

Under the hood, the script calls the panel binary:

- dump: `x-ui migrate-db --src <input> --dump <output>`
- restore: `x-ui migrate-db --restore <input> --out <output>`

Invocation syntax

```
x-ui migrateDB [file.db|file.dump] [output]
```

- **[file.db|file.dump]** — the input file (the first argument). If it is not specified, the panel's installed default database is used: `/etc/x-ui/x-ui.db`.
- **[output]** — the path to the output file (the second argument). Optional: if absent, the name is chosen automatically next to the input file (see below).

Examples:

```
x-ui migrateDB                               # dump /etc/x-ui/x-ui.db -> /etc/x-ui/x-
ui.dump
x-ui migrateDB /etc/x-ui/x-ui.db backup.dump
x-ui migrateDB backup.dump restored.db       # assemble a .db from the dump
```

How the direction is determined

The script looks at the extension of the input file:

- `*.db`, `*.sqlite`, `*.sqlite3` → **dump** mode (export to text);
- `*.dump`, `*.sql` → **restore** mode (assemble a database).

If the extension is not recognized, the script reads the first 16 bytes of the file: the signature `SQLite format 3` means a binary database (dump mode), otherwise the file is considered a dump (restore mode).

The output file name, if the second argument is not specified:

- for a dump — the same name as the input, with the `.dump` extension;
- for a restore — the same name with the `.db` extension.

Safety checks and behavior

- **Binary presence.** If the `x-ui` binary is not found or is not executable — the error "x-ui binary not found ... Is the panel installed?" is printed.
- **Feature support in the build.** The script checks that the binary supports `migrate-db --dump/--restore` (via `x-ui migrate-db -h`). If not — it suggests first updating the panel with the `x-ui update` command.
- **Existence of the input file.** If the input file is absent, an error and the invocation syntax line are printed.
- **Overwriting the output.** If the output file already exists, confirmation is requested (defaulting to "no"); without confirmation the operation is canceled. On a restore, the old output file is deleted beforehand.
- **Protection of the "live" database.** When restoring into the default database `/etc/x-ui/x-ui.db` while the panel is running, the operation is rejected with a requirement to first stop the panel (`x-ui stop`) or choose a different output path. This prevents overwriting the working database of a running service.
- On a failed database assembly, the unfinished output file is deleted.

Why this is needed

- **Backup.** A text `.dump` is human-readable, convenient for storage in version control systems and for differential viewing of the database contents.
- **Migration.** A dump is portable between machines and resilient to differences in the SQLite file format version — on a new server, a working `.db` is assembled from it.
- **Diagnostics.** From a `.dump` you can look at the structure and data of the panel by eye, without having SQLite tools at hand.

Interactive mode

In addition to the direct invocation, the conversion is available from the interactive menu. In the PostgreSQL submenu (`x-ui` → the PostgreSQL section) there is an item **9. Convert SQLite `.db` <-> `.dump`**: it asks for the path to the input file (default `/etc/x-ui/x-ui.db`) and to the output (it can be left empty for auto-naming), and the direction, as in the CLI mode, is determined automatically.

This manual was prepared from the 3X-UI source code. If something in your version's UI differs, the panel's behavior and the in-UI hints take precedence.